

Práctica 1.1: Repaso de aspectos sobre sistemas operativos y programación en C. Manejo de hilos.

Índice

1	Objetivos	1
2	Ejercicios	1
	Ejercicio 1: Introducción a GNU Make y Proyectos con Makefile	1
	Ejercicio 2: Escritura y lectura de cadenas de caracteres en ficheros	3
	Ejercicio 3 La función <code>getopt()</code> de la biblioteca estándar de “C”	3
	Ejercicio 4: Creación de procesos y ejecución de programas	5
	Ejercicio 5: Creación y paso de parámetros a hilos.	6

1 Objetivos

El principal objetivo de esta práctica es reforzar aspectos de programación en C sobre Linux, haciendo hincapié en algunas cuestiones esenciales de la biblioteca estándar de “C” y llamadas disponibles en el sistema GNU/Linux. En particular se incide en lo siguiente:

- El uso de la función `getopt()` de la biblioteca estándar de “C” para procesamiento de opciones de línea de comando
- La utilización de las llamadas al sistema de Linux para acceso a ficheros
- La creación de procesos y ejecución de programas mediante las llamadas `fork()` y `exec*()`
- La creación y manejo básico de hilos con la biblioteca POSIX Threads

Asimismo esta práctica servirá para familiarizarse con los *proyectos C con Makefile*, que se utilizarán en el laboratorio para simplificar la compilación de las prácticas. El primer ejercicio de esta práctica introduce el concepto de este tipo de proyecto así como el uso básico de la herramienta GNU make.

El archivo `ficheros_p1-1.tar.gz` contiene una serie de proyectos C, que o bien contienen el código de ejemplo de alguno de los ejercicios, o han de emplearse como punto de partida para el desarrollo de ejercicios de esta práctica.

2 Ejercicios

Ejercicio 1: Introducción a GNU Make y Proyectos con Makefile

Es frecuente durante el desarrollo del proyecto que haya que compilar dicho proyecto multitud de veces. Además, el proyecto va cambiando, añadiéndose ficheros, bibliotecas con las que se debe enlazar, ejecutables y bibliotecas que se deben generar, etc. Repetir en cada ocasión las llamadas al compilador `gcc` para construir cada objetivo del proyecto (p.ej., biblioteca, fichero objeto o ejecutable) es ineficiente y propenso a errores. La popular herramienta Make, diseñada

para entornos UNIX, tiene como objetivo facilitar y, en cierta medida, automatizar este proceso de compilación del proyecto.

Hay distintas versiones de la herramienta Make, sin embargo todas ellas funcionan de un modo muy parecido. Cuando se invoca, Make interpreta un fichero de texto con una sintaxis especial y nombre preestablecido (p.ej. *makefile*), que le indica los pasos que debe seguir para compilar el proyecto. Una vez confeccionado el *makefile* repetir la compilación es tan sencillo como ejecutar `make` desde el terminal.

En esta asignatura emplearemos la implementación de Make de GNU, que se encuentra instalada en el laboratorio. Cuando el usuario invoca el comando `make` desde el *shell*, la herramienta busca en primer lugar un fichero que se llama `GNUmakefile`. Si no se encuentra se busca un fichero llamado `makefile`, y si por último no se encontrase, se buscaría el fichero `Makefile`. Si no se encuentra en el directorio actual ninguno de esos tres ficheros, se producirá un error y `make` no continuará:

```
$ make
make: *** No targets specified and no makefile found.  Stop.
```

En caso contrario, `make` procesará el archivo de texto e invocará el conjunto mínimo de reglas requeridas para construir nuestro proyecto.

Para ilustrar el flujo de trabajo de `make`, consideremos el código de ejemplo que se encuentra en el directorio *ejercicio1*, que constituye un proyecto en C con *Makefile*. Este tipo de proyectos están formados por el código fuente C del proyecto (ficheros `.c` y `.h`) más un fichero *Makefile*, que contiene las reglas necesarias para construir el objetivo (*target*) del proyecto. En este caso particular el proyecto contiene un único fichero `.c` (*ejemplo.c*) –con el código de un programa muy simple que acepta dos argumentos obligatorio–, y un *Makefile* con las reglas necesarias para generar el ejecutable del programa y limpiar los ficheros resultantes de la compilación.

Normalmente, las reglas del fichero *Makefile* garantizan que si solo se ha modificado un archivo C desde que se invocó `make` por última vez, una nueva invocación de `make` realizará la compilación solo para el archivo C modificado y la etapa final de enlazado.

Para familiarizarte con GNU Make, abre una ventana de terminal en Linux, y realiza las siguientes acciones desde el directorio *ejercicio1* (usar `cd <ruta_ejercicio_1>`), respondiendo a las preguntas que se plantean:

- Ejecuta `make` en la línea de comandos y comprueba las órdenes que ejecuta para construir el proyecto. ¿qué comando(s) se invoca(n)? ¿Se ha generado algún fichero ejecutable como resultado? Ejecuta `ls` para ver el contenido del directorio.
- Tras construir el proyecto, ejecuta los siguientes comandos, e indica la salida generada por pantalla. ¿A que se debe este comportamiento? Consulta el código fuente usando un editor de textos.
 - `./example`
 - `./example John`
 - `./example John Smith`
- Vuelve a ejecutar el comando `make` en más ocasiones. ¿Se vuelve a generar el fichero ejecutable invocando al compilador `gcc`? ¿Por qué?
 - Marca el fichero *example.c* como modificado ejecutando `touch example.c`. Después ejecuta de nuevo `make`. ¿Qué diferencia hay con la última vez que lo ejecutaste? ¿Por qué?
- Ejecuta la orden `make clean`. ¿Qué ha sucedido?

En este curso, la mayor parte de los proyectos C de ejemplo proporcionados con las distintas prácticas se acompañan de un fichero *Makefile* específico para el proyecto. Por lo general, para compilar el proyecto asociado bastará teclear `make` en la terminal. Cabe destacar que durante el desarrollo de la asignatura los estudiantes no tendrán que elaborar ficheros *Makefile* de cero, sino adaptar mínimamente ejemplos sencillos ya proporcionados. Por lo tanto en este documento no procederemos a introducir la sintaxis de GNU `make`. No obstante, esta sintaxis está descrita en el tutorial disponible en este enlace.

Ejercicio 2: Escritura y lectura de cadenas de caracteres en ficheros

En este ejercicio se han de desarrollar dos programas sencillos `write_strings.c` y `read_strings.c` que permitan respectivamente escribir y leer de un fichero un conjunto de cadenas de caracteres de longitud variable terminadas por `'\0'`. Dicho carácter terminador deberá almacenarse en el fichero con el resto de caracteres de cada cadena. Para el desarrollo de los dos programas se utilizará la función de la biblioteca estandar `malloc()`, así como las siguientes llamadas al sistema de Linux: `open()`, `close()`, `read()`, `write()`, `seek()` y `malloc()`

El programa `write-strings.c` aceptará como primer parámetro el nombre de un fichero de texto donde se escribirán los strings pasados a continuación a la línea de comandos (argumento 2, argumento 3, etc.) - véase ejemplo de ejecución más abajo. Si el fichero destino existe, el programa reescribirá su contenido.

El programa `read-strings.c` aceptará como parámetro el nombre del fichero de texto donde se almacenen las cadenas de caracteres terminadas en `'\0'`. Este programa leerá las cadenas y las imprimirá por pantalla separadas por un salto de línea, como se muestra en el siguiente ejemplo de ejecución:

```
## Write strings to file
usuario@debian:~/exercise2$ ./write_strings out London Paris Madrid Barcelona Berlin Lisbon

## Check whether file structure is correct (null-terminated strings)
usuario@debian:~/exercise2$ $ xxd out
00000000: 4c6f 6e64 6f6e 0050 6172 6973 004d 6164  London.Paris.Mad
00000010: 7269 6400 4261 7263 656c 6f6e 6100 4265  rid.Barcelona.Be
00000020: 726c 696e 004c 6973 626f 6e00          rlin.Lisbon.

## Read strings from file
usuario@debian:~/exercise2$ ./read_strings out
London
Paris
Madrid
Barcelona
Berlin
Lisbon
```

Por simplicidad para la implementación del programa `read-strings.c`, se ha de desarrollar una función auxiliar `char* loadstr(int fd)`. Esta función lee una cadena de caracteres terminada en `'\0'` del fichero cuyo descriptor se pasa como parámetro, reservando dinámicamente la cantidad de memoria adecuada para la cadena leída y retornando dicha cadena. La función tendrá que averiguar primero el número de caracteres de la cadena que comienza a partir de la ubicación actual del puntero de posición del fichero, leyendo carácter a carácter. Una vez detectado el carácter terminador, restaurará el indicador de posición del fichero (moviéndolo hacia atrás) y, finalmente realizará una lectura de la cadena completa.

Ejercicio 3 La función `getopt()` de la biblioteca estándar de “C”

En este ejercicio, trabajaremos el uso de `getopt()` una herramienta esencial para el procesamiento de opciones en línea de comando. El objetivo del ejercicio es completar el código del fichero `getopt.c` para que sea capaz de procesar las opciones `-e` y `-l` tal y como indica el uso del programa, que puede consultarse con la opción `-h`:

```
$ make
$ ./getopt -h
Usage: ./getopt [ options ] title

options:
-h: display this help message
-e: print even numbers instead of odd (default)
-l length: length of the sequence to be printed
title: name of the sequence to be printed
```

Una vez completado, el programa deberá imprimir una secuencia de `length` números (10 por defecto; podemos cambiarlo con la opción `-l`) impares (por defecto) o pares si se incluye la opción `-e`. Los argumentos `-l` `length` y `-e` son opcionales, pero el argumento `title` siempre debe estar presente en la línea de comando.

Ejemplos de salidas para diferentes combinaciones de entrada:

```
$ ./getopt hola
Title: hola
1 3 5 7 9 11 13 15 17 19

./getopt -l 3 hola1
Title: hola1
1 3 5

./getopt -l 4 -e hola2
Title: hola2
2 4 6 8
```

Para la realización de este ejercicio es necesario familiarizarse con la función `getopt()` consultando la página de manual de `getopt()`: `man 3 getopt`

- `int getopt(int argc, char *const argv[], const char *optstring);`

La función suele invocarse desde `main()`, y sus dos primeros parámetros coinciden con los argumentos `argc` y `argv` pasados a `main()`. El parámetro `optstring` sirve para indicar de forma compacta a `getopt()` cuáles son las opciones que el programa acepta –cada una identificada por una letra–, y si éstas a su vez aceptan parámetros obligatorios u opcionales.

Deben tenerse en cuenta las siguientes consideraciones:

1. La función `getopt()` se usa en combinación con un bucle, que invoca tantas veces la función como opciones ha pasado el usuario en la línea de comandos. Cada vez que la función se invoca y encuentra una opción, `getopt()` retorna el caracter correspondiente a dicha opción. Por lo tanto, dentro del bucle suele emplearse la construcción *switch-case* de C para llevar a cabo la identificación de las distintas opciones. Es aconsejable no realizar el procesamiento de nuestro programa dentro del bucle, sino únicamente identificar las opciones que pasa el usuario y dar valor a variables/flags que serán utilizadas en el resto de nuestro código para decidir el comportamiento que debe tener.
2. Un aspecto particular de la función `getopt()` es que establece el valor de distintas variables globales tras invocarse, siendo las más relevantes las siguientes:
 - `char* optarg`: almacena el argumento pasado a la opción actual reconocida, si ésta acepta argumentos. Si la opción no incluye un argumento, entonces `optarg` se establece a `NULL`
 - `int optind`: representa el índice del siguiente elemento en el `argv` (elementos que quedan sin procesar). Se usa frecuentemente para procesar argumentos adicionales del programa que no están asociados a ninguna opción. Veremos un ejemplo de ello en la práctica.

Para completar el código, incluye las opciones `-l` y `-e` en la llamada a `getopt()` y completa la estructura *switch-case* para modificar los valores por defecto de la variable `options`. Para leer el valor numérico asociado a la opción `-l`, deberás utilizar la variable global `optarg`, teniendo en cuenta que esta variable es una cadena de caracteres (tipo `char*`) y, sin embargo, queremos almacenar la opción como un número entero (tipo `int`). Consulta el uso de la función `strtol()` en el manual (`man 3 strtol`) para saber cómo realizar esa conversión.

Asimismo, dado que el argumento `title` no será procesado por `getopt()` (pues no está precedido por una marca de opción al estilo `-l`), deberemos continuar el procesamiento de la cadena de entrada tras el bucle `for`. Para ello, se usará la variable `optind` junto con `argv` para almacenar el valor de la cadena de caracteres que será el título de nuestra secuencia.

Completa el código y responde a las siguientes preguntas:

1. ¿Qué cadena de caracteres debes utilizar como tercer argumento de `getopt()`?
2. ¿Qué línea de código utilizas para leer el argumento `title`?

Ejercicio 4: Creación de procesos y ejecución de programas

Desarrollar un programa `run_commands` que ejecute comandos especificados por el usuario, y espere a su terminación. Se proporciona un esqueleto de código con un *Makefile*, así como dos ficheros de entrada para comprobar el correcto funcionamiento del programa a desarrollar.

En el fichero `run_commands.c` proporcionado se incluye un código de prueba, que habrá que modificar para desarrollar la funcionalidad deseada. Este programa de partida acepta un único argumento donde se especifica un comando. El programa analiza dicho comando, y construye un array de cadenas de caracteres acabadas en `NULL` (formato de `argv`), que posteriormente se imprime por pantalla, liberando adecuadamente la memoria reservada con `malloc()`. El propósito de este programa es ilustrar el uso de la función `parse_command()` proporcionada, que deberá estudiarse y reusarse en la implementación del ejercicio.

El programa `run_commands` a desarrollar reconocerá un conjunto de opciones en la línea de comandos, que se deben procesar usando la función `getopt()`, empleada en prácticas previas. A continuación se presentan las opciones que aceptará el programa, que se recomienda implementar y probar en el orden en el que se describen:

- `-x <comando>`: Cuando el programa se invoque con esta opción, se creará un proceso hijo que ejecutará dicho comando. Para ello, el programa principal invocará la función `launch_command()` –a implementar–, que creará un proceso hijo con `fork()`, y hará que este ejecute el comando pasado como parámetro usando `execvp()`. La función devolverá el PID del proceso hijo, sin esperar a que termine su ejecución. El programa principal se encargará de esperar la terminación del proceso hijo creado. La función `launch_command()` tendrá el siguiente prototipo:

```
pid_t launch_command(char** argv);
```

- `-s <fichero>`: Esta opción permitirá al usuario indicar como argumento la ruta de un fichero con comandos a ejecutar. Este fichero será interpretado por líneas, tomando cada línea como un comando a ejecutar con la función `launch_command()`. Los comandos se ejecutarán de forma secuencial, esperando a que un comando termine antes de ejecutar el siguiente. **Sugerencia:** usar `fgets()` para leer del fichero por líneas. Consultar `man 3 fgets`

Ejemplo de ejecución

```
# Testing -x switch
$ ./run_commands -x ls
Makefile      run_commands  run_commands.c  run_commands.o  test1          test2

$ ./run_commands -x "echo hello"
Hello

# Testing -s switch
$ ./run_commands -s test1
@@ Running command #0: echo hello
hello
@@ Command #0 terminated (pid: 1439, status: 0)

@@ Running command #1: sleep 2
@@ Command #1 terminated (pid: 1440, status: 0)
@@ Running command #2: ls -l
total 88
-rw-r--r--@ 1 usuario  usuario  267 Oct 20 11:34 Makefile
```

```

-rwxr-xr-x  1 usuario  usuario  9960 Oct 20 11:58 run_commands
-rw-r--r--  1 usuario  usuario  4332 Oct 20 11:57 run_commands.c
-rw-r--r--  1 usuario  usuario  8984 Oct 20 11:58 run_commands.o
-rw-r--r--@ 1 usuario  usuario   31 Oct 20 11:34 test1
-rw-r--r--@ 1 usuario  usuario   41 Oct 20 11:46 test2
@@ Command #2 terminated (pid: 1443, status: 0)
@@ Running command #3: false
@@ Command #3 terminated (pid: 1444, status: 256)

```

Una vez acabado el desarrollo del programa `run_commands`, responde a las siguientes preguntas:

1. Al usar la opción `-x` del programa, el comando indicado como argumento se pasa encerrado entre comillas dobles en el caso de que este, a su vez, acepte argumentos, como por ejemplo `ls -l`. ¿Qué ocurre si el argumento de `-x` no se pasa entrecomillado? ¿Funciona correctamente el lanzamiento del programa `ls -l` si se encierra entre comillas simples en lugar de dobles? **Nota:** Para ver las diferencias prueba a ejecutar el siguiente comando: `echo $HOME`
2. ¿Es posible utilizar `execlp()` en lugar de `execvp()` para ejecutar el comando pasado como parámetro a la función `launch_command()` ? En caso afirmativo, indica las posibles limitaciones derivadas del uso de `execlp()` en este contexto.
3. ¿Qué ocurre al ejecutar el comando `"echo hello > a.txt"` con `./run_commands -x` ? ¿y con el comando `"cat run_commands.c | grep int"` ? En caso de que los comandos no se ejecuten correctamente indica el motivo.

Ejercicio 5: Creación y paso de parámetros a hilos.

En este ejercicio vamos a usar la biblioteca de `pthread`, por lo que será necesario compilar y enlazar con la opción `-pthread`.

Escribir un programa *hilos.c* que va a crear hilos cuya funcionalidad vendrá determinada por los argumentos que se le pasen en la creación. Los hilos recibirán como argumentos el puntero a una estructura que contenga dos campos: un entero, que será el número de hilo, y un caracter, que indicará si el hilo es prioritario (P) o no (N).

El programa deberá crear una variable para el argumento de cada hilo usando memoria dinámica, inicializar dicha variable con el número de hilo y su prioridad (los pares serán prioritarios y los impares no lo serán), crear los hilos y esperar a que finalicen.

Cada hilo copiará sus argumentos en variables locales, liberará la memoria dinámica reservada para los mismos, averiguará cuál es su identificador e imprimirá un mensaje con su identificador, el número de hilo y su prioridad.

El alumno debe consultar las páginas de manual de: `pthread_create`, `pthread_join`, `pthread_self`.

Probar a crear solamente una variable para el argumento de todos los hilos, dándole el valor correspondiente a cada hilo antes de la llamada a `pthread_create`. Explicar qué sucede y cuál es la razón.

Práctica 1.2: Sincronización entre hilos

Índice

Objetivos	1
Ejercicios	1
Ejercicio 1: Problema de los filósofos comensales	1
Ejercicio 2: Inserción alternada en un vector con múltiples hilos	1
Ejercicio 3: Productor-consumidor con acceso a ficheros	2

Objetivos

El principal objetivo de esta práctica es familiarizarse con los mecanismos de sincronización para hilos disponibles en Linux. Más concretamente, en los distintos ejercicios propuestos se hará uso de cerrojos, variables condicionales y semáforos sin nombre.

Ejercicios

Ejercicio 1: Problema de los filósofos comensales

Implementar dos soluciones al problema de los filósofos comensales, la primera usando mutexes y la segunda semáforos. La implementación con cada mecanismo debe incluirse en dos programas C independientes *–philosophers_mtx.c* y *philosophers_sem.c–*, cada uno con su propia función `main()`. Nótese que se proporciona un fichero *philosophers.c* como material suplementario. Este archivo debe utilizarse como punto de partida para construir ambas soluciones.

En este problema, cada filósofo se emula mediante un hilo independiente. Por lo tanto, el programa contará con 5 hilos, creados con `pthread_create()`. Para simplificar la implementación, se asignará un número (0..4) a cada tenedor y a cada filósofo.

Cada filósofo piensa y come de forma repetitiva. En concreto, la acción de *pensar* se simula mediante la función `sleep()` o `usleep()` (consulta la página de manual asociada para más detalles) con un periodo de tiempo aleatorio. El filósofo debe coger su tenedor izquierdo y derecho (no necesariamente en ese orden para evitar bloqueos...). Después de comer, el filósofo volverá a dejar los tenedores sobre la mesa, y luego dormirá durante algún tiempo antes de volver a pensar. Al implementar el programa, coger un tenedor se llevará a cabo adquiriendo un mutex o invocando la operación `wait()` en un semáforo, dependiendo del tipo de solución empleada. Del mismo modo, devolver un tenedor a la mesa se reduce a liberar un mutex o invocar la operación `signal()` de un semáforo.

Ejercicio 2: Inserción alternada en un vector con múltiples hilos

En este ejercicio se desarrollarán dos variantes de un programa multihilo con las siguientes características comunes:

- El programa principal creará un vector de enteros de tamaño especificado por el usuario (con la opción `-n <tamaño>`) reservando adecuadamente la memoria con `malloc()`. Este vector se declarará como variable global del programa.
- A continuación el programa creará una serie de hilos con `pthread_create()` y esperará a su terminación. Cuando los hilos hayan terminado el programa principal imprimirá por pantalla el contenido final del vector, con los valores separados por comas.
- Cada hilo creado realizará la inserción alternada de un valor concreto en el vector. La sincronización de la inserción alternada se implementará con semáforos POSIX sin nombre.

Parte A. Desarrollar un programa con las características arriba mencionadas y que cree 2 hilos. Estos dos hilos insertarán los numeros 1..N de forma alternada en el vector compartido. En particular, el hilo 1 insertará los números impares, y el hilo 2 los números pares. De esta forma, la impresión por pantalla del vector será como se indica en el ejemplo de ejecución:

```
$ ./alternateA -n 10
1,2,3,4,5,6,7,8,9,10
```

Parte A. Desarrollar un programa con las características arriba mencionadas y que cree 3 hilos. Estos tres hilos insertarán los numeros 1..N de forma alternada en el vector compartido. El orden de inserción será el siguiente: hilo 1, hilo 2, hilo 3, hilo 1, hilo 2, hilo 3, y así sucesivamente. La salida por pantalla deberá ser la misma que en el ejemplo anterior, alterándose solamente el mecanismo de inserción de los números en el vector en el código de los distintos hilos.

Ejercicio 3: Productor-consumidor con acceso a ficheros

Desarrollar un programa `pc-files` con el siguiente modo de uso:

```
$ ./pc-files [ -i fichero_entrada ] [ -o fichero_salida ]
```

Nótese que ninguna de las opciones es obligatoria. En el caso de que se no se especifique la opción `-i`, el fichero de entrada por defecto considerado será la entrada estándar (ya abierto y con descriptor ya predefinido `stdin` en la biblioteca estándar). En caso de no pasar la opción `-o` en la línea de comando, el fichero de salida predefinido será `out.txt`. Si el fichero de salida especificado ya existe su contenido se truncará; en caso contrario, se creará dicho fichero.

El programa `pc-files` creará dos hilos POSIX, que se comportarán respectivamente como productor y consumidor. Ambos hilos se comunicarán usando un buffer compartido (global) de cadenas de caracteres con capacidad para 4 cadenas:

```
#define MAX_SBUFFER_SIZE 4
char* shared_buffer[MAX_SBUFFER_SIZE];
```

El productor leerá cadenas de caracteres terminadas en `'\\n'` del fichero de entrada, y enviará cada cadena al consumidor usando el buffer compartido, así hasta detectar fin de fichero. La inserción se realizará añadiendo al buffer el puntero al comienzo de la cadena, cuya memoria se reservará con `malloc()` desde el productor. Para indicar al consumidor que no hay más cadenas que procesar, el productor insertará `NULL` en el buffer compartido.

El consumidor extraerá cadenas de este buffer compartido, las imprimirá en un fichero de salida en orden de recepción y liberará la memoria con `free()`. El hilo consumidor terminará al extraer la cadena `NULL` del buffer. El programa principal terminará cuando ambos hilos hayan acabado. Para la lectura por líneas del fichero se recomienda el uso de la función `fgets()` de la biblioteca estándar (consultar su página de manual).

Para implementar las restricciones de sincronización típicas del productor consumidor ha de usarse un mutex, variables condicionales, y otras variables compartidas necesarias para el correcto acceso al buffer compartido.

Ejemplo de ejecución:

```
## Creación de fichero de entrada manualmente
osuser@debian:~/ejercicio3$ echo London > in.txt
osuser@debian:~/ejercicio3$ echo Barcelona >> in.txt
osuser@debian:~/ejercicio3$ echo Madrid >> in.txt
osuser@debian:~/ejercicio3$ echo Berlin >> in.txt
osuser@debian:~/ejercicio3$ echo Lisbon >> in.txt
osuser@debian:~/ejercicio3$ echo Paris >> in.txt
osuser@debian:~/ejercicio3$ cat in.txt
London
Barcelona
Madrid
Berlin
Lisbon
Paris

## Ejecución
osuser@debian:~/ejercicio3$ ./pc-files -i in.txt
osuser@debian:~/ejercicio3$ cat out.txt
London
Barcelona
Madrid
Berlin
Lisbon
Paris
osuser@debian:~/ejercicio3$
```

Práctica 1.3: Comunicación y sincronización entre procesos

Índice

Objetivos	1
Ejercicios	1
Ejercicio 1: El problema del productor-consumidor con procesos, semáforos y memoria compartida	1
Ejercicio 2: Programa <code>fifotest</code>	1
Ejercicio 3: Desarrollo de un chat multihilo con FIFOs	3

Objetivos

Esta práctica persigue que los estudiantes se familiaricen con los dos principales mecanismos de comunicación entre procesos disponibles en Linux: regiones de memoria compartida y tuberías. Además de utilizar estos dos mecanismos (en su variante con nombre) se hará uso de semáforos POSIX con nombre para establecer sincronización entre procesos.

Ejercicios

Ejercicio 1: El problema del productor-consumidor con procesos, semáforos y memoria compartida

En esta práctica se realizará la modificación de una variante de la implementación del problema productor-consumidor analizada en clase que emplea semáforos con nombre y memoria compartida. Como punto de partida se proporciona el código de dicha solución, que constituye una versión más robusta de la implementación disponible en las transparencias. Esencialmente, después de invocar llamadas al sistema esenciales, esta implementación consulta los valores de retorno de las llamadas para comprobar la existencia de errores y, en su caso, llevar a cabo el tratamiento correspondiente.

En este ejercicio se han de realizar los siguientes cambios en la implementación proporcionada:

1. Incluir la sincronización necesaria en el código para que el productor no comience a insertar números en el buffer circular compartido hasta que el proceso consumidor haya iniciado su ejecución y esté preparado para recibir los números (es decir, cuando haya invocado la función `Consumer()`). **Sugerencia:** Se puede utilizar un semáforo adicional inicializado por el productor, cuyo contador se inicializa a cero.
2. Modificar el mecanismo de comunicación de información entre el productor y el consumidor: en lugar de utilizar un fichero real para respaldar la región de memoria compartida (es decir, un fichero proyectado en memoria), se debe hacer uso de un objeto de memoria compartida POSIX, que se creará y al que se accederá mediante la llamada al sistema `shm_open()`. Consultar la página de manual asociada para más detalles y ejemplos sobre este tipo de zonas de memoria compartida: `man shm_open`

Ejercicio 2: Programa `fifotest`

En este ejercicio se ha de estudiar la implementación y el comportamiento del programa `fifotest` introducido en clase. Este programa tiene dos modos de uso que corresponden a las opciones `-s` (*send*) y `-r` (*receive*) de la línea de

comando. Para poder usar este programa es necesario lanzar dos instancias del mismo, cada una en un terminal diferente. Los dos procesos (emisor y receptor) se comunicaran entre sí a través de un fichero FIFO creado previamente, cuya ruta se ha de pasar con la opción -f del programa.

En el ejemplo mostrado a continuación se ilustra cómo utilizar este programa de forma interactiva, ejecutando las instancias correspondientes de envío y recepción desde distintos terminales:

Terminal 1:

```
## Creamos fichero FIFO
osuser@debian:~/ejercicio2$ mkfifo myfifo

## Lanzamos instancia de envío, e insertamos cadenas de caracteres para enviar al "consumidor"
## Teclear CTRL + D para introducir el fin de fichero en la entrada estándar y acabar la comunicación
osuser@debian:~/ejercicio2$ ./fifotest -f myfifo -s
Hello
How ya doin
...
```

Terminal 2:

```
## Lanzamos instancia de recepción, y recibiremos las cadenas desde el otro extremo
osuser@debian:~/ejercicio2$ ./fifotest -f myfifo -r
Hello
How ya doin
...
```

Este programa puede ejecutarse para realizar envío de forma no interactiva, leyendo datos de un fichero de entrada en lugar de la información introducida con el teclado. Para usar este modo basta utilizar la redirección de entrada del shell como se muestra en el siguiente ejemplo, donde usaremos un fichero de texto ya existente en el directorio:

Terminal 1:

```
## Suponemos fichero fifo ya creado
## Lanzamos instancia de envío usando fichero existente como entrada al programa
osuser@debian:~/ejercicio2$ ./fifotest -f myfifo -s < test.txt
osuser@debian:~/ejercicio2$
```

Terminal 2:

```
## Lanzamos instancia de recepción, y recibiremos el contenido del fichero por el otro extremo
osuser@debian:~/ejercicio2$ ./fifotest -f myfifo -r
En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que
vivía un hidalgo de los de lanza en astillero, adarga antigua, rocín flaco y galgo
corredor. Una olla de algo más vaca que carnero, salpicón las más noches, duelos y
quebrantos los sábados, lentejas los viernes, algún palomino de añadidura los domingos,
consumían las tres partes de su hacienda. El resto della concluían sayo de velarte,
calzas de velludo para las fiestas con sus pantuflos de lo mismo, los días de entre
semana se honraba con su vellori de lo más fino. Tenía en su casa una ama que pasaba de
los cuarenta, y una sobrina que no llegaba a los veinte, y un mozo de campo y plaza, que
así ensillaba el rocín como tomaba la podadera. Frisaba la edad de nuestro hidalgo con
los cincuenta años, era de complexión recia, seco de carnes, enjuto de rostro; gran
madrugador y amigo de la caza. Quieren decir que tenía el sobrenombre de Quijada o
Quesada (que en esto hay alguna diferencia en los autores que deste caso escriben),
aunque por conjeturas verosímiles se deja entender que se llama Quijana; pero esto
importa poco a nuestro cuento; basta que en la narración dél no se salga un punto de la
verdad.
osuser@debian:~/ejercicio2$
```

Tras estudiar la implementación del programa y ejecutarlo como se muestra en los ejemplos especificados, contesta a las siguientes preguntas:

- Ejecuta solamente la instancia de envío del programa en una terminal y de forma interactiva (sin redirección de entrada). Como se puede observar, este programa se quedará bloqueado en un punto concreto de su ejecución. ¿Qué llamada al sistema provoca el bloqueo? **Pista:** Para averiguarlo, se puede ejecutar el programa paso a paso con el depurador `gdb`, o bien utilizar el comando `strace` para extraer una traza de las llamadas al sistema que ejecuta el programa en tiempo real: `strace ./fifotest -f myfifo -s`. Consultar página de manual de `strace`.
- Habiendo ejecutado ya la instancia de envío, ejecuta ahora la de recepción en otra terminal. Ahora ambos procesos (emisor y receptor) se quedan bloqueados al invocar llamadas al sistema. ¿Cuáles son dichas llamadas? **Pista:** Usar herramienta `strace` para averiguarlo.
- Ahora procede al contrario que en el primer punto, es decir, lanza solamente la instancia de recepción del programa en una terminal sin lanzar la de envío. ¿En qué llamada al sistema se queda bloqueado ahora el programa?
- Una vez lanzadas ambas instancias del programa, este utiliza paquetes de datos de longitud fija para enviar información desde el proceso emisor al receptor. ¿Qué sucede al introducir una cadena acabada en `'\n'` en la terminal del emisor (lanzado de forma interactiva)? ¿Se espera el programa a construir un paquete de datos “completo” o envía la cadena tecleada directamente a pesar de ser más corta? ¿De qué tamaño es ese paquete enviado? **Pista:** Usar herramienta `strace` para averiguarlo.

Ejercicio 3: Desarrollo de un chat multihilo con FIFOs

Implementar el programa de usuario `chat.c`, un sencillo chat que hará uso de ficheros FIFO y múltiples hilos POSIX para la comunicación. El modo de uso del programa será el siguiente:

```
./chat <usuario> <ruta-fifo-envío> <ruta-fifo-recepción>
```

donde `usuario` es una cadena de caracteres arbitraria que representa el nombre del usuario que se “conecta” al chat. El programa usará un FIFO para enviar mensajes al otro usuario del chat y otro FIFO para recibir los mensajes. Las rutas de ambos ficheros FIFO –que han de estar ya creados– se ha de pasar como argumento, como consta en el modo de uso.

Para poder usar el chat será necesario lanzar dos instancias de dicho programa en distintos terminales del siguiente modo:

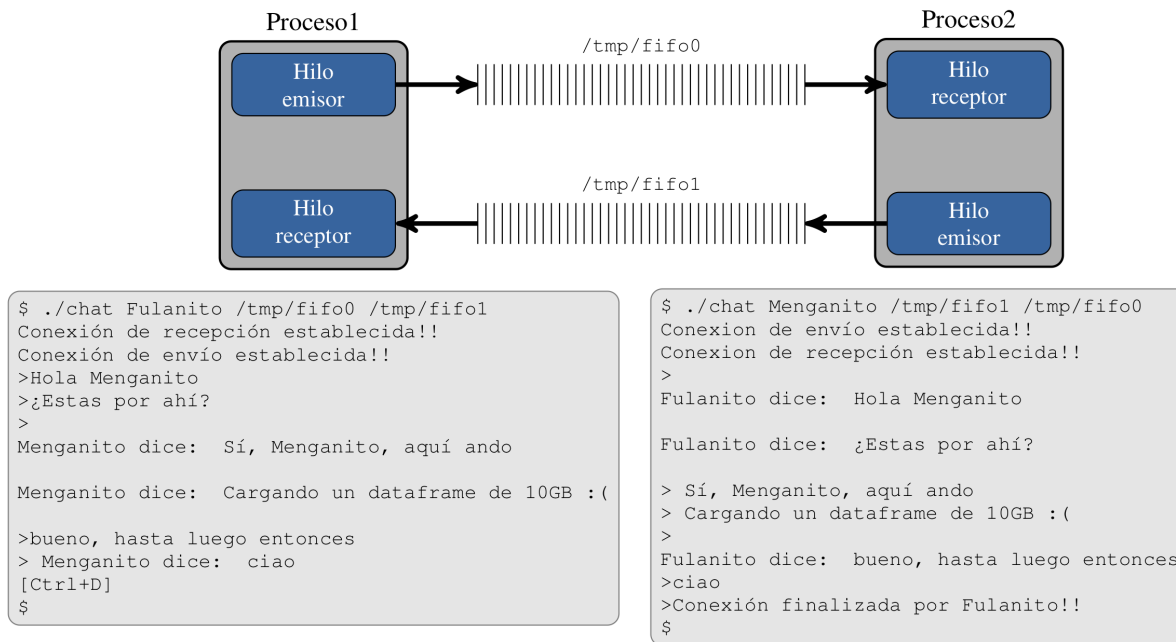
- Terminal 1: `$ ./chat <usuario1> <ruta-fifo1> <ruta-fifo2>`
- Terminal 2: `$ ./chat <usuario2> <ruta-fifo2> <ruta-fifo1>`

Es decir, en ambas instancias del programa se pasará la ruta del mismo par de ficheros FIFOs, pero en distinto orden. Al fin y al cabo, el FIFO de envío en una de las instancias del programa será el FIFO de recepción de mensajes para la otra instancia del programa.

Para la correcta utilización de los FIFOs en el modo de uso habitual de estos (modo bloqueante), el programa `chat.c` creará dos hilos (emisor y receptor) usando `pthread_create()`:

1. **Hilo emisor:** Este hilo leerá líneas por la entrada estándar (esperando la entrada del usuario). Al leer una línea, enviará un mensaje por el primer FIFO especificado en línea de comando
2. **Hilo receptor:** Este hilo permanecerá bloqueado hasta que se reciba un mensaje por el segundo FIFO especificado en línea de comando. Al recibir el mensaje, lo procesará y si procede mostrará información por pantalla

En la siguiente figura se muestra un ejemplo de ejecución del programa `chat.c`, así como un diagrama que ilustra la interacción entre los dos hilos (emisor y receptor) presentes en las dos instancias del programa. En el ejemplo, estos hilos se comunican usando los ficheros FIFO `/tmp/fifo0` y `/tmp/fifo1` previamente creados:



Por simplicidad en la implementación, los datos transferidos a través de cada FIFO se representarán mediante el tipo de datos `struct chat_message`, donde se recogen distintos tipos de mensajes:

```

#define MAX_CHARS_MSG  128
typedef enum {
    NORMAL_MSG, /* Mensaje para transferir lineas de la conversacion entre
                  ambos usuarios del chat */
    USERNAME_MSG, /* Tipo de mensaje reservado para enviar el nombre de
                   usuario al otro extremo*/
    END_MSG /* Tipo de mensaje que se envía por el FIFO cuando un extremo
              finaliza la comunicación */
}message_type_t;

struct chat_message{
    char contenido[MAX_CHARS_MSG]; //Cadena de caracteres (acabada en '\0')
    message_type_t type;
};
  
```

El comportamiento del hilo emisor y receptor de mensajes se resume a continuación, indicando también el uso de los tres tipos de mensajes (`message_type_t`) durante el protocolo de comunicación:

Comportamiento hilo emisor de mensajes

1. Abre FIFO de envío en modo escritura
2. Envía nombre de usuario (especificado en línea de comandos) al otro extremo a través de FIFO de envío usando un mensaje de tipo `USERNAME_MSG`
3. Lee líneas de la entrada estándar una a una y las envía encapsuladas en el campo contenido del mensaje (`NORMAL_MESSAGE`) al otro extremo por el FIFO
 - El procesamiento finaliza cuando usuario teclea CTRL+D (EOF - *fin de fichero*)
 - Al detectar EOF en entrada estándar, enviará `END_MESSAGE` al otro extremo
4. Cierra FIFO de envío y sale

Comportamiento hilo receptor de mensajes

1. Abre FIFO de recepción en modo lectura

2. Procesa mensaje de nombre de usuario (USERNAME_MSG) leyendo del FIFO de recepción
3. Procesa resto de mensajes hasta fin de fichero en el FIFO de recepción, error de lectura o recepción de END_MESSAGE
 - Al recibir un mensaje normal, imprime el campo `content` por pantalla con el siguiente formato:
`<nombre_usuario_emisor> dice: <contenido>`
4. Cierra FIFO de recepción y sale

Práctica 2.1: Introducción a Optimización y explotación Paralelismo de Datos

Índice

Objetivos	1
1 Antes de nada	1
1.1 Compilador Intel® oneAPI DPC++/C++ Compiler	2
Ejercicios	3
Ejercicio 1: Opciones de compilación	3
1.1.1 Tarea	3
Ejercicio 2: Vectorización automática-reportes de compilación	3
1.1.2 Tarea	4
Ejercicio 3: Vectorización automática-Cálculo potencial eléctrico	4
1.1.3 Intel Advisor: Optimización y Análisis de Rendimiento para Aplicaciones de Alto Desempeño	5
1.2 Ejercicio 4: Vectorización guiada	8
1.3 Ejercicio 5: Entrega k-nearest neighbors (KNN)	9
1.3.1 Tarea	9

Objetivos

Esta práctica persigue que los estudiantes se familiaricen con la mejora de rendimiento por la explotación de paralelismo ILP mediante flags de compilación y DLP (SIMD) de forma automática o guiada con pragmas de OpenMP.

1 Antes de nada

- ¿Cómo saber el equipo disponible?
 - Modelo del procesador
 - Número de cores: 8 Performance-cores y 4 Efficient-cores
 - Vectorización: *sse*, *avx*...
 - Especificaciones a consultar en la página de Intel

```
user@lab:~ $ more /proc/cpuinfo
...
processor : 0
...
model name : 12th Gen Intel(R) Core(TM) i7-12700
...
cpu cores : 16
...
...
```

```
flags      : ... avx ... avx2 ...
....
```

- Uso de P-cores o E-cores con `taskset` para fijar el core donde ejecutar
 - 0-15 usa P-cores
 - 16-19 usa E-cores

```
user@lab:~ $ taskset --cpu-list 0 python3 -c "import numpy as np, time; N=5000; A=np.random.rand(N, N);
Execution time: 2.976552724838257 seconds
user@lab:~ $ taskset --cpu-list 16 python3 -c "import numpy as np, time; N=5000; A=np.random.rand(N, N);
Execution time: 8.270392179489136 seconds
```

1.1 Compilador Intel® oneAPI DPC++/C++ Compiler

Durante la práctica vamos a emplear el compilador Intel® oneAPI DPC++/C++ Compiler, que permite explotar varios niveles de paralelismo que vamos a emplear durante la asignatura. La explotación de paralelismo ILP se puede lograr con flags de optimización tipo `-O[n]`, además para la explotación del paralelismo tipo DLP (SIMD) se logra de forma automática con `-O2` o activando OpenMP con el flag `-fopenmp`, para el paralelismo tipo TLP se activa OpenMP con el flag `-fopenmp`, y por último para el paralelismo tipo GPU `-fiopenmp -fopenmp-targets=spir64`.

El compilador está instalado dentro de la suite de software de Intel-oneAPI, que se puede descargar e instalar en la Toolkit Intel® oneAPI Base Toolkit. Para su activación en el laboratorio se debe de activar las variables de entorno:

```
user@lab:~ $ source /opt/intel/oneapi/setvars.sh

:: initializing oneAPI environment ...
  bash: BASH_VERSION = 5.1.16(1)-release
  args: Using "$@" for setvars.sh arguments:
:: advisor -- latest
:: ccl -- latest
:: compiler -- latest
:: dal -- latest
:: debugger -- latest
:: dev-utilities -- latest
:: dnnl -- latest
:: dpcpp-ct -- latest
:: dpl -- latest
:: ipp -- latest
:: ippcp -- latest
:: mkl -- latest
:: mpi -- latest
:: pti -- latest
:: tbb -- latest
:: umf -- latest
:: vtune -- latest
:: oneAPI environment initialized ::

user@lab:~ $ icpx --version
Intel(R) oneAPI DPC++/C++ Compiler 2025.0.4 (2025.0.4.20241205)
Target: x86_64-unknown-linux-gnu
Thread model: posix
InstalledDir: /opt/intel/oneapi/compiler/2025.0/bin/compiler
Configuration file: /opt/intel/oneapi/compiler/2025.0/bin/compiler/./icpx.cfg
```

Ejercicios

Ejercicio 1: Opciones de compilación

Los “flags de compilación” especifica las optimizaciones en compilación. Por defecto el compilador utilizar la opción -O2, aunque existen otras opciones de compilación que habilitan optimizaciones. Estas opciones se conocen como -O[n]

O0	Deshabilita cualquier optimización
O1	Habilita optimizaciones que incrementan tamaño código pero lo acelera Optimización global: análisis de flujo, análisis de vida de variables Deshabilita inlining y uso de intrínsecas
O2	Activa vectorización Optimización a nivel de bucle Optimización entre funciones
O3	O2+ Opt. más agresiva: fusión de bucles, unroll Adecuada para código basados en bucles y cómputo fp

- Con el fin de poder evaluar las ganancias generadas con las diferentes opciones de optimización tomaremos como ejemplo el código `**foo.c**`.

1.1.1 Tarea

- Evalua los tiempos de ejecución y ganancias con las opciones compilación anteriormente descritas que mejoran la explotación del paralelismo ILP.

Ejercicio 2: Vectorización automática-reportes de compilación

La opción **-O2** del compilador de Intel® oneAPI DPC++/C++ Compiler habilita al autovectorizador: O2 “Vectorization is enabled at O2 and higher levels.”

No obstante existe un flag que activa el reporte del compilación `qopt-report=n` para saber si se ha realizado la vectorización, generando un fichero **.opt rpt** donde se indica las optimizaciones realizadas. La opción de reporte tiene 3 niveles de detalle:

- n=0: Disables generation of an optimization report. This is the default when the option is not specified.
- n=1: Tells the compiler to create a report with minimum details.
- n=2: Tells the compiler to create a report with medium details. This is the default if you do not specify arg.
- n=3: Tells the compiler to create a report with maximum details.

Terminal 1:

```
user@lab:~ $ icx -o fooO2 foo.c -O2 -qopt-report=2
user@lab:~ $ more foo.opt rpt
....
LOOP BEGIN at foo.c (23, 2)
<Multiversiomed vl>
  remark #25228: Loop multiversiomed for Data Dependence
  remark #15300: LOOP WAS VECTORIZED
  remark #15305: vectorization support: vector length 2
LOOP END
```

1.1.2 Tarea

- Evalúa si el compilador es capaz de generar código vectorial para `foo.c` de forma automática.

Ejercicio 3: Vectorización automática-Cálculo potencial eléctrico

Cálculo de potencial eléctrico, suponiendo m partículas identificadas por el índice i . Cada partícula es descrita por las tres coordenadas cartesianas $\vec{r} \equiv (r_{i,x}, r_{i,y}, r_{i,z})$ y la carga q_i . El cálculo del potencial $\phi(\vec{R})$ en múltiples puntos $\vec{R}_j \equiv (R_{j,x}, R_{j,y}, R_{j,z})$, donde j denota una de las n localizaciones, se rige por la ecuación $\phi(R_j) = -\sum_{i=1}^m \frac{q_i}{\sqrt{(r_{i,x}-R_{j,x})^2+(r_{i,y}-R_{j,y})^2+(r_{i,z}-R_{j,z})^2}}$.

Para ello necesitamos calcular el potencial eléctrico $\phi(\vec{R})$, donde j indica 1 de las n posiciones. La figura visualiza el problema.

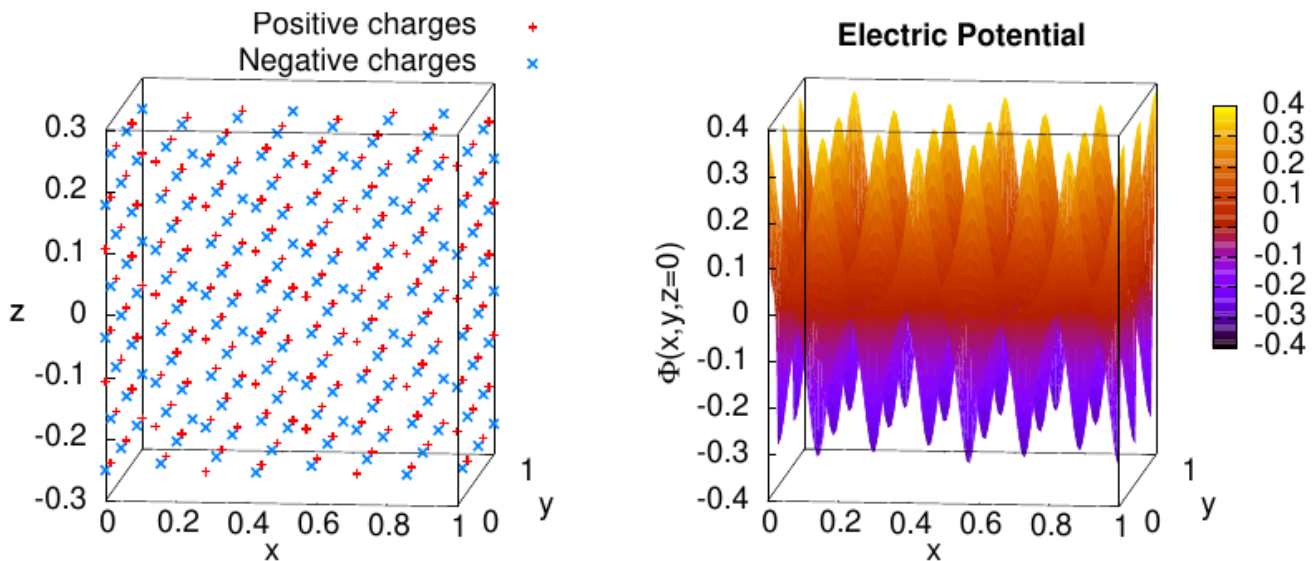


Figure 1: Izda: partículas cargadas. Derecha: Potencial eléctrico ϕ producido por cada partícula

Más información puede encontrarse en el libro donde se ha extraído el ejemplo (pág 290), y el código ha sido extraído del repositorio GitHub.

Para la solución, la codificación emplea una estructura por partícula:

```
struct Charge { // Elegant, but ineffective data layout
    float x, y, z, q; // Coordinates and value of this charge
    int type; // Positive or negative charge
};
```

y el cálculo del potencial eléctrico se puede implementar en la función **CalculateElectricPotential**:

```
void CalculateElectricPotential(
    const int m, // Number of charges
    const Charge* chg, // Charge distribution (array of classes)
    const double Rx, const double Ry, const double Rz, // Observation point
    float & phi // Output: electric potential
) {
    phi=0.0f;
    for (int i=0; i<m; i++) { // This loop will be auto-vectorized
        // Non-unit stride: (&chg[i+1].x - &chg[i].x) == sizeof(Charge)
        const double dx=chg[i].x - Rx;
```

```

const double dy=chg[i].y - Ry;
const double dz=chg[i].z - Rz;
phi -= chg[i].q / sqrt(dx*dx+dy*dy+dz*dz); // Coulomb's law
}
}

```

Existen 4 versiones:

- ver0: versión sin optimizar
 - Conviene prestar atención a las opciones de compilación (añadir arquitectura) y ver ineficiencias (conversiones de datos)
- ver1: optimizado a la arquitectura *target*, pero con ineficiencias de acceso a memoria (stride!=1)
 - Propuesta cambiar AoS por SoA
- ver2: optimización de memoria

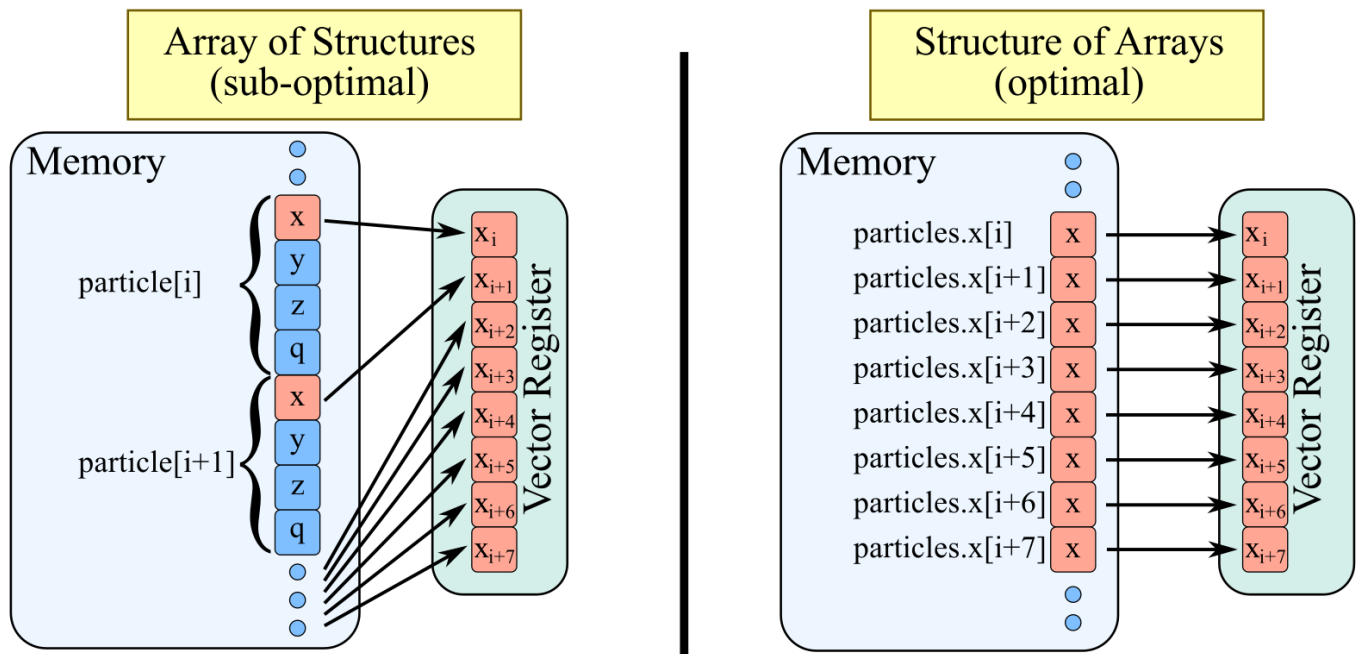


Figure 2: Array of Structure vs Structure of Arrays

- ver3: optimización de memoria a accesos alineados (*compilador no genera accesos alineados*)

Y la compilación de cada versión se puede realizar invocando la herramienta `make`.

1.1.3 Intel Advisor: Optimización y Análisis de Rendimiento para Aplicaciones de Alto Desempeño

Intel Advisor es una potente herramienta de perfilado y optimización diseñada para ayudar a los desarrolladores a mejorar el rendimiento de sus aplicaciones. Proporciona información detallada sobre la ejecución del código, identificando cuellos de botella y oportunidades de optimización, con especial enfoque en la vectorización y paralelización.

Principales Características:

- Análisis de rendimiento avanzado: Permite evaluar la eficiencia de la aplicación mediante la recopilación de métricas clave, ayudando a identificar partes del código que afectan el rendimiento.
- Detección y análisis de bucles vectorizados: Muestra qué bucles están siendo vectorizados correctamente y cuáles pueden beneficiarse de optimización, proporcionando informes detallados con sugerencias específicas.

- Recomendaciones para mejorar la eficiencia: Intel Advisor no solo detecta áreas de mejora, sino que también proporciona recomendaciones basadas en modelos avanzados de análisis para optimizar el uso de recursos del hardware.
- Interfaz intuitiva y generación de reportes: Su interfaz gráfica y opciones de línea de comandos permiten una visualización clara de los datos y la generación de informes detallados para la toma de decisiones.

El uso del Intel Advisor se puede realizar con el comando `advisor-gui`.

1.1.3.1 Ejemplo Intel Advisor

- 1 Botón de control para guardar resultados
- 2 y 3 Filtro de resultados
- 10 Modelo roofline
- 11 Resumen: funciones vectorizadas, bucles vectorizados, función escalar y para bucles escalares
- 13 Panel de recomendaciones (Optimización)
- 14 Razones para la no vectorización

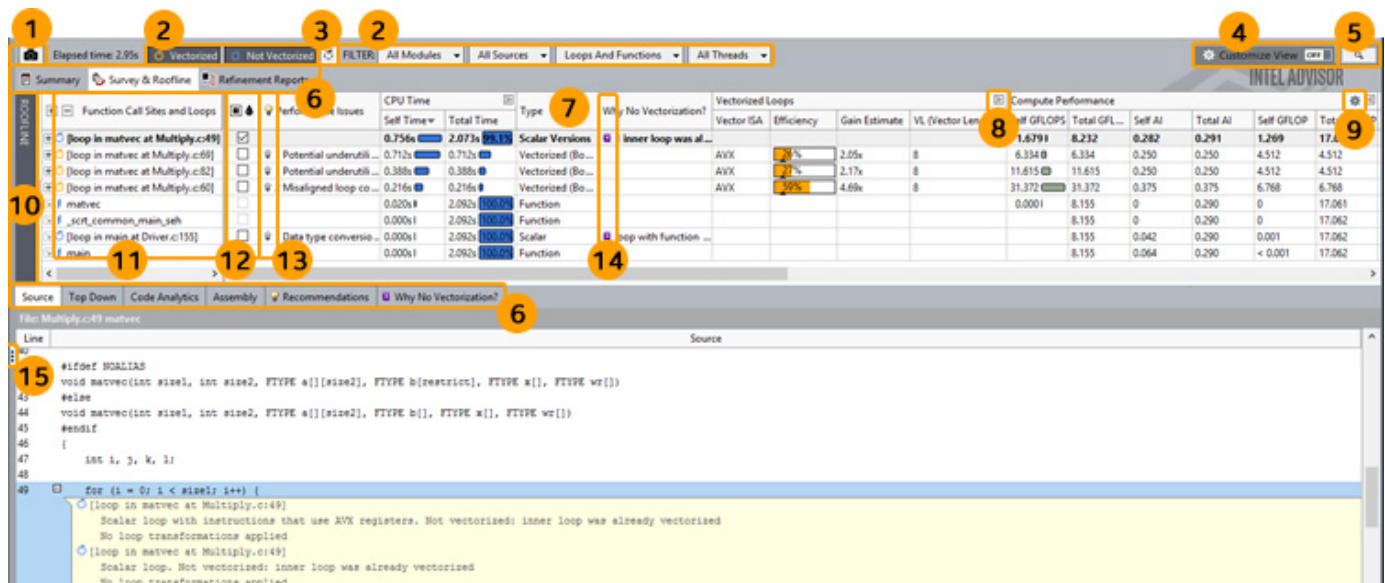


Figure 3: Intel Advisor

1.1.3.2 Panel de Recomendación (recommendation pannel)

- Si todos los bucles son vectorizados apropiadamente (>90% eficiencia): **rendimiento satisfactorio**
- Si no... En *Performance Issues* y panel *Recommendations* aparecen una serie de recomendaciones:
 - ¿Por qué no se vectoriza? Posibles dependencias, llamada a función, o que simplemente el compilador lo determina como ineficiente. Más información en URL
 - Si **vectoriza pero de forma ineficiente**: patrón de acceso a memoria ineficiente, tipo de conversiones de datos, falta de acceso a memoria alineado...

1.1.3.3 Modelo roofline

- Permite obtener modelo roofline
- Detectar posibles problemas (*Code Analysis*)

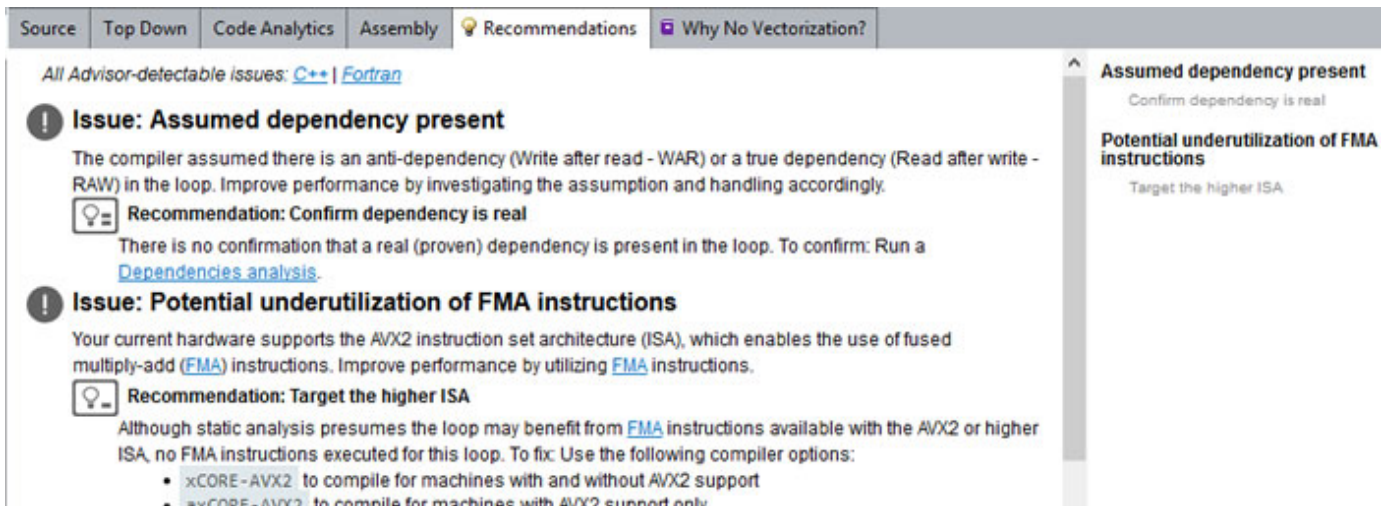


Figure 4: Intel Advisor-Panel Recomendación

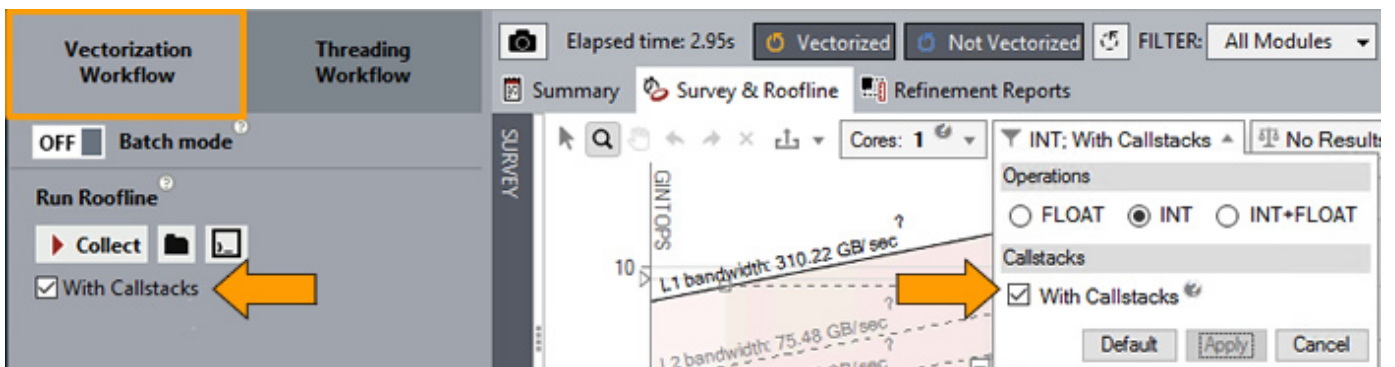


Figure 5: Intel Advisor-Roofline

1.1.3.4 Modelo de acceso a memoria

- Permite hacer un análisis de patrones de acceso a memoria: *Memory Access Patterns (MAP) Report*
 - *Uniform*: stride 0
 - *Unit stride* (stride 1)
 - *Constant stride* (stride N)
 - *Irregular stride*
 - *Gather (irregular) stride*

Site Location	Loop-Carried Dependencies	Strides Distribution	Access Pattern	Footprint Estimate	Site Name
				Max. Per-Instruction Addr. Range	
[i] [o] [loop in matvec at Multiply.c...	No Information Available	100% / 0% / 0%	All Unit Strides	20B	loop_site
[i] [o] [loop in matvec at Multiply.c...	No Information Available	100% / 0% / 0%	All Unit Strides	128B	loop_site
[i] [o] [loop in matvec at Multiply.c...	No Information Available	No Strides Found	No Strides Found	0B	loop_site
[i] [o] [loop in matvec at Multiply.c...	No Information Available	50% / 50% / 0%	Mixed Strides	4KB	loop_site
<pre> 67 #pragma unroll 68 #pragma vector always 69 for (i = 0; i < size2; i++) { 70 b[i] += a[i][i] * x[i]; 71 } </pre>					
[i] [o] [loop in matvec at Multiply.c...	No Information Available	50% / 50% / 0%	Mixed Strides	756B	loop_site
[i] [o] [loop in matvec at Multiply.c...	No Information Available	No Strides Found	No Strides Found	0B	loop_site
[i] [o] [loop in matvec at Multiply.c...	RAW:1 WAW:1	100% / 0% / 0%	All Unit Strides	140B	loop_site

ID	Stride	Type	Source	Nested Function	Variable references	Max. Per-Instruction Addr. Range	Modules	Site Name	Access Type
[i] P2	376	Constant stride	Multiply.c:70		a	4KB	vec_samples.exe	loop_site_22	Read
[i] P7		Parallel site information	Multiply.c:69				vec_samples.exe	loop_site_22	
[i] P21	1	Unit stride	Multiply.c:70		x	128B	vec_samples.exe	loop_site_22	Read

Figure 6: Intel Advisor-Patrón de acceso a memoria

1.2 Ejercicio 4: Vectorización guiada

Existen ocasiones donde la vectorización automática no puede realizarse, bien por la complejidad del código en el que el compilador no es capaz de hacer uso del paralelismo SIMD, u otros factores que **inhiben la auto-vectorización** como uso de punteros, patrones de acceso irregulares, dependencias de datos u rendimiento esperado pobre. En estos casos el programador puede guiar al compilador mediante el uso de directivas o pragmas como `#pragma ivdep` o `pragma omp simd`. El código `guided_vec.cpp` presenta este problema:

```

user@lab:~ $ more guided_vec.optrpt
Begin optimization report for: compute_with_pointers(int, double*, double*, double*, double&)

LOOP BEGIN at guided_vec.cpp (9, 5)
    remark #25439: Loop unrolled with remainder by 4
LOOP END

LOOP BEGIN at guided_vec.cpp (9, 5)
<Remainder loop>
LOOP END

LOOP BEGIN at guided_vec.cpp (14, 5)
<Multiversiomed v2>
LOOP END

LOOP BEGIN at guided_vec.cpp (14, 5)
<Multiversiomed v1>
    remark #25228: Loop multiversiomed for Data Dependence
    remark #25564: Store sinked out of the loop
    remark #25439: Loop unrolled with remainder by 8

```

1.3 Ejercicio 5: Entrega k-nearest neighbors (KNN)

El algoritmo **K-Nearest Neighbors (KNN)** es un método de clasificación supervisado que asigna una etiqueta a una nueva instancia en función de la mayoría de sus vecinos más cercanos. Para ello vamos a describir el funcionamiento del algoritmo KNN:

1. **Preparación de Datos:**

- Se tiene un conjunto de datos con características (*features*) y etiquetas (*labels*). - Cada instancia del conjunto de datos es representada como un punto en un espacio multidimensional.

2. **Cálculo de Distancias:**

- Para clasificar un nuevo punto, se calcula la distancia entre este y todos los puntos del conjunto de entrenamiento.
- Se suele utilizar la **distancia Euclidia**, aunque también pueden emplearse otras métricas como la distancia Manhattan o Coseno.

3. **Selección de Vecinos Más Cercanos:**

- Se eligen los **K** puntos más cercanos en el conjunto de entrenamiento (donde **K** es un número entero definido por el usuario).

4. **Votación de Clases:**

- Se cuentan las etiquetas de los **K** vecinos más cercanos.
- La nueva instancia se clasifica en la categoría que tenga la mayoría de votos.

5. **Clasificación Final:**

- La instancia es asignada a la clase predominante entre los K vecinos.

Veamos con el ejemplo de la figura, como para clasificar un número punto (*punto azul*), buscamos los puntos con la distancia más cercada (k=5 significa los 5 vecinos más cercanos), y los clasificamos al nuevo punto con la clase predominante, en este ejemplo la clase verde.

1.3.1 Tarea

El entregable consistirá en modificar el código para que el alumno logre una implementación que explote de manera eficiente el procesamiento vectorial o SIMD, o bien de forma automática o guiada por medio de pragmas. Recomendaciones: - Evaluar las rutinas más costosas entre la medidas de tiempo, pudiendo hacer uso de los reportes o el profiler Intel-Advisor

```
clock_t start = clock();

// Classify each test sample
int k = 4;
for (int i = 0; i < test_data.n_samples; i++) {
    y_pred[i] = classify(train_data, test_data.samples[i].X, k, distances, labels);
}

clock_t end = clock();
double duration = (double)(end - start) / CLOCKS_PER_SEC;
```

- Posibles ineficiencias del código en la función `classify`:
 - Uso de memoria en Array-of-Structures que inhiban o reduzcan la aceleración potencial
 - Empleo de conversiones de datos
 - No utilización del repertorio de instrucciones del procesador
 - Dependencias de datos en bucles paralelos... ect

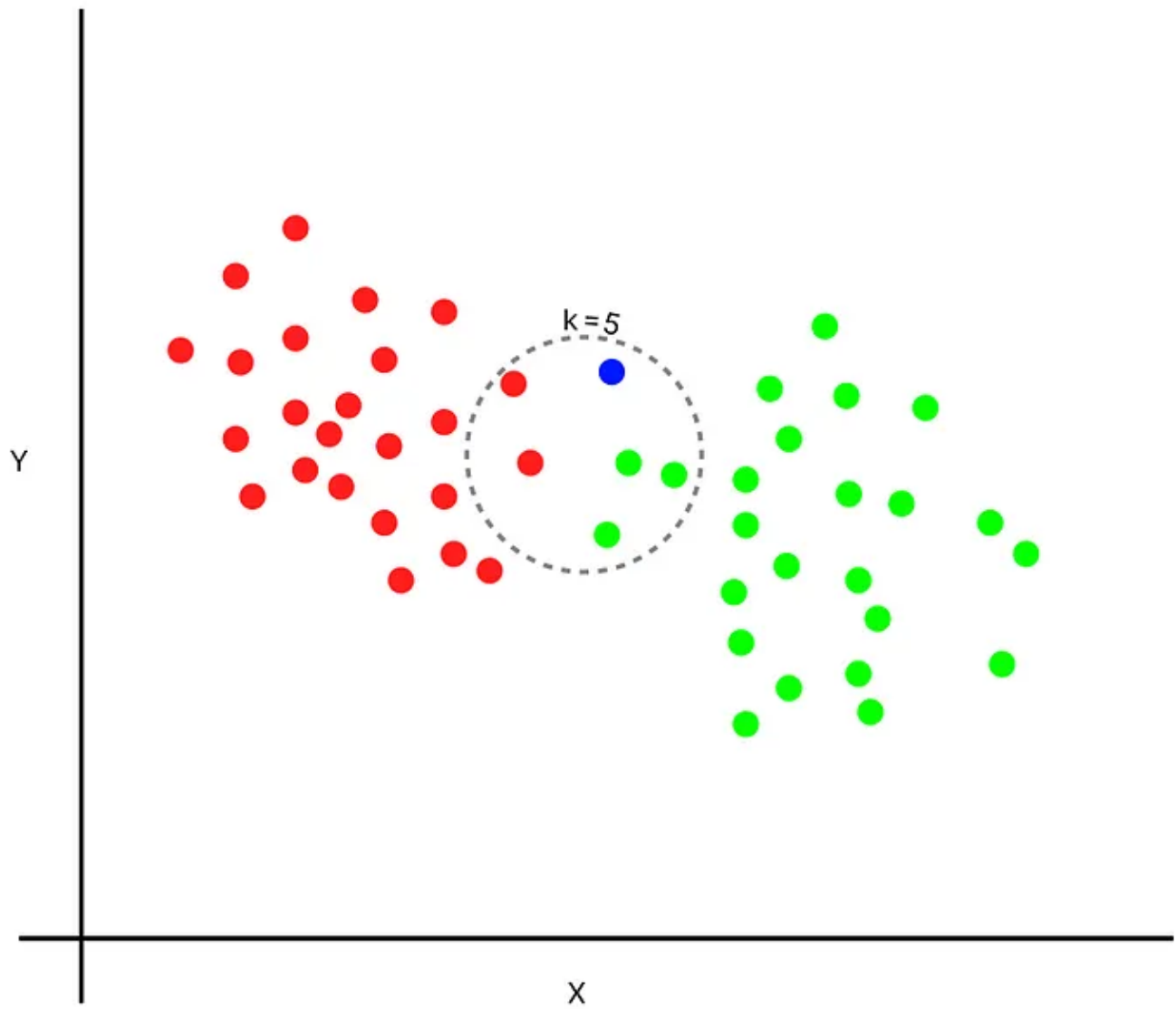


Figure 7: Ejemplo algoritmo KNN

Práctica 2.1: Introducción a Optimización y explotación Paralelismo de Datos

Índice

Objetivos	1
Ejercicio 1: Hello World	1
Ejercicio 2: Directivas de trabajo compartido	2
Expresión de paralelismo	3
Tarea	3
Ejercicio 3: Condiciones de Carrera	3
Ejercicio 4: Tareas OMP	5
A tener en cuenta	5
Tareas a realizar por el alumno	6
Tareas a considerar	6
Ejercicio 5: Quick-sort	6
Ejercicio 6: K-means	6
¿Como funciona el algoritmo?	6
Tarea	8
Herramienta de perfilado VTune	9
Intel VTune	9
Opciones de análisis	9
Intel VTune: Análisis Hotspots	10
Intel VTune: Parallelism Analysis	10
Intel VTune: Threading Efficiency	11
Intel VTune: Threading Efficiency	11
Intel VTune: Microarchitecture	13

Objetivos

Esta práctica persigue que los estudiantes se familiaricen con la mejora de rendimiento por la explotación de paralelismo TLP mediante el uso de OpenMP. Este esquema de paralelismo es compatible con los estudiados hasta el momento en la asignatura.

Ejercicio 1: Hello World

El código “hello.c” versará sobre la compilación y control de hilos de ejecución

```
#include <stdio.h>
#include <omp.h>
```

```
int main(){

    // This code is executed by 1 thread
    printf("OpenMP with %d max threads\n", omp_get_max_threads ());
    printf("OpenMP with %d procs available\n", omp_get_num_procs ());

    #pragma omp parallel
    {
        // This code is executed in parallel
        // by multiple threads
        printf("Hello World from thread %d of %d threads\n",
            omp_get_thread_num(), omp_get_num_threads ());
    }
}
```

- Pudiendo compilar con ambos compiladores y generando el fichero de reportes

```
user@lab:~ $ icx -o hello.icx hello.c -qopenmp -qopt-report=1
user@lab:~ $ more hello.optrpt
```

```
...
COMPILE FUNC: main
-> BROKER: __kmpc_fork_call hello.c(14,2) (main.extracted)
```

- Para controlar el número de hilos de ejecución se puede hacer de varias formas:
 - Con la variable de entorno `OMP_NUM_THREADS`
 - En la pragma `#pragma omp parallel num_threads(3)`

```
user@lab:~ $ export OMP_NUM_THREADS=3
user@lab:~ $ ./hello.icx
OpenMP with 3 max threads
OpenMP with 4 procs available
Hello World from thread 0 of 3 threads
Hello World from thread 1 of 3 threads
Hello World from thread 2 of 3 threads
```

- Para controlar el tipo de cores (P-cores o E-cores) donde ejecutar el código se recomienda emplear el comando `taskset`

```
user@lab:~ $ OMP_NUM_THREADS=2 taskset --cpu-list 0,1 ./hello.icx
OpenMP with 2 max threads
OpenMP with 2 procs available
Hello World from thread 0 of 2 threads
Hello World from thread 1 of 2 threads
```

Ejercicio 2: Directivas de trabajo compartido

- En OpenMP existen varias formas de explicitar la distribución del trabajo entre los hilos
 - `#pragma omp for/#pragma omp parallel for` para distribuir las iteraciones de los bucles entre los hilos disponibles
 - `#pragma omp sections` para crear secciones de código concurrente entre hilos
 - `#pragma omp single` para expresar una región secuencial dentro de una región paralela
- En este ejemplo, el código **prime.c** trabaja dos aspectos:
 - Distribución de trabajo entre hilos
 - Gestión de variables (privadas, compartidas...)
- El ejemplo “prime.c” calcula los números primos menores a un determinado `n` (parámetro de entrada)
 - `i` es primo si no tiene divisores ($2, i/2$)
 - `#define DEBUG` visualiza la lista de primos

```
//#pragma omp parallel...
for (i=2; i<n; i++)
{
    not_flag = 0;
    j=2;
    while(j<=i/2 && !not_flag)
    {
        if(i%j==0) // not prime
            not_flag=1;
        j++;
    }
    if (j>=i/2 && !not_flag)
        primes[++k] = i;
}
```

Expresión de paralelismo

- Iteraciones del bucle **i** potencialmente paralelas
 - Iteraciones independientes
- A tener en cuenta
 - Uso de variables: privadas, compartidas... ect
 - Variable ++k es el índice de la lista de números primos
 - * Posible carrera: problema **read&update&write**
 - * Solución: `#omp critical`
 - * Lista desordenada: implementar un `sort(primes)`

Tarea

- **A Evaluar** Se recomienda estudiar el impacto de las diferentes políticas de distribución de carga
 - *STATIC*
 - *STATIC, chunk*
 - *DYNAMIC[, chunk]*
 - *GUIDED[, chunk]*
 - *AUTO*

Ejercicio 3: Condiciones de Carrera

- El tercer ejercicio se basa en el código “trapezoidal.c” que calcula la integral mediante el método del trapecio de acuerdo a la siguiente ecuación
 - Area del trapecio $S = \frac{1}{2}(f(a') + f(b')) * h$
 - Integral como suma de trapecios: $\int_a^b f(x) \partial x = \sum \frac{f(a') + f(b')}{2} h$
- Podemos destacar **dos tipos tareas**:
 - Cálculo de areas de cada trapecio individual
 - Acumulación de trapecios (*integral*) donde potencialmente pueden aparecer condiciones de carrera

```
double Trap(double a, double b, int n, double h) {
    double integral, area;
    int k;

    integral = 0.0;
    for (k = 1; k <= n; k++) {
        area = h * (f(a+k*h) + f(a+(k-1)*h)) / 2.0;
        integral += area;
    }
}
```

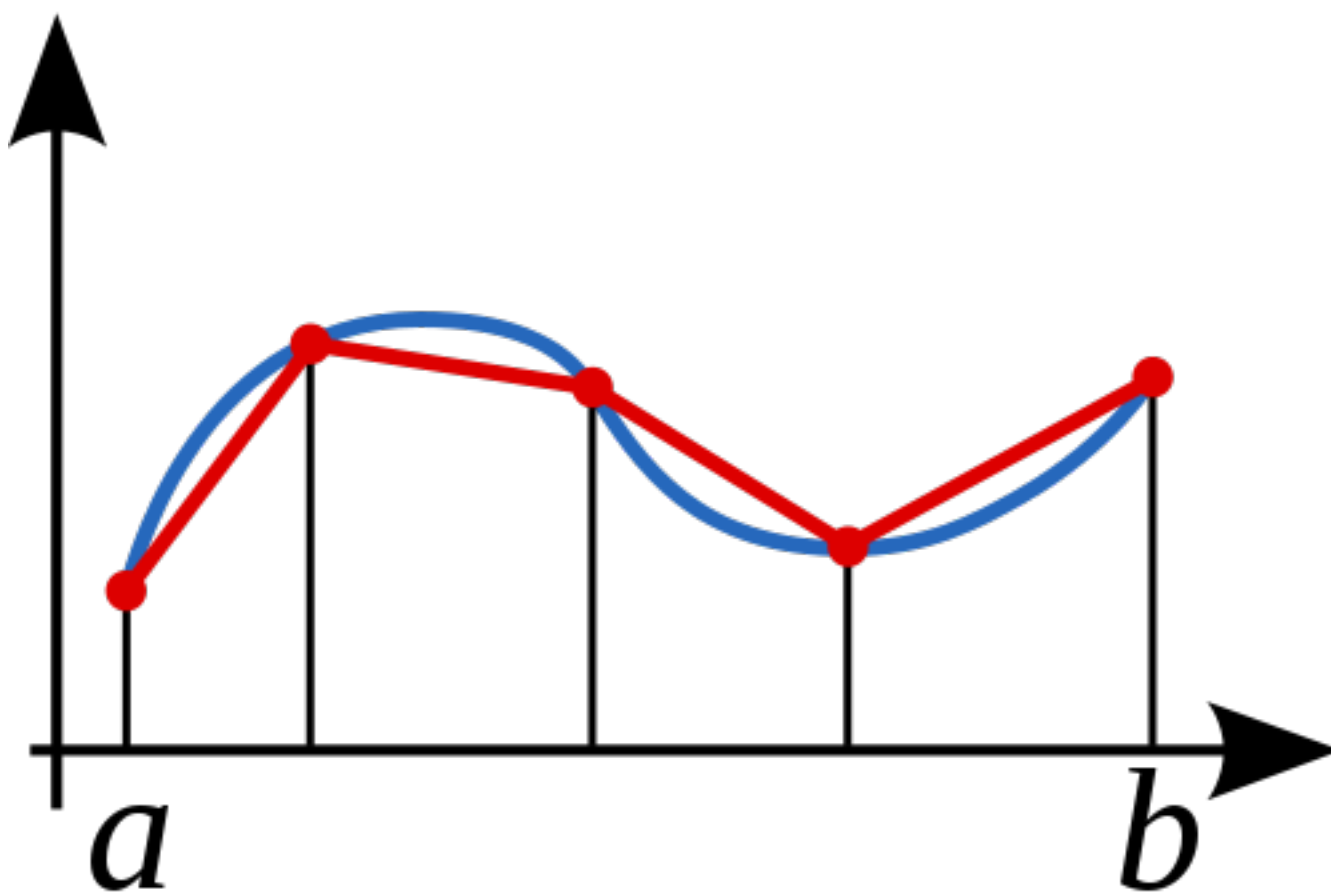


Figure 1: Aproximación de integral a trapezoide

```

    return integral;
} /* Trap */

```

Ejercicio 4: Tareas OMP

- El cuarto ejercicio se basa en “fibonacci.c” donde se pone en valor la explotación del paralelismo mediante tareas haciendo uso del algoritmo de Fibonacci
 - Tareas OMP = unidades de trabajo (ejecución puede diferirse)
 - Las tareas se componen de:
 - * código para ejecutar y datos
 - Hilos pueden **cooperar** para ejecutarlas

```

long comp_fib_numbers(int n)
{
    long fnm1, fnm2, fn;
    if ( n == 0 || n == 1 ) return(1);
    if ( n<20 ) return(comp_fib_numbers(n-1) +comp_fib_numbers(n-2));

    #pragma omp task...
    {fnm1 = comp_fib_numbers(n-1);}
    #pragma omp task...
    {fnm2 = comp_fib_numbers(n-2);}
    #pragma omp...
    fn = fnm1 + fnm2;

    return(fn);
}

int main(int argc, char* argv[]) {
    ....
    #pragma omp parallel
    {
        #pragma omp single
        fibo = comp_fib_numbers(n);
    }
    ...
} /* main */

```

A tener en cuenta

- Posible carrera: problema **read&update&write** en la variable *integral*
 - Estudiar las diferentes alternativas:
 - * Uso de `atomic`, `critical`
 - * Expresión de paralelismo con `#pragma omp for` pero usando la cláusula **reduction**

```

#pragma omp parallel private(...) firstprivate(...) shared(integral)
{
    id = ...
    nth = ...
    n_per_thread = n/nths;

    k_init_thread = ...
    k_end_thread = ...

    integral = 0.0;
    for (k = k_init_thread; k <= k_end_thread; k++) {
        area = h*(f(a+k*h)+f(a+(k-1)*h))/2.0;
    }
}

```

```

        integral_thread+=area;
    }

    #pragma omp critical
    {integral += integral_thread;}

}

return integral;

```

Tareas a realizar por el alumno

- Las tareas a entregar por el alumno corresponde a la implementación paralela mediante el paradigma de programación de OpenMP sobre de tres algoritmos:
 1. Quick-sort
 2. Kmeans

Tareas a considerar

- Paralelizar los códigos
 - Se recomienda utilizar la herramienta Intel vTune para encontrar los cuellos de botella y evaluar la escalabilidad de la paralelización OpenMP
 - No olvides que puedes combinar paralelismo del tipo SIMD con paralelismo multi-hilos en los cores (ambos soportados por OpenMP)

Ejercicio 5: Quick-sort

- Algoritmo de ordenación basado en el concepto divide y vencerás
 1. Elegir un elemento al que llamaremos pivote.
 2. Mover los demás elementos de la lista a cada lado del pivote
 - La lista inicial está separada en dos sublistas: con elementos menores y mayores al pivote
 3. Repetir el proceso de forma recursiva para cada sublista

El código de **quicksort.cpp** hace una llamada a la función `quickSort` que calcula la posición del pivote (invocación a `partition`) lo que permite realizar llamadas recursivas a `quickSort` para cada una de las sublistas:

```

void quickSort(int arr[], int low, int high){
    if(low < high){
        int pivot = arr[high];
        int pos = partition(arr, low, high, pivot);

        quickSort(arr, low, pos-1);
        quickSort(arr, pos+1, high);
    }
}

```

Ejercicio 6: K-means

El K-Means es un algoritmo de aprendizaje no supervisado utilizado para la agrupación de datos en K clusters. Su objetivo es minimizar la suma de las distancias cuadradas entre los puntos de datos y el centroide más cercano.

¿Como funciona el algoritmo?

1. *Inicialización*: Se eligen K centroides de manera aleatoria.

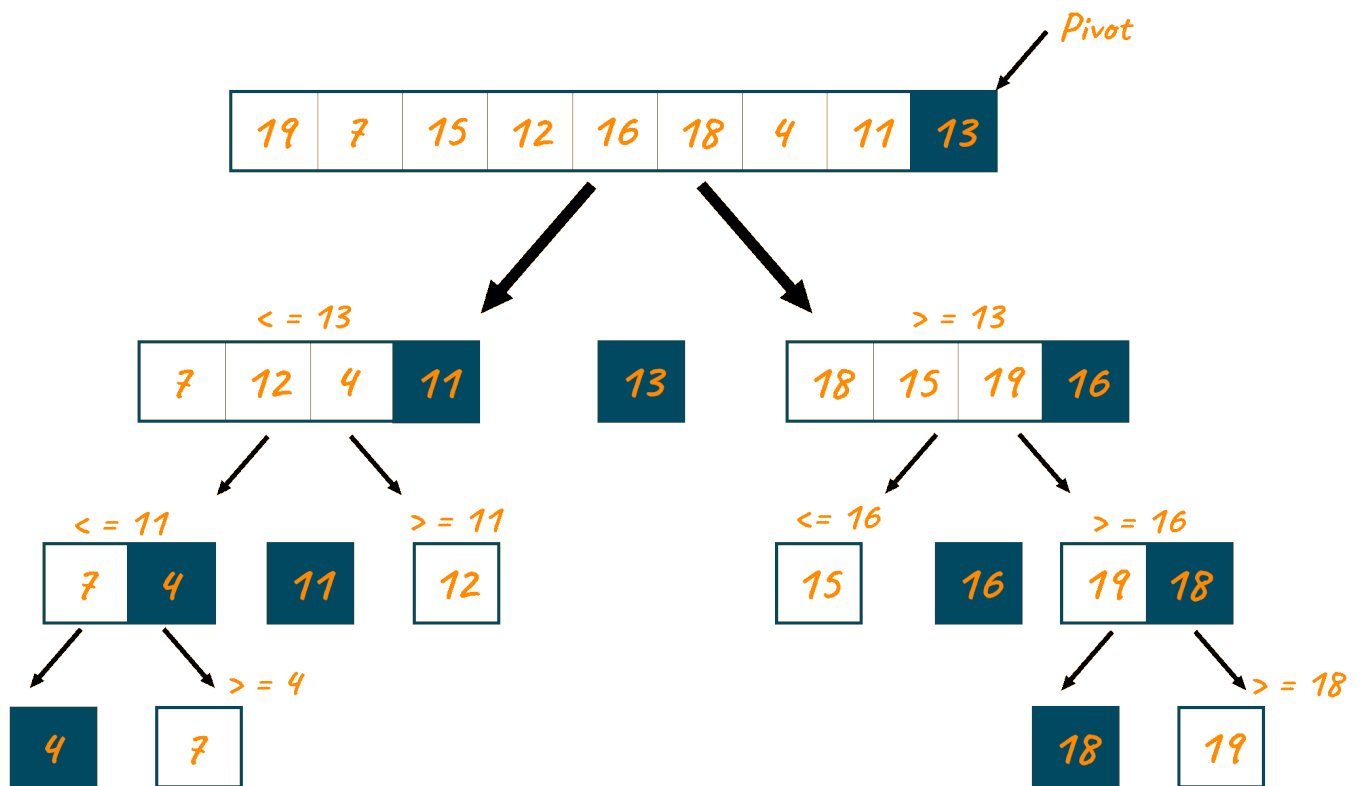


Figure 2: Algoritmo QuickSort

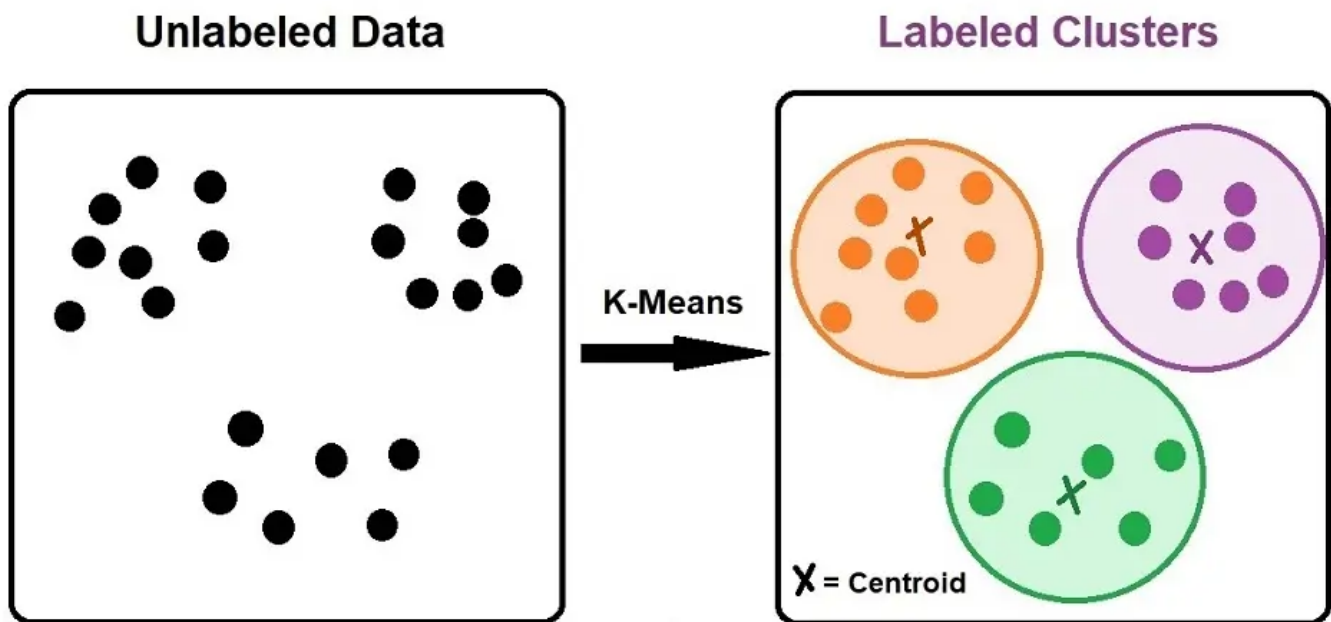


Figure 3: Algoritmo K-means

2. *Asignación del Clúster*: Cada punto se asigna al centroide más cercano.
3. *Reubicación de Centroides*: Se calculan los nuevos centroides como el promedio de los puntos asignados a cada cluster.
4. *Repetición*: Se repiten los pasos 2 y 3 hasta que los centroides no cambian significativamente (convergencia).

La implementación del algoritmo se realiza en el fichero **kmeans.cpp** donde la función `kmeans` es la encargada de realizar las etapas anteriormente mencionadas:

- La etapa de inicialización se realice en la función `initializeCentroid` donde se seleccionan como centroides algunos de los puntos del dataset almacenado en la estructura `data`
- La asignación de cluster se procesa en la función `assignCluster` donde se calcula las distancias euclídeas a los centroides y se queda con la distancia menor a todos los centroides
- La actualización de los centroides se realiza como la media de las posiciones asociadas a cada clase en la función `updateCentroid`

```
void kmeans(std::vector<Point>& data, int k, int max_iters = 20000) {
    int dimensions = data[0].coords.size();
    std::vector<Centroid> centroids(k);
    initializeCentroids(centroids, data, k);

    for (int iter = 0; iter < max_iters; iter++) {
        assignClusters(data, centroids, k);

        std::vector<Centroid> newCentroids = centroids;
        updateCentroids(data, newCentroids, k, dimensions);

        centroids = newCentroids;
    }

    std::cout << "Final cluster centers:\n";
    for (int i = 0; i < k; i++) {
        std::cout << "Cluster " << i << ": (";
        for (size_t d = 0; d < centroids[i].coords.size(); d++) {
            std::cout << centroids[i].coords[d] << (d == centroids[i].coords.size() - 1 ? ")" : ", ");
        }
        std::cout << "\n";
    }
}
```

El contenido tanto de los puntos del dataset a entrenar como el de los centroides es una estructura con las coordenadas de cada punto N-dimensional y el tipo de cluster que viene descrito en el fichero `*csv_dataset.h`:

```
struct Point {
    std::vector<double> coords;
    int cluster;
};

struct Centroid {
    std::vector<double> coords;
};
```

Tarea

- Realizar una implementación en OpenMP que explote el paralelismo TLP mediante hilos teniendo en cuenta:
 - La función `assignCluster` puede paralelizarse sencillamente ya que el cálculo de la distancia a los centroides y la selección del centroide más cercano es una tarea completamente independiente
 - La función `updateCentroids` puede también paralelizarse pero presenta un reto mayor: **existen varias reducciones** como el proceso en el array `count[cluster]`, o `sumCoords[cluster][d]`

Herramienta de perfilado VTune

Intel VTune

- Intel® VTune™ Profiler
- Como mejorar el rendimiento con varios análisis
 - Hotspots
 - Threading Efficiency
 - Microarchitecture
 - Memory Access
- Variables de entorno

```
user@lab:~ $ export ONEAPI_PATH=/opt/intel/oneapi/  
user@lab:~ $ source $ONEAPI_PATH/vtune/latest/env/vars.sh
```

- La Herramienta Intel Vtune Amplifier nos permite realizar un perfilado de la aplicación paralela para detectar posibles mejoras
 - Lanzamiento herramienta gráfica `amplxe-gui`, más moderno **vtune-gui**
 - Lanzamiento por línea de comandos `amplxe-cl`, más moderno **vtune-collect**

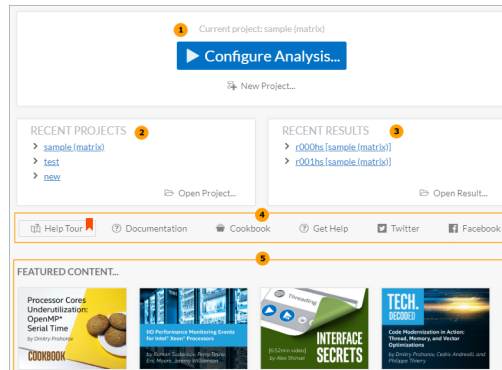


Figure 4: Inicio VTune

Comienzo Herramienta

1. Proyecto actual con el análisis de configuración (*New Project*)
2. Proyectos recientes
3. Resultados recientes
4. Recursos y documentación online
5. Artículos de y últimas noticias

Opciones de análisis

- Hotspots
 - Utilización de CPU
- Memory Consumption:
 - Consumo de memoria (malloc-free) en cada rutina
- Microarchitecture Exploration:
 - Analiza cuellos de botella que afectan al rendimiento
 - Contadores HW
- Memory Access
 - Identifica los accesos a jerarquía memoria (NUMA, DRAM, L2...)

- Análisis
 - Más info en web de Intel-VTune y vídeo
 - A la hora de crear el proyecto con *New Project* seleccionar *Local Host* para analizar en el equipo local
 - * Introducir el ejecutable (*path*) y primer análisis *Hotspots*

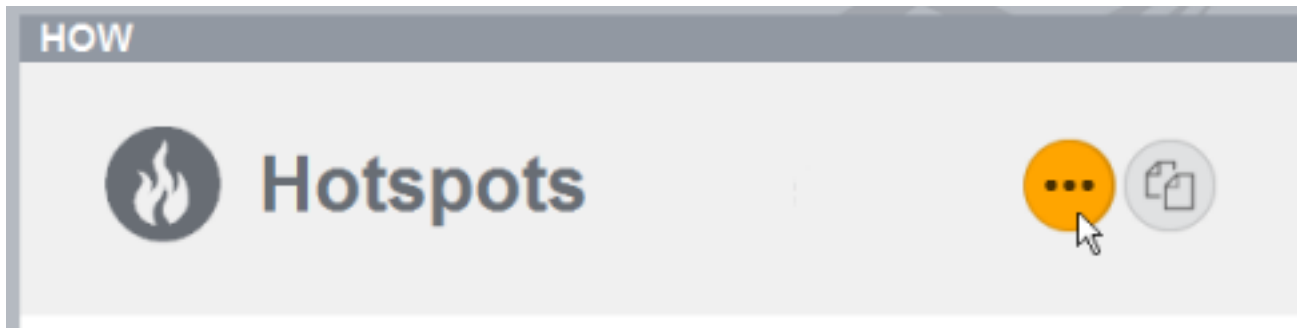


Figure 5: Análisis VTune

Intel VTune: Análisis Hotspots

- *Hotspots* dirigido a la optimización de software y conocer dónde pasa tiempo la aplicación: **analizar la eficiencia**
- Incluye los análisis:
 - *Hotspots* para identificar las funciones más costosas y muestra la actividad de cada hilo
 - *Memory consumption* para analizar el consumo de memoria: caches y RAM

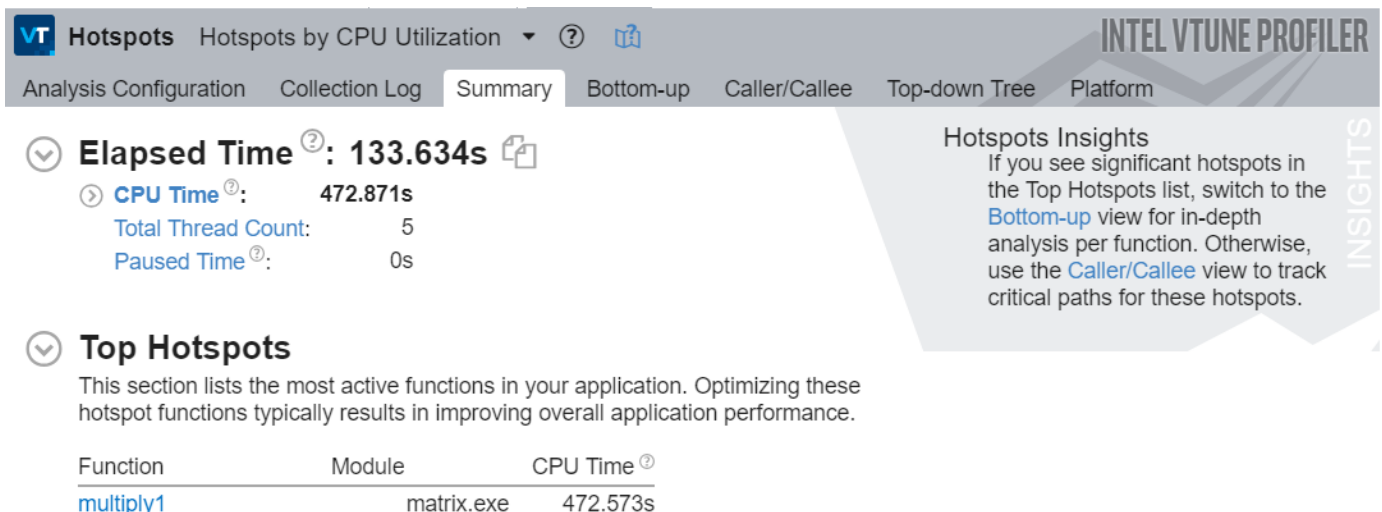


Figure 6: Análisis Hotspot

Intel VTune: Parallelism Analysis

- Tipos de análisis para aplicaciones sensibles en cómputo: análisis general del rendimiento
- Incluye los análisis:
 - *Threading*: muestra balanceo de trabajo entre hilos y puntos de sincronización
 - *Compute-intensive Application*: caracterización rendimiento de HPC (punto flotante y la eficiencia de la memoria)

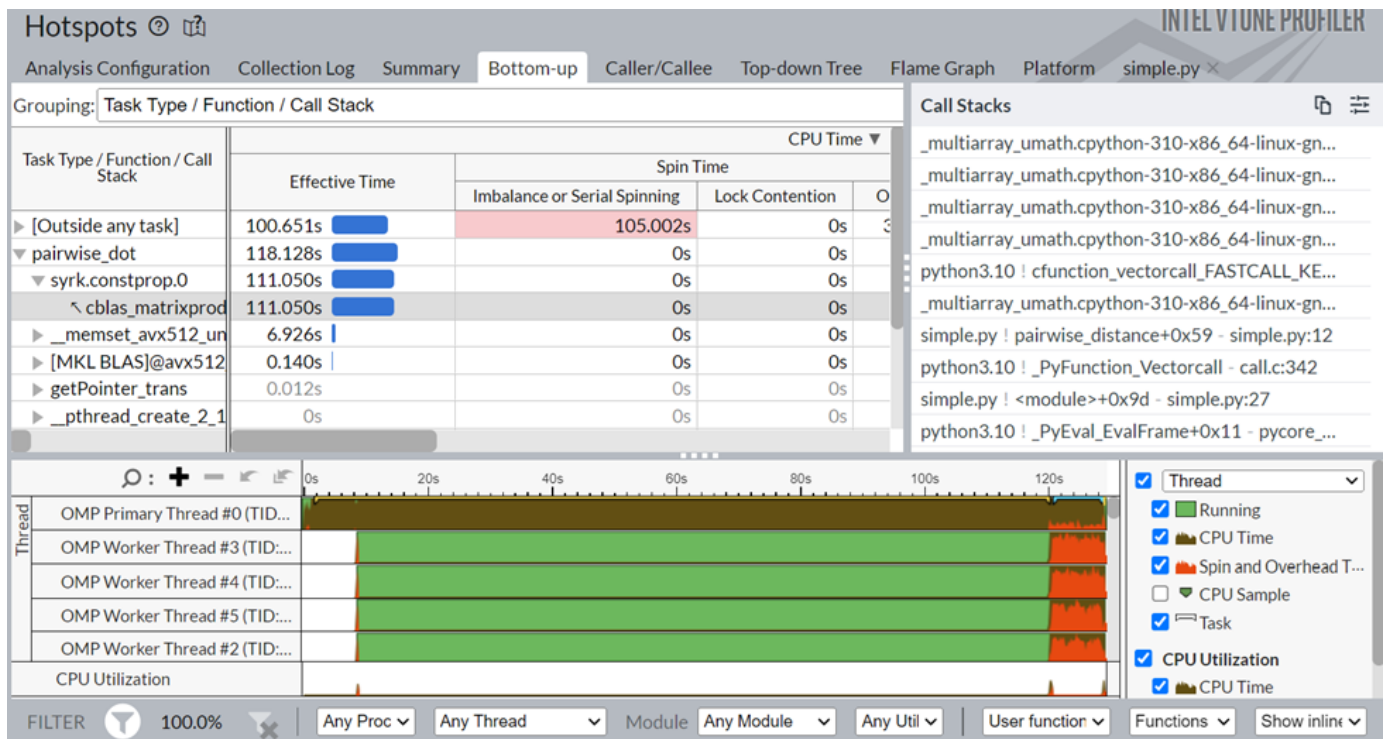


Figure 7: Análisis Paralelismo

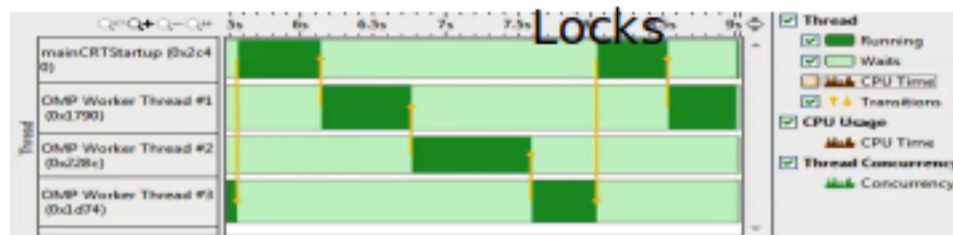
Intel VTune: Threading Efficiency

- Permite explorar
 - Wait Time: esperas prologadas por regiones de sincronización
 - Spin y Overhead Time: esperas y sobrecoste asociado al manejo de las regiones paralelas
 - Thread count: tiempo espera debido a sobresubscripción, espera a que los recursos compartidos estén disponibles cuando se ejecutan más hilos lógicos que cores físicos
- Problemas asociados al paralelismo
 - Asegurarse que se ejecutan todos los hilos disponibles
 - Problemas comunes a la concurrencia pueden identificarse
 - Análisis para detectar la contención en operaciones tipo Locks/Waits

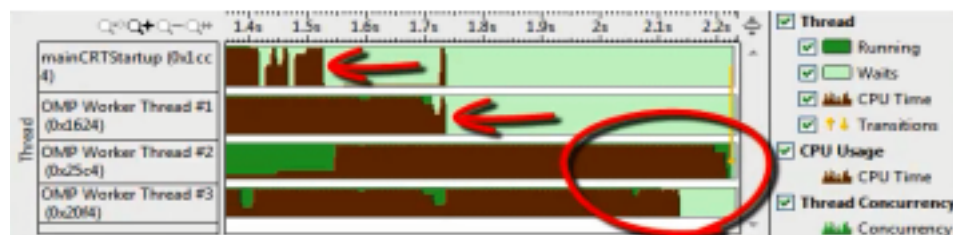
Intel VTune: Threading Efficiency

- Demo:
 - Paralelización bucle for

Coarse-Grain



Thread Imbalance



High Lock Contention

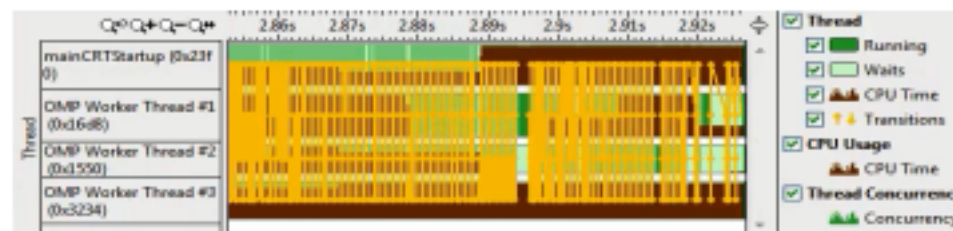
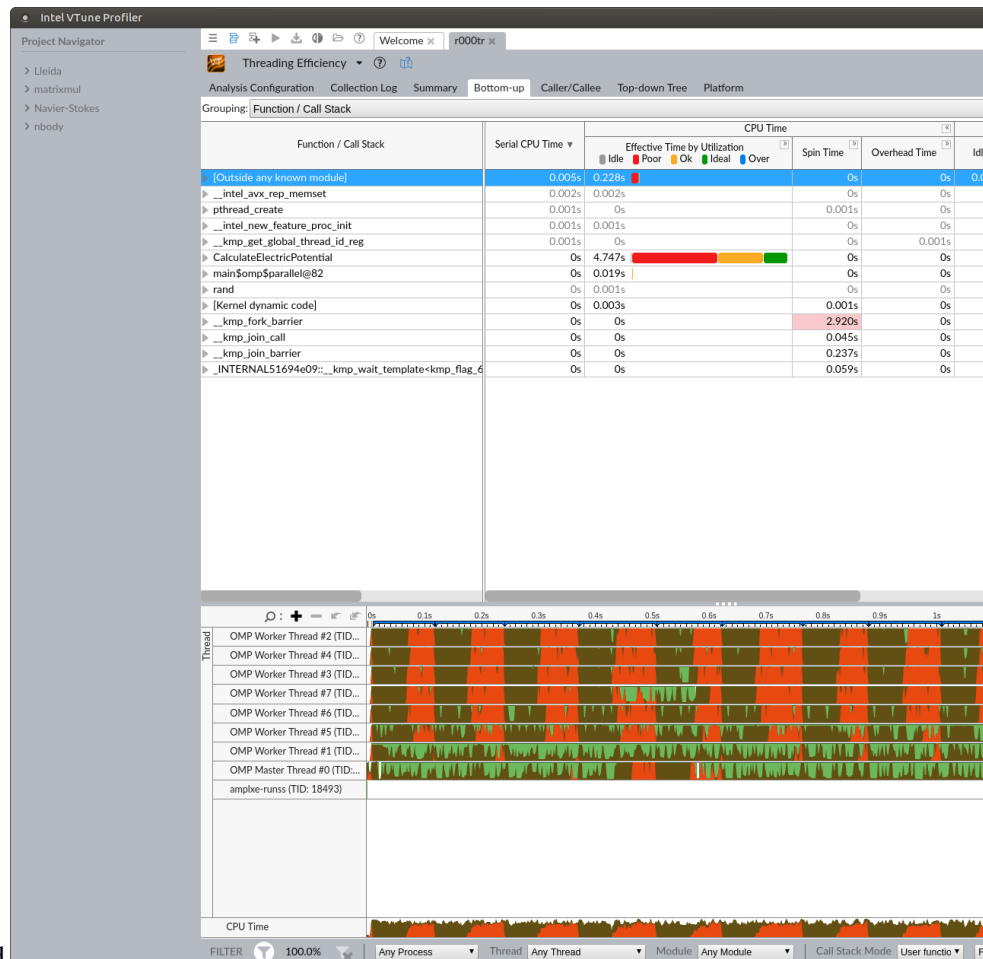


Figure 8: Eficiencia de Hilos

- schedules: static, dynamic, guided



Intel VTune: Microarchitecture

- **Microarchitecture analysis Group** introduce un tipo de análisis que ayuda a estimar los motivos de las ineficiencias en los procesadores modernos
 - *Microarchitecture Exploration* ayuda a identificar los problemas más comunes que afectan al rendimiento de la aplicación. Se puede considerar este análisis como punto de partida al análisis a nivel hw
 - *Memory Access* mide una serie de métricas para identificar accesos a los diferentes niveles de la jerarquía de memoria (como por ejemplo en arquitecturas NUMA)
- Una vez completado el análisis **Hotspots** para conocer ineficiencias en tu código...
 - Se recomienda efectuar el análisis *Microarchitecture Exploration analysis* para comprender como las ineficiencias se manifiestan en el uso del pipeline del core
 - VTune Profiler recolecta una lista complete de eventos hw
 - Calcula unas métricas predefinidas = identifica a nivel hw problemas asociados a ineficiencias
- Problemas relacionados:
 - Backend bound
 - * Uops posibles que no se puede lanzar al Fetch+Dec
 - Retiring
 - * Uops completadas (1uop/cycle)
 - Bad speculation
 - * Uops especuladas de forma incorrecta y no “retiradas”
 - Backend bound
 - * Uops completadas, pero no 1 por ciclo (fallos cache)

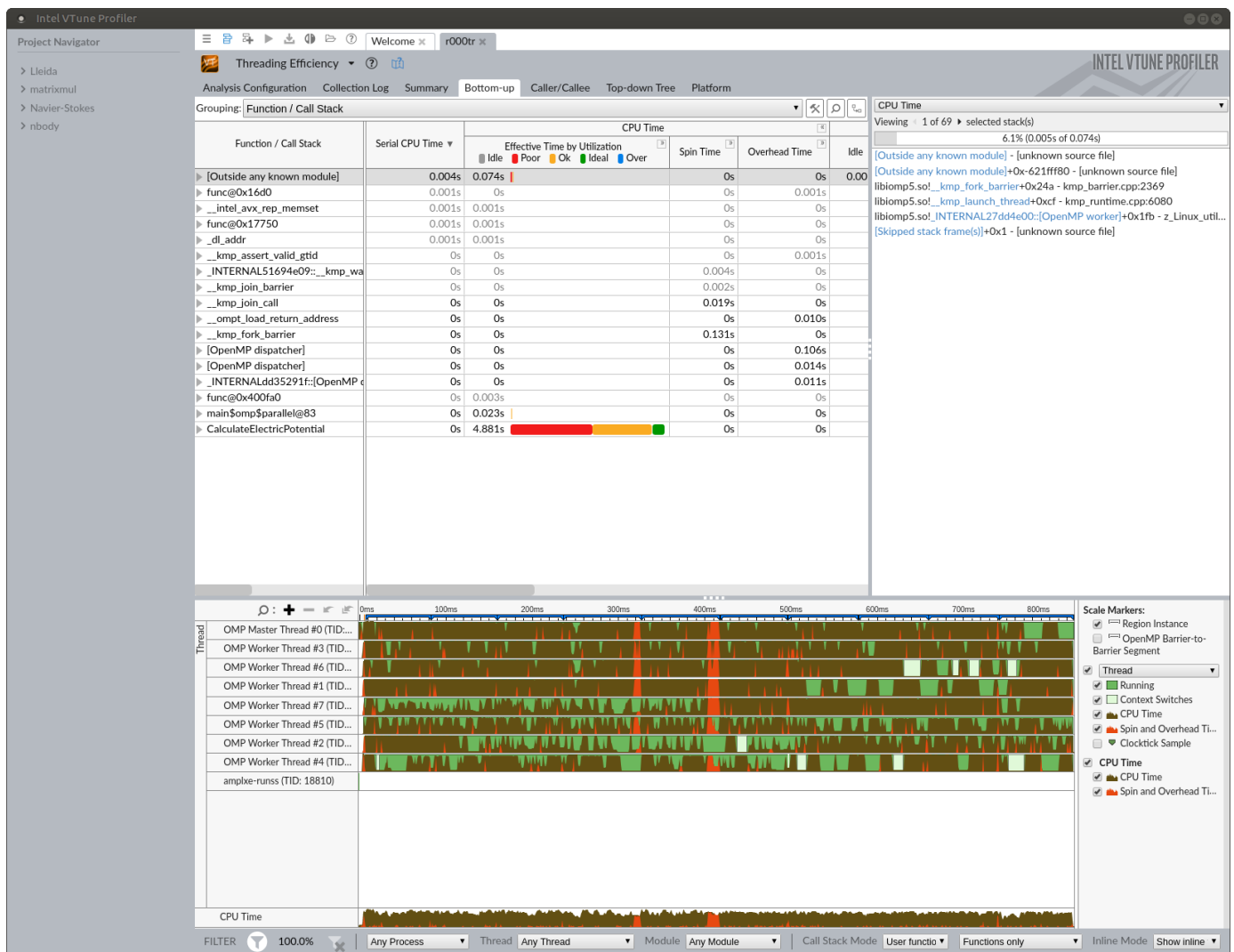


Figure 9: Ejemplo Nbody-coulomb: dynamic

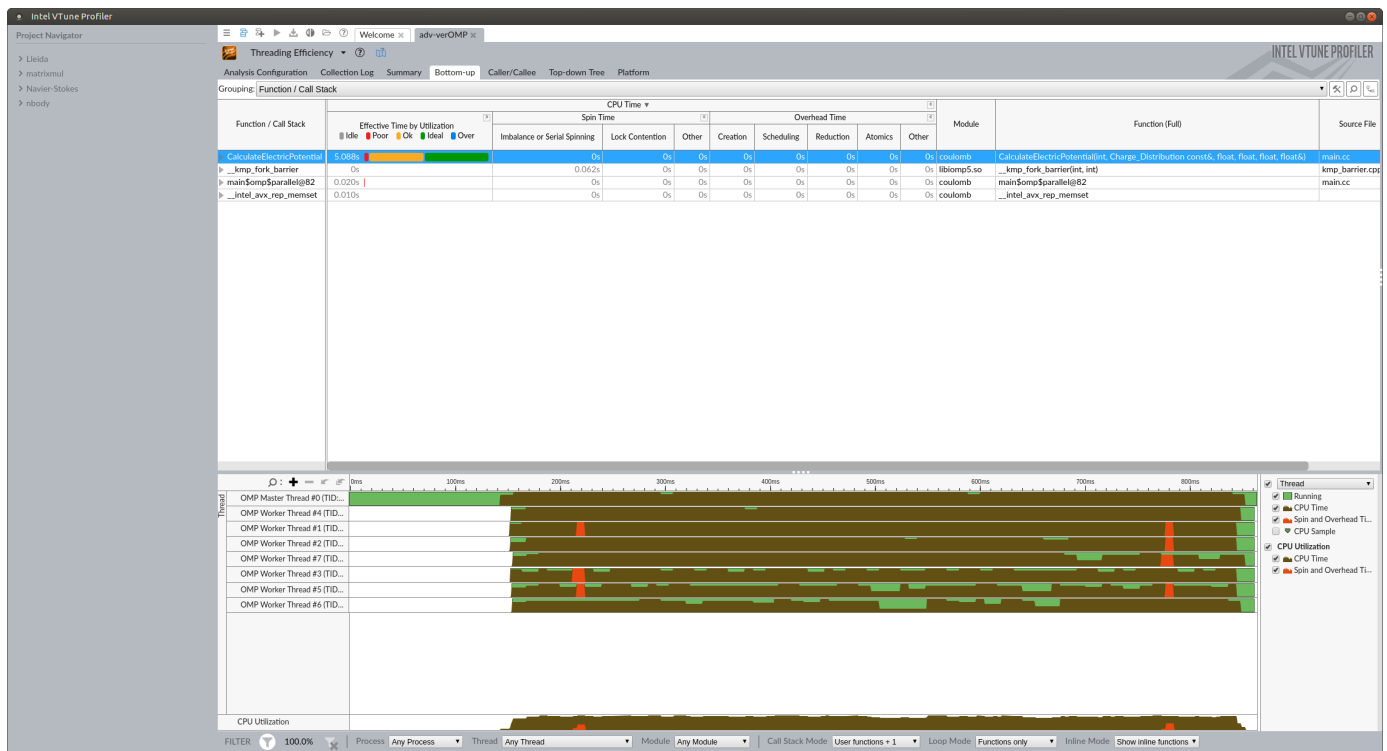


Figure 10: Ejemplo Nbody-coulomb: guided

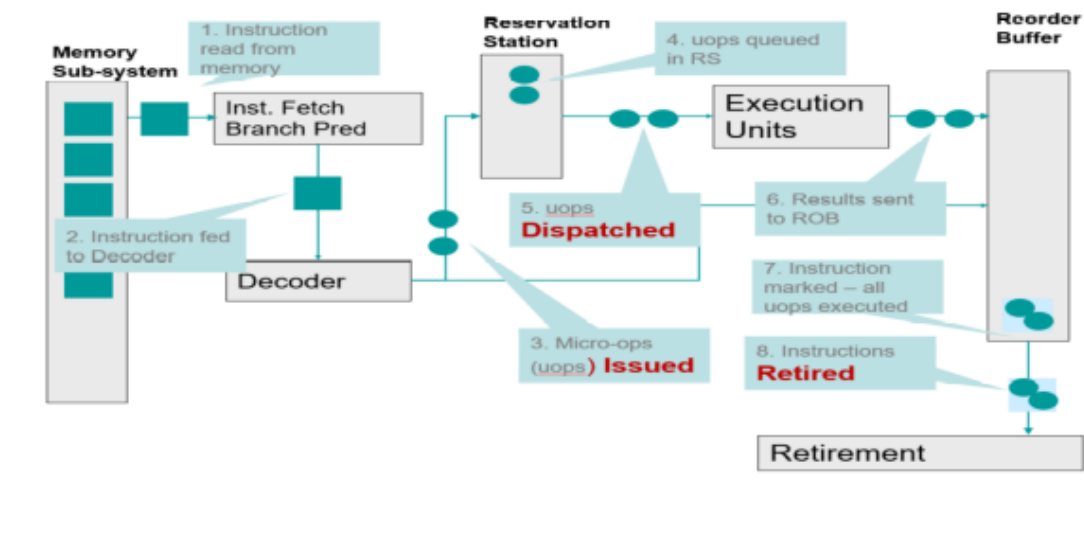


Figure 11: Flujo de Instrucciones

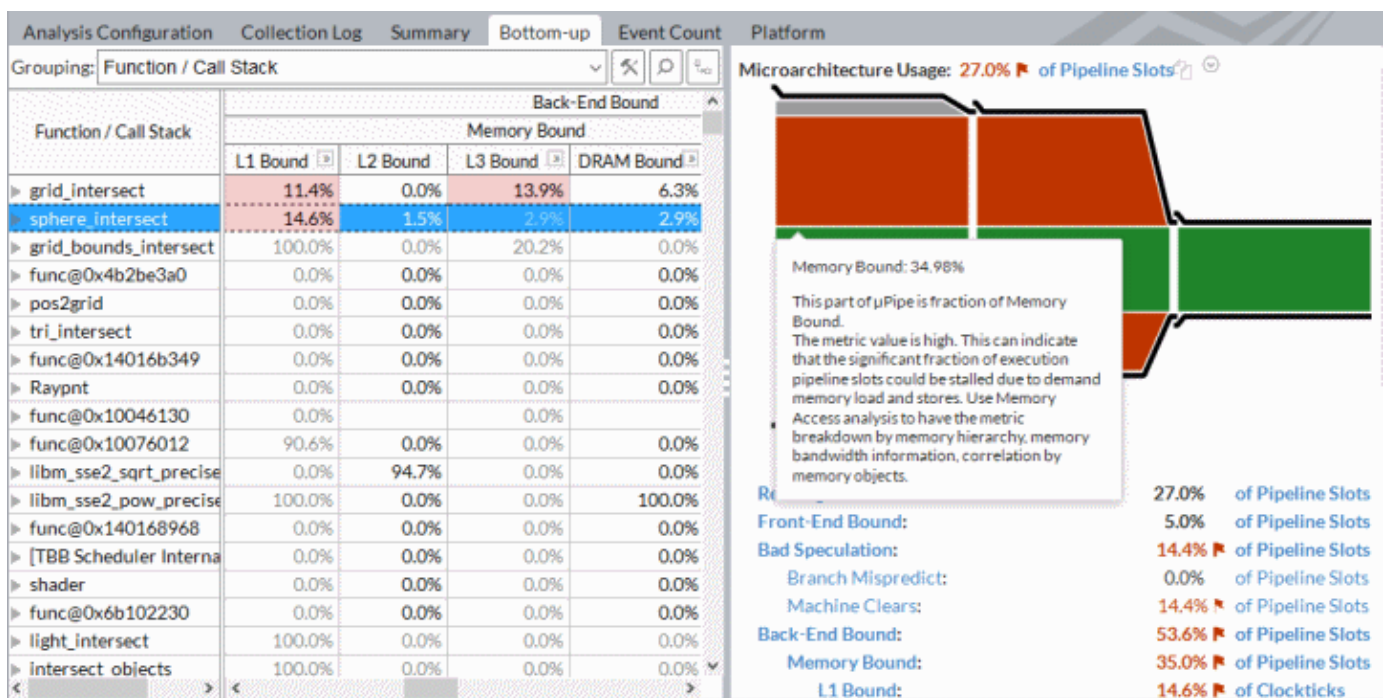


Figure 12: Análisis Microarquitectura

Práctica 3.1: Introducción a Optimización y explotación Paralelismo en GPUs

Índice

Objetivos	1
Ejercicio 1: Hello World	1
Monitorización de uso de GPU	2
Variables de entorno	4
Ejecución en GPU de NVIDIA	4
Ejemplo 2: mejoras en la programación con OpenMP para explotar el paralelismo heterogéneo	4
Maximizar el uso de recursos	6
Minimización transferencias CPU-GPU	7
Ejercicio 3: Inferencia CNN con OpenMP (entrega)	9
Arquitectura del modelo	9
Tareas del alumno	11
Recomendaciones propuestas	11
Resultado esperado	11

Objetivos

Esta práctica persigue que los estudiantes se familiaricen con la mejora de rendimiento por la explotación de paralelismo heterogéneo en GPUs mediante el uso de OpenMP. Este esquema de paralelismo es compatible otros estudiados hasta el momento en la asignatura, sin embargo nos centraremos en las cuestiones relacionadas con la descarga de trabajos (kernels) en un acelerador del tipo GPU.

Ejercicio 1: Hello World

El primer ejemplo “simple.cpp” inicializa en el host un array `data`, lo duplica en el device (construcción **target**). Entre las cuestiones a tener en cuenta está el movimiento de datos se produce con la clausula **map**. Durante las clases hemos visto como se puede indicar la direccionalidad del movimiento de datos, con(*from*, *to*, *fromto*, ect) por lo que se recomienda al alumnado repasar estos aspectos.

```
...
// Initialization
for (int i = 0; i < N; i++) data[i] = i;

// Add the target directive here, including the map clause.
#pragma omp target map(from : is_cpu) map(tofrom : data [0:N])
{
    is_cpu = omp_is_initial_device();
#pragma omp parallel for
    for (int i = 0; i < N; i++) {
        data[i] *= 2;
    }
}
```

```

    }
}
...

```

En referencia a la compilación, el compilador Intel® oneAPI DPC++/C++ Compiler tiene soporte de OpenMP. La especificación completa de OpenMP está disponible en la página web del estándar de OpenMP. El soporte para GPUs se denomina “offloading” haciendo referencia a la posible descarga de código en el acelerador. El compilador Intel® oneAPI DPC++/C++ tiene soporte de offloading mediante la activación en la fase de compilación con el flag `-fiopenmp -fopenmp-targets=spir64`. (Recordar la activación de la compilación cargando las variables de entorno correspondientes con `source /opt/intel/oneapi/setvars.sh`).

Además de forma análoga a las prácticas anteriores se puede activar un fichero de reportes para verificar que el código generado se descarga en el acelerador, añadiendo en fase de compilación el flag `-qopt-report`. En este caso además del fichero “**.optrpt**” también se genera un equivalente con el formato “**-openmp-spir64.optrpt**” donde se indica la descarga a partir de la línea 23.

```

user@lab:~ $ icpx -o simple simple.cpp -fiopenmp -fopenmp-targets=spir64 -qopt-report=2
user@lab:~ $ more simple-openmp-spir64.optrpt
*****
***** Optimization Report - Device Compilation *****
*****
Begin optimization report for: OMP TARGET outline from main line 23
...

```

Monitorización de uso de GPU

Es interesante conocer el proceso de compilación y ejecución del binario, puesto que emplea la técnica de “JIT Compilation” para en tiempo de ejecución generar el binario final para el acelerador o GPU.

Esta fase tiene un impacto directo en la ejecución del código, permitiendo la monitorización de la ejecución con la variable de entorno `LIBOMPTARGET_PLUGIN_PROFILE`. En el ejemplo se muestra este aspecto con la fase de “Compiling”, además de conocer el tiempo que conlleva el movimiento de datos: **DataRead/DataWrite** así como el tiempo del kernel: ****__omp_offloading_10303_16a5834__Z4main_l23****.

```

user@lab:~ $ LIBOMPTARGET_PLUGIN_PROFILE=T ./simple
Num devices=1
Running on GPU
0
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
=====
LIBOMPTARGET_PLUGIN_PROFILE (LEVEL_ZERO) for OMP DEVICE(0) Intel(R) Graphics, Thread 0
=====
Kernel 0                : __omp_offloading_10303_16a5834__Z4main_l23
=====

```

Just-In-Time (JIT) Compilation Flow

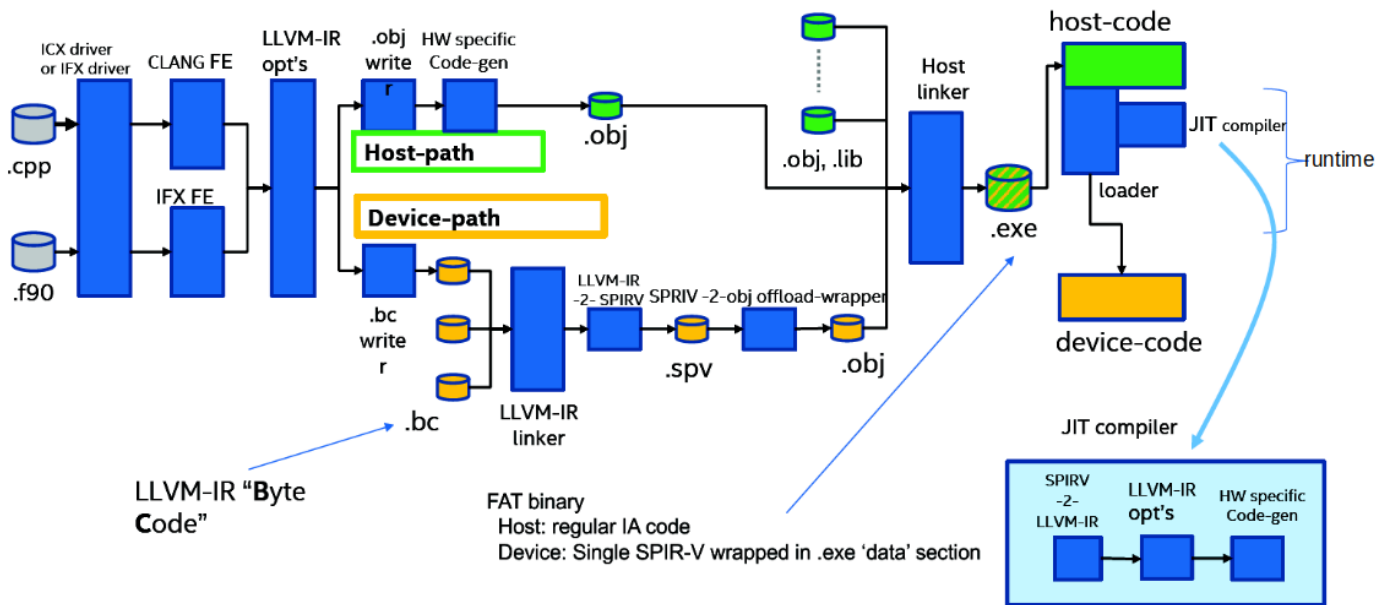


Figure 1: Flujo de compilación y ejecución de código OpenMP-target

Name		Host Time (msec)			Device Time (msec)				Min	
		Total	Average	Min	Max	Total	Average			
Compiling	:	147.31	147.31	147.31	147.31	0.00	0.00	0.00	0	0
DataAlloc	:	2.20	0.22	0.00	1.89	0.00	0.00	0.00	0	0
DataRead (Device to Host)	:	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0	0
DataWrite (Host to Device)	:	0.02	0.02	0.02	0.02	0.00	0.00	0.00	0	0
Kernel 0	:	1.43	1.43	1.43	1.43	0.01	0.01	0.01	0	0
Linking	:	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0	0
OffloadEntriesInit	:	1.94	1.94	1.94	1.94	0.00	0.00	0.00	0	0

Variables de entorno

Existen otras variables de entorno que permiten controlar la ejecución del código en los dispositivos así como activar opciones de depuración:

```
OMP_TARGET_OFFLOAD = mandatory | disabled | default
```

Donde: * mandatory: la región *target* ejecuta en GPU u otro acelerador * disabled: región *target* en CPU * default: región *target* en GPU (si hubiese), sino en CPU

En incluso existen opciones para depuración: * LIBOMPTARGET_DEBUG= [1 | 2] * Más información en el runtime del LLVM

Ejecución en GPU de NVIDIA

También existe soporte de OpenMP en gráficas de NVIDIA, con el SDK para hpc de NVIDIA El compilador NVC++ tiene soporte OpenMP offloading. Para ello, la activación de OpenMP se efectúa con el flag -mp=gpu que a su vez tiene soporte **offloading**. De igual forma la herramienta de profiling NVIDIA Nsight Systems permite monitorizar el tiempo de uso de cada kernel en la GPU de NVIDIA:

```
user@system:~$ /opt/nvidia/hpc_sdk/Linux_x86_64/2024/compilers/bin/nvc++ -o simpleNVC simple.cpp -mp=gpu
user@system:~$ nsys nvprof ./simpleNVC
....
Time (%)   Total Time (ns)   Instances   Avg (ns)   Med (ns)   Min (ns)   Max (ns)   StdDev (ns)   GridXYZ
-----
    100,0           33.024             1  33.024,0  33.024,0   33.024    33.024           0,0       30    1
....
```

Ejemplo 2: mejoras en la programación con OpenMP para explotar el paralelismo heterogéneo

El modelo de ejecución se conoce con el nombre de modelo host-device, donde un *host* (habitualmente una CPU) está conectada a un acelerador o *device* (GPU). Las GPUs tiene diferentes construcciones, y jerarquías de unidades de ejecución. En el caso concreto de las GPU de Intel la distribución se realiza de la siguiente manera: - Cada dispositivo GPU posee varias **GPU compute Units (CE)** - Cada CE posee varias **Unidades de Ejecución (EUs)** o también denominados recientemente **Xe Vector Engines (XVE)** - Existe también un modelo de memoria: memoria Global (memoria de GPU), una **memoria compartida: SLM** a nivel de CE o también denominado Slice - Además cada EUs posee unidades SIMD

En lo referente a OpenMP, existen varias niveles de pragma que permiten expresar el paralelismo: - target: descarga en el acelerador - teams: replica cómputo a nivel de “CE” o “Slice” - distributed: distribuye iteraciones a nivel de “work-group” - parallel for: distribuye iteraciones a nivel de EU - simd: distribuye iteraciones a nivel de sub-group (simdlen=8, 16, 32)

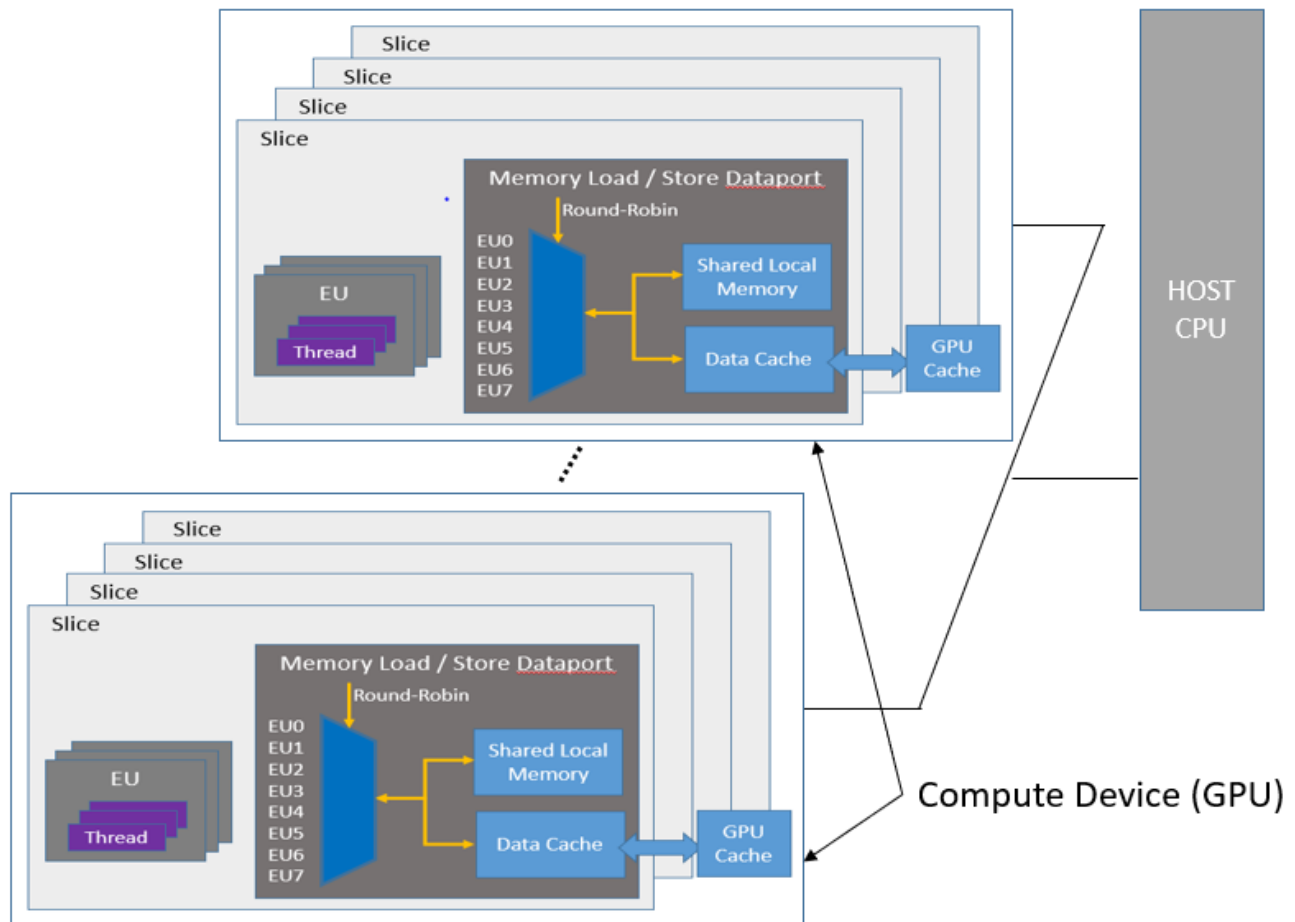


Figure 2: Fig: Modelo de ejecución en GPU

Concept	SYCL	OpenMP	CUDA
Work-item	Work-item	OpenMP thread or SIMD lane	CUDA thread
Work-group	Work-group	Team	Thread block
Work-group size	Work-group size	Team size	Thread block size
Number of work-groups	Number of work-groups	Number of teams	Number of thread blocks
Sub-group	Sub-group	SIMD chunk (simdlen = 8, 16, 32)	Warp (size = 32)
Max work-items per group	Maximum number of work-items per work-group	Thread limit	Maximum number of CUDA threads per thread block

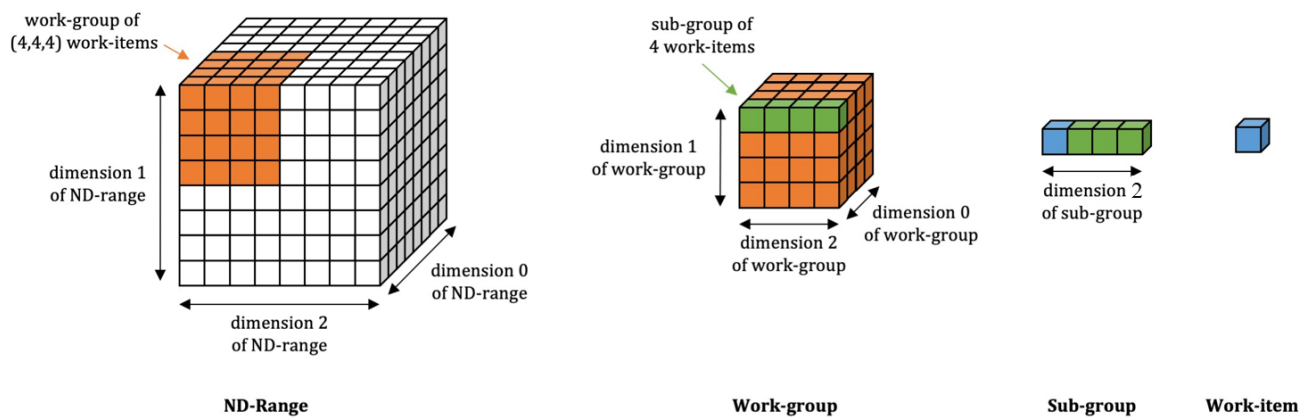


Figure 3: Fig: Relación de ND-Range, work-group, sub-group, work-item

Maximizar el uso de recursos

El código “test_no_collapse.cpp” aplica la paralelización a nivel de bucle `for` (`b = 0; b < BLOCKS; b++`) siendo **BLOCKS 4**

```

...
/* offload the kernel with no collapse clause */
#pragma omp target teams distribute parallel for \
    private(b, i, j, k, l)
for (b = 0; b < BLOCKS; b++) {
    for (i = 0; i < P; i++) {
        for (j = 0; j < P; j++) {
            for (k = 0; k < P; k++) {
                float ur = 0.;
                float us = 0.;
                float ut = 0.;

                for (int t=0 ; t < TIMES; t++)
                    for (l = 0; l < P; l++) {
                        ur += dx[IDX2(i, l)] * u[IDX4(b, l, j, k)];
                        us += dx[IDX2(k, l)] * u[IDX4(b, i, l, k)];
                        ut += dx[IDX2(j, l)] * u[IDX4(b, i, j, l)];
                    }
            }
        }
    }
}

```

```

    }
    w[IDX4(b, i, j, k)] = ur * us * ut;
  }
}
}
...

```

Evaluando la ejecución con variable entorno `LIBOMPTARGET_DEBUG=1` las iteraciones del bucle=4 porque `BLOCKS=4` se observa que el número de teams es **Number of teams = {4, 1, 1}**, es decir que **solamente 4 EUs están siendo utilizadas simultáneamente**

```

user@system:~$ icpx -fiopenmp -fopenmp-targets=spir64 test_no_collapse.cpp
user@system:~$ OMP_TARGET_OFFLOAD=MANDATORY LIBOMPTARGET_DEBUG=1 ./a.out
...
Target LEVEL_ZERO RTL --> Assumed kernel SIMD width is 16
Target LEVEL_ZERO RTL --> Preferred team size is multiple of 32
Target LEVEL_ZERO RTL --> Loop 0: lower bound = 0, upper bound = 3, Stride = 1
Target LEVEL_ZERO RTL --> Team sizes = {1, 1, 1}
Target LEVEL_ZERO RTL --> Number of teams = {4, 1, 1}
...

```

Si añadimos las cláusulas *collapse* se mejora utilización de recursos. Por ejemplo en el código “test_collapse2.cpp” se añade la cláusula **collapse(2)**, las iteraciones del bucle=8 porque `BLOCKS*P = 4*8 = 32 * SIMD: 16 * Team sizes: {1,1,1}` y **Número de teams: {8,4,1}** luego **Número total teams: {8,4,1}=32**

```

user@system:~$ icpx -fiopenmp -fopenmp-targets=spir64 test_collapse2.cpp
user@system:~$ OMP_TARGET_OFFLOAD=MANDATORY LIBOMPTARGET_DEBUG=1 ./a.out
Target LEVEL_ZERO RTL --> Assumed kernel SIMD width is 16
Target LEVEL_ZERO RTL --> Preferred team size is multiple of 32
Target LEVEL_ZERO RTL --> Loop 0: lower bound = 0, upper bound = 7, Stride = 1
Target LEVEL_ZERO RTL --> Loop 1: lower bound = 0, upper bound = 3, Stride = 1
Target LEVEL_ZERO RTL --> Team sizes = {1, 1, 1}
Target LEVEL_ZERO RTL --> Number of teams = {8, 4, 1}
...

```

De igual manera están los códigos “test_collapse3.cpp” (*Team sizes: {8,1,1}* y *Número de teams: {1,8,4}*) y “test_collapse4.cpp” (*Team sizes: {32,1,1}* y *Número de teams: {64,1,1}*) mejorando la distribución de paralelismo como se verifica en los tiempos de ejecución:

- time sin-collapse = 0.196160 s.
- time collapse(2) = 0.024447 s.
- time collapse(3) = 0.003280 s.
- time collapse(4) = 0.001072 s.

Minimización transferencias CPU-GPU

Otro aspecto a tener en cuenta a la hora de explotar el paralelismo heterogéneo es la **minimización de transferencias entre memorias del host y device**. El uso del bus PCIe que conecta ambos dispositivos redonda significativamente en el rendimiento final de una aplicación.

El código disponible “test_no_target_enter_exit_data.cpp” muestra el impacto de transferir `u[0:SIZE]`, `dx[0:P * P]`, `w[0:SIZE]` en el kernel#2, cuando ya está disponible en la memoria de la GPU tras la ejecución del kernel#1.

```

...
/* offload kernel #1 */
#pragma omp target teams distribute parallel for collapse(4) \
    map(to: u[0:SIZE], dx[0:P * P]) map(from: w[0:SIZE]) \

```

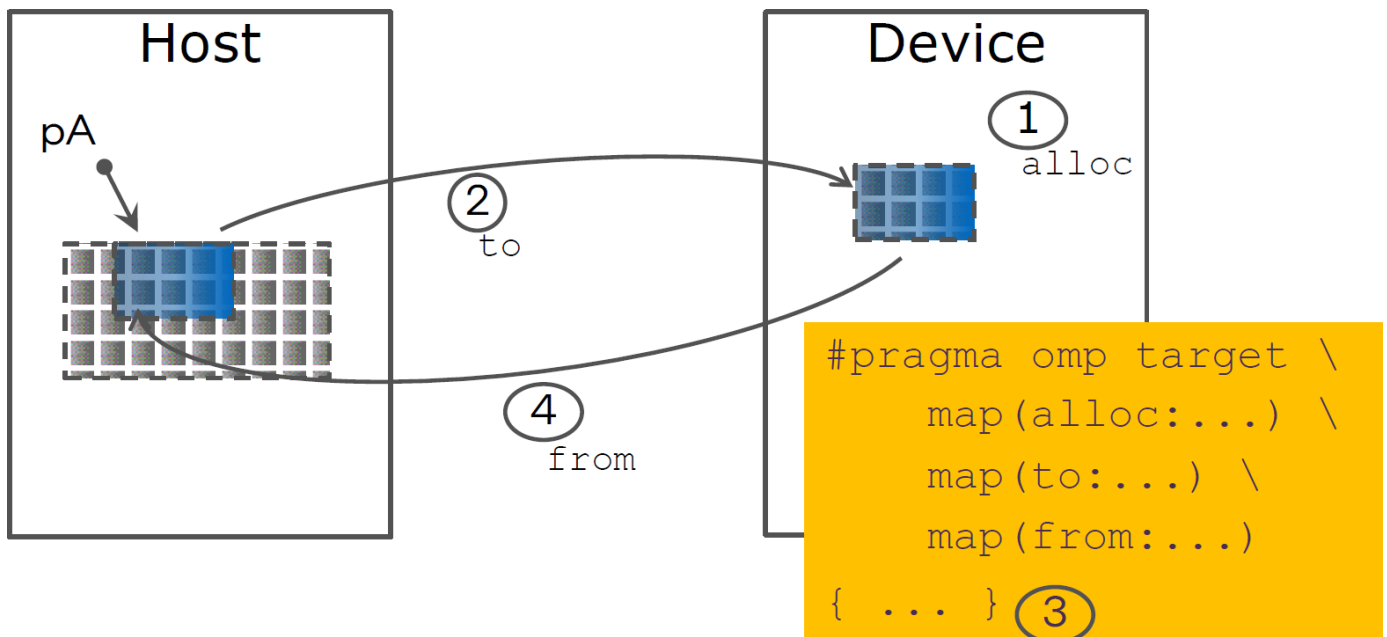


Figure 4: Fig: Transferencia entre GPU-CPU

```
private(b, i, j, k, l)
for (b = 0; b < BLOCKS; b++) {
    for (i = 0; i < P; i++) {
        for (j = 0; j < P; j++) {
            for (k = 0; k < P; k++) {
                .....
            }
        }
    }
}

/* offload kernel #2 */
#pragma omp target teams distribute parallel for collapse(4) \
    map(to: u[0:SIZE], dx[0:P * P]) map(tofrom: w[0:SIZE]) \
    private(b, i, j, k, l)
for (b = 0; b < BLOCKS; b++) {
    for (i = 0; i < P; i++) {
        for (j = 0; j < P; j++) {
            for (k = 0; k < P; k++) {
                .....
            }
        }
    }
}
...

```

```
user@system:~$ OMP_TARGET_OFFLOAD=MANDATORY LIBOMPTARGET_DEBUG=1 ./a.out &> test_no_target_enter_exit_data
user@system:~$ grep "omptarget --> Moving" test_no_target_enter_exit_data.debug
omptarget --> Moving 8192 bytes (hst:0x00007ffea8b9a1c0) -> (tgt:0x00007598cec50000)
omptarget --> Moving 256 bytes (hst:0x00007ffea8b9e1c0) -> (tgt:0x00007598cc6d0000)
omptarget --> Moving 8192 bytes (tgt:0x00007598cec52000) -> (hst:0x00007ffea8b9c1c0)
omptarget --> Moving 8192 bytes (hst:0x00007ffea8b9c1c0) -> (tgt:0x00007598cec52000)
omptarget --> Moving 8192 bytes (tgt:0x00007598cec52000) -> (hst:0x00007ffea8b9c1c0)
```

[!WARNING]

¿hace falta en el kernel#2 enviar 'u', 'dx' y 'w'?

- Usar `omp target enter data map` y `omp target exit data map` como en el código “test_target_enter_exit_data.cpp”

```
...
/* map data to device. alloc for w avoids map(tofrom: w[0:SIZE]) on target by default. */
#pragma omp target enter data map(to: u[0:SIZE], dx[0:P * P]) \
    map(alloc: w[0:SIZE])

/* offload kernel #1 */
...

/* offload kernel #2 */
...

#pragma omp target exit data map(from: w[0:SIZE])
...
```

Ejercicio 3: Inferencia CNN con OpenMP (entrega)

El objetivo de esta práctica es modificar un código en C++ que realiza inferencia sobre imágenes del dataset MNIST, para añadir soporte de **paralelismo heterogéneo** mediante **OpenMP-offloading**.

El punto de partida es el fichero fuente `cnn.cpp`, el cual sigue el siguiente flujo de trabajo:

1. **Carga los pesos** del modelo previamente entrenado.
2. **Infiere el modelo** sobre una imagen del dataset MNIST.
3. **Devuelve la predicción**, reconociendo el número escrito a mano (del 0 al 9).

Arquitectura del modelo

El modelo implementado es una red neuronal convolucional (CNN) **sencilla**, compuesta por las siguientes capas:

- Conv2D: una convolución con una máscara de tamaño **3x3**.
- ReLU: función de activación rectificada.
- MaxPool2D: operación de reducción espacial (submuestreo).
- Fully Connected (FC): capa densa que produce la **probabilidad** de que la imagen corresponda a un dígito entre el **0 y el 9**.

El modelo implementado está basado en una arquitectura similar a propuesta en PyTorch en las siguiente líneas de código:

```
import torch
import torch.nn as nn
# Define the model
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=1, kernel_size=3, padding=1) # 28x28 -> 28x28
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2) # 28x28 -> 14x14
        self.fcl = nn.Linear(1 * 14 * 14, 10) # Fully Connected Layer (10 classes)
```

La siguiente imagen muestra un diagrama de la arquitectura del modelo seguido que puede resumirse en el procesado de las capas: [Input] → [Conv2d] → [ReLU] → [MaxPool2d] → [FullyConected] → [Output]

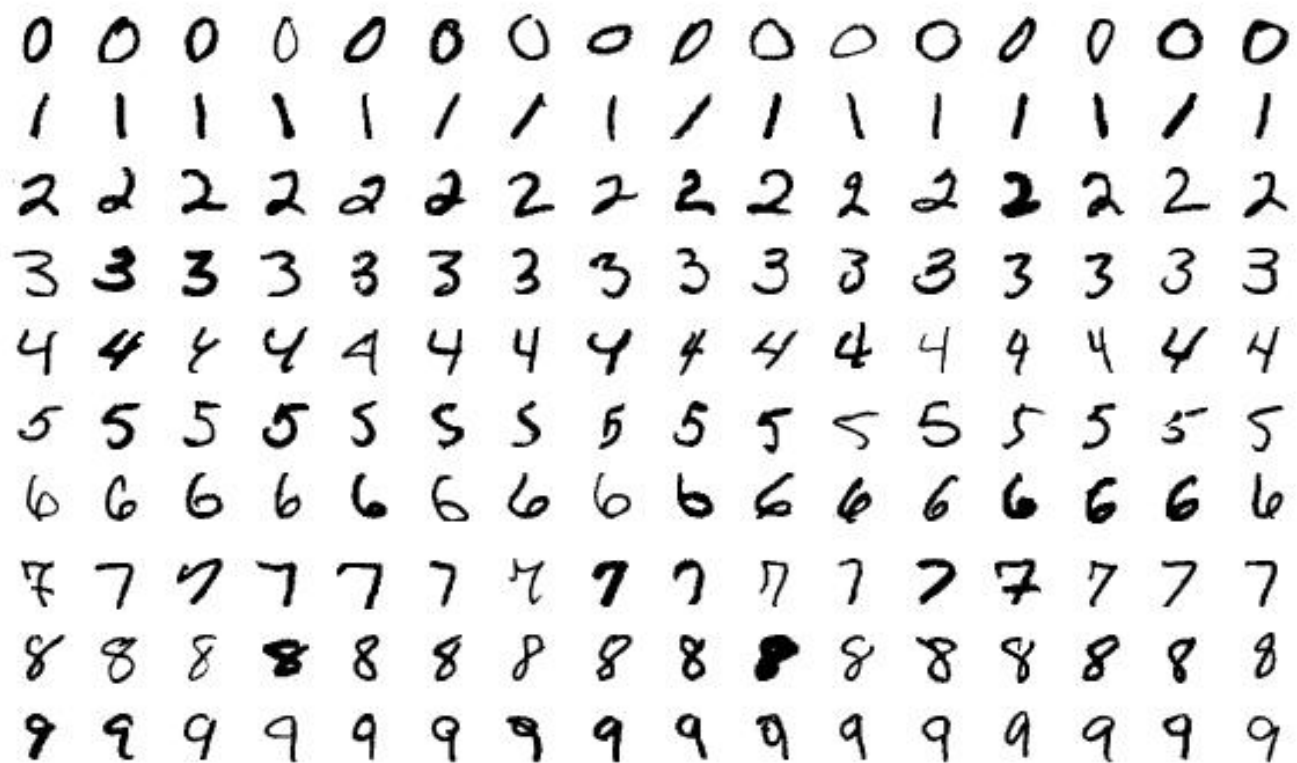


Figure 5: Fig: La base de datos MNIST (Modified National Institute of Standards and Technology) es una gran colección de dígitos escritos a mano. Contiene un conjunto de entrenamiento con 60,000 ejemplos y un conjunto de prueba con 10,000 ejemplos.

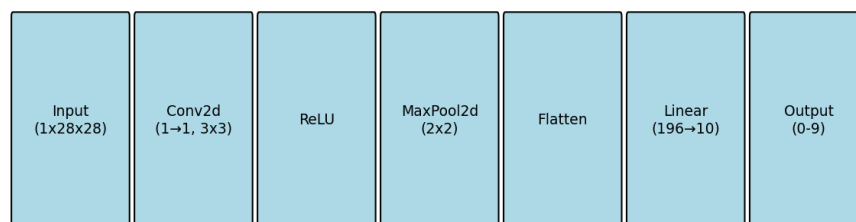


Figure 6: Fig: Arquitectura del modelo CNN

Tareas del alumno

El estudiante debe **optimizar y paralelizar** las funciones críticas del proceso de inferencia utilizando **OpenMP target offloading**, para ello se centrará en el código que implementan las funciones Conv2d+ReLU, MaxPool2d y FullyConnected que aparecen resaltadas a continuación:

```
conv2d(input, conv_out, conv1_weights, height, width, *conv1_biases);  
  
maxpool2d(conv_out, pooled, height, width);  
  
fully_connected(pooled, fcl_weights, fcl_biases, output, size_fcl_biases, fc_input_size);
```

Para identificar con más sencillez dicha parte del código, estas líneas corresponde a la monitorización del tiempo de ejecución del modelo entre las llamadas a la función `get_ms()` que captura el tiempo del sistema.

Recomendaciones propuestas

Para facilitar una implementación eficaz y comprensible, se presentan las siguientes recomendaciones:

1. Separación en kernels independientes:

Cada una de las funciones (`conv2d`, `maxpool2d`, `fully_connected`) tiene dependencias de datos con las demás. Por ello, deben ser tratadas como **kernels independientes** en el contexto de OpenMP target offloading. Es decir, cada función debe ser procesada offloading por separado, respetando el orden lógico de ejecución.

2. Minimización de transferencias CPU-GPU:

Recuerda que transferir datos entre CPU y GPU tiene un coste elevado. OpenMP permite controlar esto con la cláusula `map`. Por ejemplo, `conv2d` genera el array `conv_out`, que es inmediatamente usado por `maxpool2d`. **Evita volver a mapear datos que ya están en el dispositivo.**

3. Aprovechamiento del paralelismo en la GPU:

La cláusula `target` por sí sola solo descarga el kernel al dispositivo. Para sacar el máximo rendimiento, es fundamental añadir otras cláusulas como `teams`, `distribute`, `parallel for`, `simd` etc., que permiten explotar toda la jerarquía de paralelismo de la GPU.

4. Uso de `collapse` para fusionar bucles:

En muchos algoritmos, los bucles anidados son candidatos a ser “fusionados” para paralelizar más iteraciones de manera efectiva. Usa la cláusula `collapse(N)` para fusionar N niveles de bucles cuando sea coherente con la lógica del algoritmo.

5. Precaución con las reducciones:

Las operaciones de reducción (sumas acumulativas, o cálculo de valores máximos; por ejemplo) no suelen ser eficientes en GPU, y a veces tampoco en CPU. **Evítalas si es posible**, aunque eso signifique reducir la forma de expresar el paralelismo, suele tener más impacto en el rendimiento una reducción que una posible ganancia extra por poder expresar más paralelismo.

6. Escalabilidad limitada por el problema:

El modelo CNN utilizado es intencionadamente simple, para que pueda ser optimizado sin mucho tiempo de desarrollo, además la imagen de entrada es bastante pequeña (28x28 píxeles). Por tanto, no esperes grandes mejoras de rendimiento al usar GPU. **El objetivo es más pedagógico-uso de GPU-que de ganancia en el rendimiento.**

Resultado esperado

Se espera que el alumno sea capaz de:

- Implementar correctamente el offloading con OpenMP, que los dígitos reconocidos sean los mismo que la versión secuencial.
- Evaluar los sobrecostes mediante el uso de herramientas de perfilado o herramientas que informen del paralelismo expresado en la GPU, sobre-coste de trasiego de datos. . . .
- Comprender las ventajas del paralelismo heterogéneo en la inferencia de modelos de deep learning.

Práctica 3.2: Programación de OMP en Python

Índice

Objetivos	1
Introducción a PyOMP	1
Instalación	1
Ejercicio 1: Hello World	2
Ejercicio 2: Explotación de Hilos con OpenMP	2
Ejercicio 3: Explotación de GPU con OpenMP	4
Ejercicio 4: Kmeans explotación multihilo en CPU (entrega)	4
Ejercicio 5: Multiplicación matrices en GPU (entrega)	5
Resultado esperado	5

Objetivos

- Familiarizarse con PyOMP
- Relacionar OpenMP con PyOMP así como la corelación entre las directivas `parallel`, `for`, `task` y `target`
- Evaluar el rendimiento de las soluciones paralelas frente a las versiones secuenciales

Introducción a PyOMP

PyOMP es una interfaz que permite integrar la paralelización mediante OpenMP directamente desde Python, ofreciendo al desarrollador una herramienta sencilla pero potente para acelerar la ejecución de código utilizando múltiples hilos del procesador. A través de esta librería, es posible paralelizar tareas intensivas en cómputo, como operaciones matriciales y ciclos repetitivos, aprovechando efectivamente los recursos disponibles en CPU o GPU.

Una de las grandes ventajas de PyOMP es que facilita la implementación de instrucciones paralelas en código Python, que habitualmente es secuencial. Esto permite obtener importantes ganancias en rendimiento sin tener que recurrir a lenguajes tradicionalmente usados para alto rendimiento, como C o Fortran. En combinación con herramientas como Numba, PyOMP puede generar códigos numéricos optimizados, manteniendo la simplicidad y legibilidad del código Python original.

Instalación

- La instalación está soportada a través de entornos conda como viene descrito en el repositorio. Antes de nada vamos a crear un entorno con la versión 3.9 de python, para posteriormente activar el entorno e instalar los paquetes de PyOMP:

```
# Crear entorno
user@host% conda create -n PyOMP python=3.9

# Activación
```

```
user@host% conda activate PyOMP

# Instalar paquetes
(PyOMP) user@host% conda install -c python-for-hpc -c conda-forge pyomp
```

Ejercicio 1: Hello World

El primer ejemplo “hello.py” utiliza la librería Numba que permite acelerar la ejecución de funciones numéricas, mediante la compilación de código Python a código máquina optimizado con el mecanismo Just-in-Time, o JIT, lo que a su vez el estándar de compilación del LLVM permitiendo la interacción de Python con lenguajes compilados como C/C++ o Fortran.

Es importante destacar en el código el decorador `@njit` que permite compilar la función `hello()` usando Numba, mejorando su rendimiento. Además este aspecto permite la utilización de librerías de paralelización como las estudiadas en la asignatura como OpenMP con el decorador `with openmp("parallel")` : que inicia una región paralela utilizando OpenMP. Esto significa que las instrucciones dentro de este bloque pueden ejecutarse simultáneamente por múltiples hilos.

```
from numba import njit
from numba.openmp import openmp_context as openmp

@njit
def hello():
    with openmp("parallel"):
        print("hello")
        print("world")
hello()
print("DONE")
```

Al igual que en la práctica de OpenMP, se puede controlar el número de hilos con la variable de entorno `OMP_NUM_THREADS` como se puede observar en la siguiente ejecución:

```
(PyOMP) user@host% OMP_NUM_THREADS=2 python3 codes/hello_world.py
hello
world
hello
world
Done!!
```

Ejercicio 2: Explotación de Hilos con OpenMP

- Vamos a tratar el ejemplo de cálculo de π como aproximación de la integral:

$$\pi = \int_0^1 \frac{4}{1+x^2} \delta x$$

- Podemos aproximar la integral como la suma de los rectángulos
 - donde cada rectángulo tiene de ancho Δx y la altura $F(x_i)$

$$\pi \simeq \sum_{i=0}^{N-1} F(x_i) \Delta x$$

La versión secuencial recogida en el código `pi_serie.py` es el punto de partida más sencillo para calcular π mediante integración numérica usando la regla del punto medio. Este algoritmo consiste en sumar áreas de rectángulos bajo una curva determinada. En Python, esta implementación es simple y utiliza un único ciclo iterativo, lo cual limita su velocidad al uso de un solo núcleo del procesador.

```
def piFunc(NumSteps):
    step=1.0/NumSteps
    sum = 0.0
```

```

x = 0.5
for i in range(NumSteps):
    x+=step
    sum += 4.0/(1.0+x*x)
pi=step*sum
return pi

```

La versión de OpenMP que explota la distribución de iteraciones entre los hilos OpenMP recogida en el código `pi_omp_loop.py`, emplea la directiva `parallel` for implementando una reducción eficiente de la variable `sum`:

```

@njit
def piFunc(NumSteps):
    step = 1.0/NumSteps
    sum = 0.0
    with openmp("parallel for private(x) reduction(+:sum)"):
        for i in range(NumSteps):
            x = (i+0.5)*step
            sum += 4.0/(1.0 + x*x)
    pi=step*sum
    return pi

```

El código `pi_omp_spmc.py` explota el paralelismo SPMD (Single Program Multiple Data) distribuyendo manualmente bloques de iteraciones entre diferentes hilos. Cada hilo realiza su suma parcial, almacenando los resultados en un arreglo que luego se combina para obtener el valor final. Esta estrategia permite control explícito sobre cómo se asignan las iteraciones a cada hilo.

```

@njit
def piFunc(NumSteps):
    step = 1.0/NumSteps
    partialSums = np.zeros(NTHREADS)
    with openmp("parallel shared(partialSums) private(threadID,numThrds,i,x,localSum) num_threads(8)"):
        threadID = omp_get_thread_num()
        numThrds = omp_get_num_threads()
        localSum = 0.0
        for i in range(threadID, NumSteps, numThrds):
            x = (i+0.5)*step
            localSum = localSum + 4.0/(1.0 + x*x)
        partialSums[threadID] = localSum
    return step*np.sum(partialSums)

```

Por último la versión de tareas o `tasks` del fichero `pi_omp_tasks.py` divide recursivamente el problema en subproblemas más pequeños. Cada tarea se asigna dinámicamente a los hilos disponibles, facilitando la gestión dinámica del balance de carga, especialmente útil en sistemas heterogéneos o cuando el número de iteraciones es muy grande.

```

MIN_BLK = 1024
@njit
def piComp(Nstart, Nfinish, step):
    iblk = Nfinish-Nstart
    if (iblk<MIN_BLK):
        sum = 0.0
        for i in range(Nstart,Nfinish):
            x= (i+0.5)*step
            sum += 4.0/(1.0 + x*x)
    else:
        sum1 = 0.0
        sum2 = 0.0
        with openmp ("task shared(sum1)"):
            sum1 = piComp(Nstart, Nfinish-iblk/2,step)
        with openmp ("task shared(sum2)"):
            sum2 = piComp(Nfinish-iblk/2,Nfinish,step)

```

```

        with openmp ("taskwait"):
            sum = sum1 + sum2
    return sum

@njit
def piFunc(NumSteps):
    step = 1.0/NumSteps
    sum = 0.0
    with openmp ("parallel"):
        with openmp ("single"):
            sum = piComp(0, NumSteps, step)
    pi = step*sum
    return step*sum

```

Ejercicio 3: Explotación de GPU con OpenMP

El ejemplo `axpy.py` utiliza la operación matemática básica conocida como AXPY (que significa “A times X Plus Y”), para ilustrar una implementación descargando un código a la GPU. Para ello al igual que en OpenMP la directiva `omp target` descarga el kernel al acelerador.

Pero antes se ha de trasladar la información necesaria a la memoria del acelerador con la cláusula `map`.

```

@njit
def axpy_gpu(x, y, a, N):
    with openmp("target map(tofrom: y) map(to: x)"):
        for i in range(N):
            y[i] = a*x[i] + y[i]

    return y

```

De igual manera que en ejercicio anterior se muestra el código relativo al cálculo de π realizando el cómputo en la GPU mediante la directiva `target` en el fichero `pi_target.py`. En este caso se pone el foco en la distribución de la carga de trabajo entre los diferentes niveles jerárquico de la GPU (SMs, CUDA-core...):

```

@njit
def piFunc(NumSteps):
    step = 1.0/NumSteps
    sum = 0.0
    with openmp("target teams distribute parallel for private(x) reduction(+:sum)"):
        for i in range(NumSteps):
            x = (i+0.5)*step
            sum += 4.0/(1.0 + x*x)

    pi = step * sum
    return pi

```

Ejercicio 4: Kmeans explotación multihilo en CPU (entrega)

K-means es un algoritmo de aprendizaje no supervisado utilizado para resolver problemas de clustering o agrupamiento. Su objetivo es particionar un conjunto de datos en K grupos (clusters) de tal forma que los elementos dentro de un mismo grupo sean lo más similares posible entre sí, y lo más distintos posible de los elementos de otros grupos.

El funcionamiento básico del algoritmo sigue los siguientes pasos:

1. Inicialización: Se seleccionan aleatoriamente K centroides iniciales (puntos representativos de cada grupo).
2. Asignación: Cada punto del conjunto de datos se asigna al centroide más cercano (según distancia euclídea u otra métrica).
3. Actualización: Se recalculan los centroides como el promedio de los puntos asignados a cada grupo.

4. Repetición: Los pasos de asignación y actualización se repiten hasta que los centroides no cambian significativamente o se alcanza un número máximo de iteraciones.

El código `kmeans_serial.py` lee un dataset de un fichero tipo CSV, e invoca a la función `kmeans` donde para un número de iteraciones, calcula la distancias a los centroides para asignar la clase con la función `compute_distances` y actualiza los centroides con `update_centroids`.

La implementación paralela de ambas funciones tiene los siguiente retos:

- `compute_distances` ofrece una explotación de paralelismo mediante hilo evidente en el primer de los bucles `for i in range(n_samples)` puesto que calcula la distancia al centroide menor y le asigna dicha etiqueta
- `update_centroids` presenta el problema de reducción ya estudiado a lo largo de la asignatura, por un lado se acumulan los puntos asociado a cada cluster en `counts[c]` (reducción) para posteriormente calcular la nueva posición del centroide como valor medio las posiciones de los puntos del dataset que pertenecen a ese cluster `new_centroids[j, f] /= counts[j]` (reducción).

Ejercicio 5: Multiplicación matrices en GPU (entrega)

La multiplicación de matrices es una operación fundamental del álgebra lineal donde dos matrices se combinan para generar una tercera.

La multiplicación de matrices aparece constantemente en tareas de ciencia de datos y aprendizaje automático, ya que permite representar cálculos complejos de forma compacta y eficiente:

- Regresión Lineal y Logística: las predicciones se obtienen calculando una suma ponderada de las variables de entrada: $y = \Phi X$, donde X es la matriz de características y Φ el vector de parámetros
- Redes Neuronales Convolucionales (CNNs): utilizan operaciones de convolución, que internamente se transforman en multiplicaciones de matrices.
- Reducción de Dimensionalidad por Componentes Principales (PCA): requieren calcular matrices de covarianza, autovalores y proyecciones, todos ellos basados en productos matriciales.

Debido a que la multiplicación de matrices es tan común y costosa computacionalmente, optimizar su implementación (por ejemplo, usando bibliotecas como BLAS, cuBLAS o MKL) tiene un impacto directo en la velocidad de entrenamiento e inferencia de modelos.

Resultado esperado

Al finalizar esta práctica, el alumno será capaz de:

- Comprender el funcionamiento básico de PyOMP y cómo se integra con Numba para aprovechar la paralelización en CPU y GPU desde Python
- Explotar el paralelismo de hilos con la directiva `parallel`
- Explotar el paralelismo heterogéneo en GPU mediante la directiva `target`

Práctica 3.3: Programación de paralela en Python

Índice

Objetivos	1
Introducción a oneAPI	1
Instalación y setup de Intel Distribution for Python	3
Ejercicios	4
Ejercicio 1: Vectorización en numpy	4
Uso de paralelismo	4
Paquetes extra	5
Ejercicio 2: Uso de numpy	5
Ejercicio 3: Uso de DPNP y el Data Parallel Extension en Python	5
Extensión de DataParallel para Numpy - dpnp	7
Contenido del laboratorio	7
Ejercicio 4	8
Ejemplo inicial	8
Ejemplo 1 (cálculo de π con el método de Montecarlo)	8
Ejemplo 2 (Kmeans)	8
Resultados esperado	10

Objetivos

- Comprender conceptos básicos de paralelismo en Python y relacionarlo con la librería Numpy
- Explotación de aceleradores tipo GPU con Python
- Uso de bibliotecas como Numpy, DPNP para realizar operaciones vectoriales y paralelas

Introducción a oneAPI

oneAPI es una es una plataforma de desarrollo abierta y unificada creada por Intel, diseñada para permitir que los desarrolladores escriban código una sola vez y lo ejecuten eficientemente en diversos tipos de hardware, como: CPUs (procesadores tradicionales), GPUs (tarjetas gráficas), FPGAs (chips reconfigurables) u otro tipo de aceleradores.

Incluye desde compiladores (ya usados en la asignatura como el Intel® oneAPI DPC++/C++ Compiler), librerías optimizadas (oneMKL, oneDNN, oneTBB...), que soportan modelos de programación de HPC y herramientas de perfilado como Intel Advisor o VTune.

El conjunto herramientas de desarrollo permiten trabajar desde CPU a XPU, y están disponibles en varios Toolkits para su uso.

Para esta práctica nosotros nos centraremos en la AI Tools que incorpora:

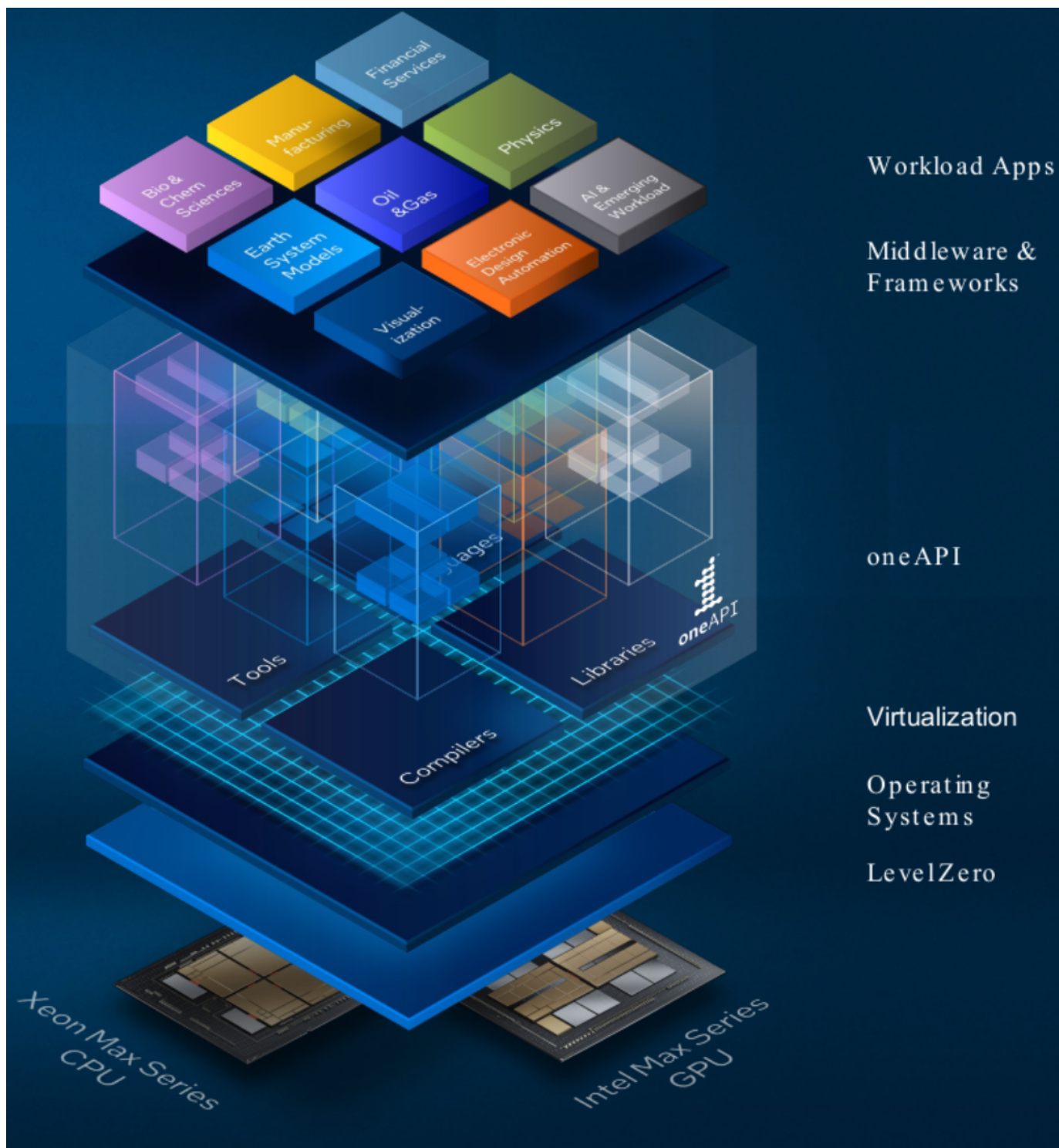


Figure 1: Fig: Modelo de programación unificado

intel AI TOOLS 1 oneAPI	Data Analytics at Scale:	Modin*	pandas*	NumPy*	SciPy*			
	DL Inference and Training:	TensorFlow*	PyTorch*	OpenVINO™	Intel® Neural Compressor			
	Classical ML:	Scikit-learn*	XGBoost*	Python*				
intel HPC TOOLKIT 1 oneAPI	Intel® Fortran CompilerIntel® MPI LibraryIntel® SHMEM Library							
intel 1 oneAPI BASE TOOLKIT	Tools	Intel® DPC++/C++ Compatibility Tool	Intel® VTune™ Profiler	Intel® Advisor	Intel® Distribution for GDB	Intel® Distribution for Python		
	Performance Libraries:	oneMKL	oneDNN	oneDAL	oneCCL	oneTBB	oneDPL	Intel® IPP
	Direct Programming:	C++ with SYCL*		C++	Python*	OpenMP*		
	Compilers:	Intel® oneAPI DPC++/C++ Compiler						
	Hardware Interface – oneAPI Level Zero & OpenCL*							

Figure 2: Fig: Toolkits y herramientas disponibles en oneAPI

- Intel® Distribution for Python: rendimiento cercano al del código nativo con este conjunto de paquetes esenciales
- Intel® Extension for PyTorch (CPU & GPU): extensión para PyTorch
- Intel® Extension for TensorFlow* (CPU & GPU): para lograr alto rendimiento en XPU en entornos Tensorflow
- Intel® Extension for Scikit-learn: para acelerar aplicaciones del ámbito de la biblioteca Scikit-learn
- Intel® Neural Compressor: para reducir los modelos de IA y acelerar el despliegue en CPUs y GPUs empleando técnicas de compresión de modelos, cuantización y pruning
- OpenVINO™ Toolkit: para facilitar el despliegue de modelos de IA en GPUs, CPUs y NPUs

En este lab nos vamos a centrar en la **distribución de Python optimizada: Intel Distribution for Python**

Instalación y setup de Intel Distribution for Python

Intel® Distribution for Python permite aprovechar todos los núcleos de CPU y las instrucciones más recientes para obtener un rendimiento escalable y cercano al nativo, especialmente en tareas numéricas y de aprendizaje automático. Gracias a extensiones como **dpnp**, es posible ejecutar código Python de forma acelerada en CPUs y GPUs sin necesidad de programar a bajo nivel. Estas herramientas están pensadas tanto para desarrolladores de IA, científicos de datos y expertos en HPC, como para estudiantes que desean aprender programación de alto rendimiento con Python. La plataforma integra bibliotecas optimizadas como oneMKL y herramientas de paralelismo como OpenMP

Para su instalación hay que descargar el paquete en Linux desde la URL y se obtendrá un fichero del tipo “**intelpython3-version-Linux-x86_64.sh**”. Para su instalación, seguir los siguientes pasos:

1. En una consola se ejecutar el script descargado `sh ./intelpython3-version.sh`
2. Presiona ENTER para aceptar las condiciones de la licencia
3. Selecciona la ruta de instalación o pulsar ENTER para instalarlo en la ruta por defecto

```
user@host% sh ./intelpython3-2025.1.0_196-Linux-x86_64.sh
```

```
Welcome to intelpython3 2025.1.0_196
```

```
In order to continue the installation process, please review the license agreement.
Please, press ENTER to continue
```

```
>>>
```

```
...
Do you accept the license terms? [yes|no]
>>> yes
...

intelpython3 will now be installed into this location:
/home/XXXX/intelpython3 >>>
```

Una vez instalado se puede usar la distribución de python de Intel cargando las variables de entorno con el comando `source env/vars.sh` desde la ruta donde se ha instalado por defecto. Si todo fue correcto el interprete de Python será **Python 3.12.8 | Intel Corporation:**

```
user@host: ~/intelpython3$ source env/vars.sh
user@host: ~/intelpython3$ python3
Python 3.12.8 | Intel Corporation | (main, Feb 13 2025, 23:28:03) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
```

A continuación vamos a instalar el entorno de **Jupyter** en el nuevo interprete del Intel Distribution for Python con el comando `python3 -m pip install jupyter` y podremos arrancar el entorno de Jupyter usando el nuevo intérprete con `python3 -m jupyter notebook`

```
user@host: ~/intelpython3$ source env/vars.sh
user@host: ~/intelpython3$ python3 -m pip install jupyter
....
user@host: ~/intelpython3$ python3 -m jupyter notebook
```

Ejercicios

Ejercicio 1: Vectorización en numpy

La explotación de la vectorización no es una teoría Es altamente recomendable utilizar librerías optimizadas en Intel oneAPI como NumPy, SciPy y otra

- Numpy explota el paralelismo inherente haciendo uso de librerías optimizadas en oneAPI como oneMKL
- Ej: 100x de aceleración en Numpy broadcasting respecto al código descrito como bucle
- Además el uso de numpy permite hacer uso de la memoria de forma eficiente

Uso de paralelismo

Para ver que se hace uso de la librería MKL de oneAPI podemos usar el comando de Numpy `np.show_config()`

```
import numpy as np
np.show_config()
....
SIMD Extensions:
  baseline:
    - SSE
    - SSE2
    - SSE3
    - SSSE3
    - SSE41
    - POPCNT
    - SSE42
  found:
    - AVX
    - F16C
    - FMA3
```

```
- AVX2
not found:
- AVX512F
- AVX512CD
- AVX512_SKX
- AVX512_CLX
- AVX512_CNL
- AVX512_ICL
```

En el cuaderno de Jupyter `numpy.ipynb` vamos a comprender:

- El uso de “ufuncs” es mucho más eficiente hacerlo directamente en **Numpy** que mediante bucles
- La indexación o desplazamientos de nuevo es más eficiente hacerla en **Numpy** que mediante bucles
- Las operaciones de agregación son más eficientes en **Numpy**
- Las operaciones de broadcast son más eficientes en **Numpy**
- Las operaciones condicionales son más eficiente en **Numpy**

Paquetes extra

Seguramente sea necesario instalar los paquetes de “pandas” y “matplotlib” para ejecutar los ejercicios del cuaderno:

```
user@host: ~/intelpython3$ source env/vars.sh
user@host: ~/intelpython3$ pip install pandas
user@host: ~/intelpython3$ pip install matplotlib
```

Ejercicio 2: Uso de numpy

En el cuaderno de Jupyter del ejercicio 2 denominado `numpy_ejercicio.ipynb` completar:

- Producto escalar de dos vectores eficientemente en Numpy
- Tabla de multiplicar filtrada aquellos números que sean múltiplos de 12 y múltiplos de 9.
- Black-Scholes es un modelo matemático que permite calcular el precio teórico de opciones financieras, en el cuaderno hay una implementación en bucle, se recomendando utilizar el formato “vectorial” de Numpy para acelerar los cálculos

Ejercicio 3: Uso de DPNP y el Data Parallel Extension en Python

El Data Parallel para Python es una extensión de Data Parallel para Python que amplían las capacidades numéricas de Python.

Permite utilizar la CPU o la GPU como un dispositivo de alto rendimiento donde acelerar los cálculos. Para su uso hay varios paquetes básicos:

- **dpctl**: librería de control para seleccionar de dispositivos, la asignación de datos, datos tensoriales
- **dpnp**: extensiones paralelas para Numpy

El siguiente código muestra los dispositivos disponible en Python donde poder procesar los cálculos empleando la librería de control `dpctl`:

```
import dpctl
d = dpctl.select_gpu_device()
d.print_device_info()
```

Data Parallel Extensions

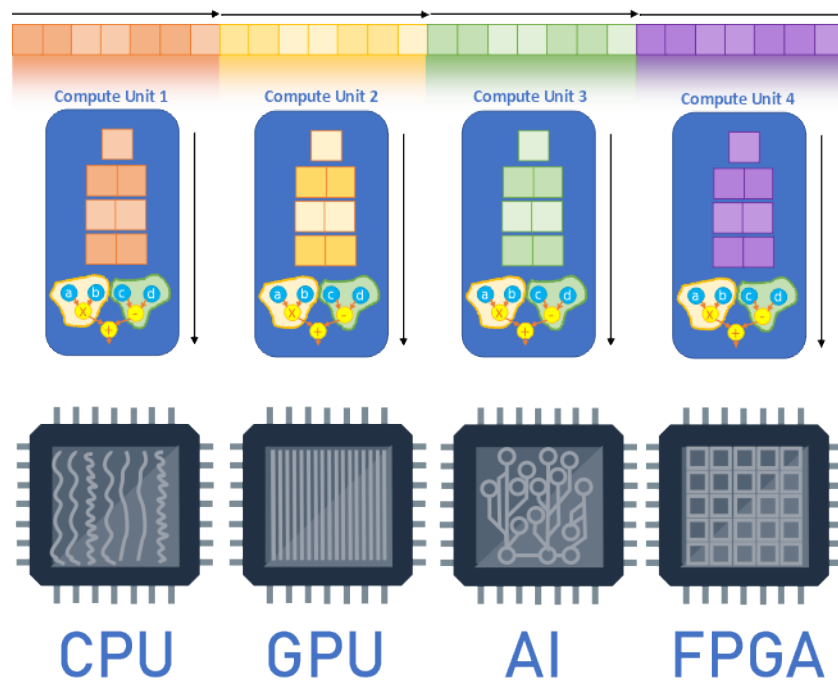


Figure 3: Fig: Data Parallel Extension para Python

```

user@host: ~/intelpython3$ python3 GPUdevice.py
Name           Intel(R) Graphics
Driver version  1.6.31294.120000
Vendor         Intel(R) Corporation
Filter string   level_zero:gpu:0

```

Además la librería `dpctl` permite almacenar datos en el dispositivo de forma sencilla con la clase `dpctl.tensor`:

```

from dpctl import tensor
import numpy
x_cpu = tensor.ones((2, 2), device="cpu")
x_gpu = tensor.ones((2, 2), device="gpu")
x_to_gpu = x_cpu.to_device("gpu")
x_np = numpy.ones((2,2))
x2_to_gpu = tensor.asarray(x_np, device="gpu")
w = tensor.asarray([x_cpu, x_gpu, x_np, x_to_gpu, x2_to_gpu], device="cpu")
y = w + w + 1
print("Tensor y device:", y.device)
print("Tensor y shape:", y.shape)
y_np = tensor.asnumpy(y)
print("max: ", numpy.max(y_np))

```

Desde el punto de vista del cómputo sigue el concepto Compute-follow-data que en lugar de mover grandes cantidades de datos, **el procesamiento se realiza allí donde los datos ya residen**.

Extensión de DataParallel para Numpy - dpnp

La librería `dpnp` (NumPy Drop-In Replacement for Intel® XPU) permite desarrollar aplicaciones en entorno *NumPy* en dispositivos paralelos. Básicamente permite seguir los conceptos expresados en el código desarrollado en *NumPy*.

Es decir, en un script Python escrito en *numpy*, donde una vez inicializado un array se multiplica cada componente por 4 que se procesa en la CPU:

```

# Original numpy code
import numpy
X = np.array([1,2,3])
Y = X * 4

```

se puede transformar fácilmente con la librería `dpnp` permitiendo realizar dichas operaciones en las XPU's (CPU o GPU) de forma sencilla:

```

# Transformed numpy into dpnp
import dpnp as np
X = np.array([1,2,3])
Y = X * 4
print("Array X allocated on the device:", X.device)
red = np.sum(Y)
print(red)

```

Contenido del laboratorio

En la carpeta “Ejercicio3” están dos cuadernos de Jupyter que permiten a alumno comprender estos conceptos:

- `get_started.ipynb` muestra el uso de la librería de control `dpctl`
- `dpep_scalability_journey_example-pairwise_distance.ipynb` muestra el procesado del calculo de la distancia entre puntos de tres componentes (ej. `nbody`) para 10x1024 elementos haciendo uso de Numpy, `dpnp` (en CPU y GPU)

Ejercicio 4

El cuaderno de Jupyter `ejercicio_dpnp.ipynb` muestra como poder utilizar la biblioteca DPNP para acelerar el cómputo en el XPU.

Ejemplo inicial

Inicialmente se muestra una multiplicación de matrices empleando el método `numpy.dot`, mostrando el código en Numpy y el equivalente en DPNP. Nótese que en la función `matrix_multiplication_dpnp` se selecciona inicialmente el dispositivo XPU que puede actuar como CPU o GPU mediante el método de la biblioteca de control: `select_cpu_device()/select_gpu_device()`. Posteriormente se envía desde el host los datos de a y b al acelerador (a_{xpu} , b_{xpu}) con la llamada de `np_dp.asarray(b, device=dev)` por último se calcula el producto matricial invocando a `np_dp.dot`:

```
def matrix_multiplication_dpnp(dev):  
  
    if (dev=='cpu'):  
        d = dpctl.select_cpu_device()  
    elif (dev=='gpu'):  
        d = dpctl.select_gpu_device()  
    d.print_device_info()  
  
    size = 8192  
    a = np.random.rand(size, size).astype(np.float32)  
    a_xpu = np_dp.asarray(a, device=dev)  
    b = np.random.rand(size, size).astype(np.float32)  
    b_xpu = np_dp.asarray(b, device=dev)  
  
    start_time = time.time()  
    c = np_dp.dot(a_xpu, b_xpu)  
  
    _ = c.asnumpy() #Ensure all computations are completed  
    end_time = time.time()  
  
    print(f"Matrix multiplication (DPNP on {dev}) completed.")  
    print(f"Time taken (DPNP on {dev}):", end_time - start_time, "seconds")  
    print("DPNP harnesses GPU acceleration for enhanced performance! Give it a try!")  
    return end_time - start_time
```

Ejemplo 1 (cálculo de π con el método de Montecarlo)

El método de Monte-Carlo es una técnica probabilística utilizada para estimar el valor de π generando puntos aleatorios dentro de un cuadrado y determinando cuántos caen dentro de un círculo inscrito.

Consideremos un círculo de radio $r = 1$ inscrito dentro de un cuadrado de lado $2r = 2$. La ecuación del círculo centrado en el origen es: $x^2 + y^2 \leq r^2$. Dado que el área del cuadrado es 4 y el área del círculo es π , la proporción de puntos dentro del círculo respecto al total de puntos generados aleatoriamente será aproximadamente: $\frac{\text{puntos dentro en el círculo}}{\text{puntos totales}} \approx \frac{\pi}{4}$

El alumno deberá de implementar la versión equivalente con DPNP, teniendo en cuenta que la inicialización aleatoria de “x” e “y” puede controlarse donde se alojan con el parámetro `device` de `dpnp.random.random` (más info en la documentación)

Ejemplo 2 (Kmeans)

El funcionamiento básico del algoritmo kmeans ya trabajado en la asignatura sigue los siguientes pasos:

La función `kmeans` implementa el algoritmo de K-means con los siguiente pasos:

1. **Inicialización:** Se seleccionan aleatoriamente K centroides iniciales (puntos representativos de cada grupo), esto se realiza con la variable `random_indices` que indexa a `centroids` en el código
2. **Asignación:** Cada punto del conjunto de datos se asigna al centroide más cercano (según distancia euclídea u otra métrica). En el código se calcula la distancia de cada punto a los centroides y se almacena en la variable `distances` y se le asigna el clúster correspondiente con la distancia menor con la función de numpy `np.argmin`
3. **Actualización:** Se recalculan los centroides como el promedio de los puntos asignados a cada grupo en la variable `new_centroids`
4. **Repetición:** Los pasos de asignación y actualización se repiten hasta que los centroides no cambian significativamente o se alcanza un número máximo de iteraciones. El “no cambio significativo” se implementa con el cómputo de la convergencia en la instrucción `shift = np.linalg.norm(centroids - new_centroids)`, mientras que la condición de parada por alcanzar el número máximo de iteraciones se implementa en el bucle `for iteration in range(max_iters):`

```
import numpy as np

def kmeans(X, k, max_iters=100, tol=1e-7, verbose=False):
    # X es un array (n_samples, n_features)
    n_samples, n_features = X.shape

    # Elegir k puntos iniciales aleatorios como centroides
    rng = np.random.default_rng(42)
    random_indices = rng.choice(n_samples, size=k, replace=False)
    centroids = X[random_indices]

    for iteration in range(max_iters):
        # Calcular distancias (n_samples x k)
        distances = np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)
        # Asignar cada punto al centroide más cercano
        labels = np.argmin(distances, axis=1)

        # Calcular nuevos centroides
        new_centroids = np.array([X[labels == i].mean(axis=0) for i in range(k)])

        # Verificar convergencia
        shift = np.linalg.norm(centroids - new_centroids)
        if verbose:
            print(f"Iter {iteration}, shift = {shift:.6f}")
        if shift < tol:
            break

        centroids = new_centroids

    return centroids, labels
```

ToDo: implementación con DPNP El alumno desarrollará la implementación con DPNP. Para ello se proporciona un esqueleto donde ya se refleja el movimiento del dataset original al procesador XPU mediante la operación `X_dnp = np.asarray(X_np, device="cpu")`. Posteriormente para mostrar el resultado de los nuevos centroides, al estar almacenados en la XPU, hay que moverlos a la memoria de “host” para poder ser visualizados pudiendo hacerse con el comando `dnp.asarray()`. Además de estos aspectos se recomienda al estudiantado las siguientes recomendaciones:

- Elegir k puntos iniciales aleatorios como centroides, pero no existe una correspondencia en la librería DPNP de la función `rng.choice` por lo que el cálculo de `rng` y `random_indices` se deben realizar como en *Numpy* original, y luego enviar la variable `random_indices` al dispositivo XPU con el comando `dnp.asarray()`. La indexación posterior al hacerse en el dispositivo con datos que están en el dispositivo es correcta

- La verificación de la convergencia, hay que tener en cuenta que la variable `shift` que calcula la norma de los centroides “antiguos” respecto a los “nuevos”, se efectúa en la XPU por lo que la variable “shift” estará almacenada en el dispositivo, pero la condicional `if shift < tol` se procesa en el host no pudiendo comparar ambas variables al estar en ámbitos distintos (XPU para `shift` vs `tol` en el host). En este caso se recomienda traer el contenido de `shift` al host para procesarlo de forma coherente. Esto es posible con la función `dpnp.asnumpy` que convierte una variable alojada en el XPU a `numpy`.

```
import numpy as npy
import dpnp as np

def kmeans_dpnp(X, k, max_iters=100, tol=1e-7, verbose=False):
    # X es un array (n_samples, n_features)
    n_samples, n_features = X.shape

    # Elegir k puntos iniciales aleatorios como centroides
    #.....

    return centroids, labels

X_dpnp = np.asarray(X_np, device="gpu") # Convert from numpy to dpnp array

start = time.time()
centroids, labels = kmeans_dpnp(X_dpnp, k=5, verbose=False)
print("Time to Kmeans with DPNP:", time.time() - start)
print("Centroids:\n", centroids.asnumpy())
```

Resultados esperado

Al finalizar esta práctica, el alumno será capaz de:

- Comprender explotar paralelismo SIMD y multi-hilo en Python de forma transparente mediante la biblioteca de Numpy
- Comprender el funcionamiento básico de la extensión de Intel® DPNP y cómo se integra con la explotación de aceleradores

Práctica 4.1: Manejo de sockets y explotación de concurrencia en Python

Índice

1	Objetivos	1
2	Preparación del entorno de trabajo	1
3	Ejercicios	2
	Ejercicio 1. Concurrencia en Python usando <code>threading</code> y <code>multiprocessing</code>	2
	Limitaciones del intérprete de Python y casos de uso	2
	Modo de uso	3
	Efectos del GIL en el tiempo de ejecución	4
	Ejercicio 2. Sincronización entre hilos en Python	5
	Ejercicio 3. Uso práctico de sockets, procesos e hilos para comunicación y procesamiento en entornos cliente-servidor	6
	Ejercicio 4. Despliegue de cliente-servidor en nodos diferentes	7

1 Objetivos

El principal objetivo de esta práctica es familiarizarse con el manejo básico de hilos, procesos y sockets en Python. Como parte de la práctica se proporciona un breve tutorial que ilustra las diferencias entre los hilos y los procesos en Python, en cuanto a programación, limitaciones y casos de uso típicos.

En el fichero comprimido con el código y otros recursos de la práctica se proporciona una carpeta con el código fuente asociado a cada ejercicio, así como un proyecto de VSCode y los ficheros necesarios para construir una imagen de Docker que se utilizará para realizar la práctica con el proyecto VSCode proporcionado.

2 Preparación del entorno de trabajo

Para poder trabajar en la práctica es preciso tener un equipo con Docker instalado. La máquina virtual VMware de Debian disponible en el campus virtual puede usarse para ello, ya que incluye Docker preinstalado. No obstante, los estudiantes que lo deseen pueden simplemente utilizar su portátil con Docker instalado (sin la citada máquina virtual), siendo Docker Desktop lo que se ha de instalar sobre Mac OS o Windows nativo.

Como se indicó anteriormente, entre los ficheros de la práctica se proporcionan dos componentes básicos que se usarán sobre Docker:

- *DockerContainer*: Este directorio contiene los ficheros necesarios para construir la imagen `ray-os` que se usará para las últimas prácticas. Esta imagen contendrá un entorno funcional de Python con todas las dependencias necesarias instaladas.
- *PythonProject*: Es un proyecto VSCode basado en DevContainers. Para usarlo es preciso construir antes la imagen de Docker `ray-os`.

El primer paso para preparar el entorno de trabajo consiste en crear la imagen de Docker. Para ello abriremos una terminal, cambiaremos al directorio *DockerContainer* y, desde ese directorio, ejecutaremos el siguiente comando:

```
$ docker image build -t ray-os .
```

La creación de esta imagen puede llevar bastante tiempo (al menos 20 minutos) ya que requiere descargar la imagen de una versión reciente de Debian, sobre la que se instalan una serie de utilidades esenciales, así como un *virtual environment* de Python con las dependencias necesarias. Como aspecto positivo, una vez creada la imagen, será posible crear múltiples contenedores con ella, como el que requiere el proyecto de VSCode.

Tras crear la imagen podremos trabajar en la práctica con el proyecto *PythonProject* donde debemos mover o copiar las carpetas de los distintos ejercicios de la práctica. Finalmente, abriremos el proyecto con VSCode. Al hacer esto pedirá arrancar un contenedor tras la apertura del mismo. Como en el proyecto DevContainers facilitado para trabajar en las prácticas del Módulo 1, la raíz del proyecto *PythonProject* funciona como una carpeta compartida con nuestro *host*. Todo fichero que alteremos/añadamos allí estará disponible desde el sistema operativo anfitrión.

Muy importante:

Una vez arrancado el contenedor con VSCode, debemos hacer dos cosas más para que el proyecto esté totalmente operativo:

1. Abrir una ventana de terminal y crear un enlace simbólico para que VSCode trabaje por defecto con el *virtual environment* de Python ya instalado en el contenedor. Para ello ejecutaremos el siguiente comando:

```
$ ln -s /root/venv .venv
```

2. Por último esperaremos a que VSCode acabe de instalar una serie de extensiones de VSCode, cuya instalación se inicia automáticamente al abrir el proyecto por primera vez. Sabremos que la instalación de estas extensiones ha terminado cuando desaparezca el icono del reloj de la barra lateral izquierda del entorno donde se accede a las extensiones de VSCode.

3 Ejercicios

Ejercicio 1. Concurrencia en Python usando **threading** y **multiprocessing**

Para explotar concurrencia en los programas, Python ofrece dos módulos básicos: **threading** y **multiprocessing**. Aunque ambos permiten la ejecución simultánea de tareas, se basan en modelos diferentes. El módulo **threading** permite ejecutar varios hilos (*threads*) dentro de un mismo proceso –intérprete de Python–, compartiendo memoria. Por contra, **multiprocessing** permite la creación de varios procesos (intérpretes) independientes, cada uno con su propia imagen de memoria.

Mediante estos módulos podemos explotar concurrencia para acceder a cierta funcionalidad de gran utilidad como:

- Aprovechar múltiples núcleos de la CPU
- Realizar tareas que se bloquean esperando por entrada-salida sin detener todo el programa
- Modelar sistemas con componentes independientes que se comunican entre sí (como productor-consumidor, cliente-servidor, etc.).

Limitaciones del intérprete de Python y casos de uso

A la hora de explotar concurrencia, además de tener en cuenta las diferencias entre procesos e hilos introducidas en el primer módulo de la asignatura, debemos considerar un aspecto clave de la implementación más extendida de Python, llamada CPython. Esta implementación hace uso de un cerrojo global del intérprete denominado *Global Interpreter Lock* (GIL), que **restringe la ejecución simultánea de múltiples hilos en un único proceso de Python**. Es decir, aunque creemos varios hilos en nuestro programa, **solo uno puede ejecutar bytecode de Python a la vez**. Esto significa que

incluso en máquinas con múltiples núcleos, los hilos de Python **no pueden ejecutar código Python en paralelo** si están haciendo trabajo intensivo de CPU.

El GIL existe por varias razones históricas y técnicas:

- CPython **no tiene un recolector de basura con contadores de referencias seguros en entornos multihilo**. El GIL simplifica la gestión de memoria y evita que haya que usar bloqueos (*locks*) finos alrededor de cada objeto.
- El GIL hace que ciertas operaciones en programas de un solo hilo sean más rápidas y fáciles de mantener.
- Es más sencillo escribir extensiones en C para Python cuando se puede asumir que no hay acceso concurrente.

Debida a la existencia del GIL, los módulos `threading` y `multiprocessing` tienen distintos casos de uso, como recoge la siguiente tabla:

Caso de uso	Módulo Recomendado
El procesamiento de tareas incluye muchas esperas por entrada-salida (p.ej., red, disco, etc.) o sincronización	<code>threading</code>
Las tareas realizan cálculos intensivos en CPU	<code>multiprocessing</code>
Las tareas no son muy intensivas en CPU (tienen esperas) y requieren compartir memoria a la que se accede muy frecuentemente	<code>threading</code>
Se precisa tener aislamiento entre tareas (intérprete de Python propio para cada una)	<code>multiprocessing</code>

Modo de uso

Tanto `threading` como `multiprocessing` ofrecen al programador una API muy similar para la creación, terminación y espera de hilos y procesos, respectivamente. Sendos módulos exportan una clase específica para ello: `Thread` y `Process`. Para crear un hilo o proceso basta crear un objeto de esa clase e invocar el método `start()`. Análogamente el método `join()` permite que otro proceso/hilo se quede esperando a la terminación del proceso/hilo lanzado con `start()`.

Para que el programador indique qué código (función) ejecutará el hilo/proceso a crear existen dos alternativas:

1. Crear una clase extendiendo la clase base correspondiente (`Thread` o `Process`) y definir su método `run()`, donde indicaremos el procesamiento realizado por el hilo.
2. Crear un objeto `Thread` o `Process`, pasando como parámetro en el constructor la función que deseamos que el hilo ejecute, así como sus argumentos.

A continuación se muestra un ejemplo de la primera aproximación, incluyendo la variante con procesos e hilos. En ambos casos, se define una clase `MiTarea` para especificar el procesamiento realizado por el nuevo proceso/hilo.

Procesos

```
from multiprocessing import Process

class MiTarea(Process):
    def __init__(self):
        super().__init__()

    def run(self):
        print("Ejecutando tarea")

if __name__ == '__main__':
    process = MiTarea()
    process.start()
    process.join()
```

Hilos

```

from threading import Thread

class MiTarea(Thread):
    def __init__(self):
        super().__init__()

    def run(self):
        print("Ejecutando tarea")

if __name__ == '__main__':
    thread = MiTarea()
    thread.start()
    thread.join()

```

Para ilustrar el uso de la segunda alternativa, donde se pasa la función del hilo/proceso como argumento, se proporciona un ejemplo adicional a continuación. En este caso el programa crea un hilo y un proceso que ejecutan la misma función:

```

#!/usr/bin/env python
import threading
import multiprocessing
import time

def worker(name):
    print(f"Inicio de {name}")
    time.sleep(2)
    print(f"Fin de {name}")

if __name__ == '__main__':
    # Hilo
    t = threading.Thread(target=worker, args=("Hilo",))
    # Proceso
    p = multiprocessing.Process(target=worker, args=("Proceso",))
    t.start()
    p.start()
    t.join()
    p.join()

```

Efectos del GIL en el tiempo de ejecución

Como se indicó anteriormente el GIL impide la ejecución simultánea de código Python en múltiples hilos del mismo proceso. Para observar este aspecto, se propone la realización del siguiente ejercicio:

1. Ejecuta el siguiente programa (*timing.py*) que temporiza la ejecución de una misma tarea invocada por múltiples hilos y procesos simultáneos:

```

#!/usr/bin/env python
import time
from threading import Thread
from multiprocessing import Process

N = 5

def tarea():
    time.sleep(1)

def program():
    # Con hilos
    inicio = time.time()
    hilos = [Thread(target=tarea) for _ in range(N)]

```

```
[t.start() for t in hilos]
[t.join() for t in hilos]
print("Tiempo con hilos:", time.time() - inicio)

# Con procesos
inicio = time.time()
procesos = [Process(target=tarea) for _ in range(N)]
[p.start() for p in procesos]
[p.join() for p in procesos]
print("Tiempo con procesos:", time.time() - inicio)
```

2. Toma nota de los tiempos mostrados por el programa. ¿Qué alternativa es más rápida?
3. Modifica la función tarea empleando la nueva definición proporcionada a continuación, donde no se hace una espera como antes, sino un cómputo:

```
def tarea():
    suma = 0
    for i in range(10**7):
        suma += i
```

4. Toma nota de los tiempos mostrados por el programa. ¿Qué alternativa es más rápida? ¿Coincide con la respuesta del punto 2? ¿A qué se debe este resultado?

Ejercicio 2. Sincronización entre hilos en Python

A pesar del uso del GIL en la implementación CPython, el acceso atómico a variables y objetos en programas multihilo no está asegurado automáticamente. Para permitir la ejecución segura de métodos y evitar la corrupción de estructuras de datos y variables compartidas durante el acceso concurrente es preciso utilizar mecanismos de sincronización de Python.

El módulo `threading` ofrece implementaciones de semáforos, cerrojos y variables condicionales, representadas mediante las clases `Semaphore`, `Lock` y `Condition`, respectivamente. Una característica distintiva de la API de estos recursos de sincronización es que todos ellos ofrecen las operaciones `acquire()` y `release()`, que tienen diferente propósito en cada uno de los recursos de sincronización, como se resume en la tabla.

Recurso	Operaciones realizadas por <code>acquire()</code> y <code>release()</code> , respectivamente
<code>threading.Semaphore</code>	<code>wait</code> y <code>signal</code> del semáforo
<code>threading.Lock</code>	<code>lock</code> y <code>unlock</code> del cerrojo
<code>threading.Condition</code>	<code>lock</code> y <code>unlock</code> sobre el mutex asociado a la variable condición.

Para familiarizarse con el uso de primitivas de sincronización en Python se pide que el estudiante realice lo siguiente:

- Con el código de la práctica se proporcionan 3 ejemplos de uso de cerrojos, semáforos y variables condicionales: *lock.py*, *sem.py* y *prodcons.py*. Ejecuta estos ejemplos y analiza la implementación.
 - **Nota:** El constructor por defecto de las variables condicionales (sin argumentos) crea un cerrojo privado para la variable condicional. No obstante, durante la construcción del objeto es posible pasar un cerrojo creado previamente, como en el ejemplo *prodcons.py*, donde esto es necesario al usar múltiples variables condicionales para la implementación.
- Implementa una versión en Python del ejercicio 2A de la práctica 1.2: Inserción alternada de enteros por parte de dos hilos en un vector usando semáforos. En esta implementación se usará una lista de Python, en la que se insertará al final usando el método `append()`.

Ejercicio 3. Uso práctico de sockets, procesos e hilos para comunicación y procesamiento en entornos cliente-servidor

En este ejercicio se pide desarrollar un servidor en Python que implemente un servicio que proporciona la temperatura media mensual en una ciudad. Esta información se ha de inferir de un fichero *temperatures.csv* ya proporcionado, y cuyo contenido se cargará en la memoria del servidor como un `DataFrame` de Pandas. La columna “temp” fichero contiene desglosada la temperatura media de cada día del año 2024 para 4 ciudades: París, Madrid, Berlín, Roma. Los nombres de las ciudades figuran en inglés en la columna “name” del fichero.

El servidor aceptará conexiones y peticiones de los clientes a través de un socket TCP en el puerto 10000. Para hacer una petición, el cliente deberá iniciar una conexión con el servidor, y a continuación enviarle una cadena de caracteres con el formato "<Ciudad>:<NúmMes>". De este modo, la cadena "Madrid:3" indica la petición de la temperatura media en Madrid durante el mes de marzo. Al recibir esta petición, el servidor filtrará las entradas correspondientes del `DataFrame` (fechas del 1-03-2024 al 31-03-2024) y calculará la media de esas entradas, que devolverá al cliente.

En este ejercicio se ha de realizar lo siguiente:

- **(Parte A)** Crear una versión básica del servidor (*basic-server.py*) que solo atienda a un cliente de forma simultánea. Es decir, tras aceptar una nueva conexión, el servidor interactuará con el cliente hasta que cierre la conexión. A continuación el servidor volverá a quedarse en espera de nuevas conexiones.

Para depurar este servidor se puede usar el programa `telnet` como cliente interactivo, a lanzar tras ejecutar el servidor en otra ventana de terminal de la misma máquina o contenedor Docker. Este programa acepta como parámetros el nombre de host o IP del servidor, y el número de puerto:

```
$ telnet localhost 10000
Trying ::1...
telnet: connect to address ::1: Connection refused
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
Madrid:12
6.745161290322581°C
Rome:8
28.203225806451616°C
Berlin:4
11.653333333333334°C
Paris:13
Wrong month
Sevilla:2
Unknown city
END
Connection closed by foreign host.
```

Como se puede observar al teclear una cadena correcta, el servidor ha de respondernos con la temperatura media de ese mes. Para las cadenas incorrectas el servidor devuelve un mensaje de error. Al hacer una petición con la cadena “END” se termina la ejecución.

Posible salida en el servidor:

```
$ python3 basic-server.py
Server listening on 0.0.0.0:10000
Accepted connection from ('127.0.0.1', 53945)
Handling connection from ('127.0.0.1', 53945)
Received: Madrid:12
Received: Rome:8
Received: Berlin:4
Received: Paris:13
Received: Sevilla:2
```

- **(Parte B)** Empleando como base el servidor desarrollado en el punto anterior, crear otra versión del servidor (*multithreaded-server.py*) donde un hilo separado se encargará de interactuar con cada cliente. El hilo se creará tras establecer una nueva conexión (cuando la llamada `accept()` retorne) y terminará tan pronto como el cliente cierre la conexión.

Para verificar que el servidor está funcionando correctamente, se pueden invocar varias instancias simultáneas del programa `telnet` con los mismos argumentos pero desde distintos terminales. Esto permitirá emitir peticiones desde distintos clientes, con puertos diferentes asignados automáticamente por el sistema operativo.

- **(Parte C)** Desarrollar otra variante del servidor (*multiprocess-server.py*) donde se creen procesos auxiliares por cliente –en lugar de hilos– para el procesamiento de las peticiones.
- **(Parte D)** Desarrollar un programa Python –llamado *request-generator.py*– que permita hacer peticiones continuas automáticamente al servidor utilizando múltiples hilos concurrentes durante un tiempo T especificado por el usuario. En particular, el programa creará N hilos que actuarán como clientes, realizando cada uno de ellos peticiones secuenciales y aleatorias, introduciendo un retardo de M milisegundos entre ellas. Al finalizar la ejecución, se reportará la tasa de peticiones por segundo observadas durante la misma.

Al lanzar el programa desde línea de comandos se pasarán los siguientes argumentos

```
request-generator.py <host_servidor:puerto> <T> <N> <M>
```

Para el cálculo de la tasa de peticiones por segundo al final de la ejecución, se compartirá una variable entera entre los hilos para llevar la cuenta del número de peticiones realizadas. Este contador se actualizará de forma segura cada vez que algún cliente reciba la respuesta del servidor (valor de temperatura media).

Utilizar el generador de peticiones para determinar qué implementación del servidor (Parte B o C) resulta más eficiente.

Nota: Para esta parte del ejercicio se proporciona un esqueleto del programa en *request-generator.py*

Ejemplo de ejecución (usando otra terminal distinta a la empleada para lanzar el servidor)

```
$ python3 ./request-generator.py localhost:10000 20 10 10

Peticiones totales: 15770
Duración: 20.01 segundos
Tasa de peticiones: 788.15 peticiones/segundo
```

Ejercicio 4. Despliegue de cliente-servidor en nodos diferentes

En este ejercicio guiado usaremos dos nodos diferentes para ejecutar el cliente (*request-generator.py*), y el servidor (*multithreaded-server.py*). Para simular los dos nodos, crearemos dos contenedores Docker que estarán conectados usando una red privada. Los contenedores usarán la imagen de Docker `ray-os` ya creada, y compartirán con el *host* la carpeta *code*, que se encuentra en el directorio *ejercicio4*.

El primer paso consiste en copiar a la carpeta *code* los ficheros con el código Python de cliente y servidor, así como el fichero *temperatures.csv*.

A continuación, abriremos una shell en la carpeta del *ejercicio4*, crearemos una red privada de Docker llamada “test”:

```
$ cd <ruta_ejercicio_4>
$ docker network create test
206f74cd457685cb70c719b7330091f52579fce84bf2fbb6d5fe1bb18088f1d9
```

Crearemos el contenedor del servidor, llamado “python-server” con nombre de host “server”. Para ello, ejecutaremos el siguiente comando de docker que abrirá un shell dentro del contenedor:

```
$ docker container run -it --rm --name python-server \
    -h server --net test -v "${PWD}/code:/root/code" \
    ray-os /bin/bash
root@server:/#
```

Posteriormente volveremos a esa terminal para ejecutar el servidor.

Para lanzar el cliente abriremos otra terminal, accediendo al directorio del ejercicio 4, y ejecutaremos el siguiente comando de docker.

```
$ docker container run -it --rm --name python-client \
    -h client --net test -v "${PWD}/code:/root/code" \
    ray-os /bin/bash
root@client:/#
```

Con el comando anterior se crea un contenedor cliente llamado “python-client” y con nombre de host “client”.

Ahora que ambos contenedores están lanzados procedemos a arrancar el servidor:

```
root@server:/# cd /root/code
root@server:~/code# python3 ./multithreaded-server.py
Server listening on 0.0.0.0:10000
```

Y por último lanzaremos el cliente durante 20 segundos usando 10 hilos y 10ms de retardo entre cada petición. Para ello usaremos el siguiente comando, que asume que el servidor escucha en el puerto 10000 de la máquina “server”:

```
root@client:~/code# python3 ./request-generator.py server:10000 20 10 10

Peticiones totales: 15770
Duración: 20.01 segundos
Tasa de peticiones: 788.15 peticiones/segundo
```

Práctica 4.2: Programación distribuida en Python con Ray

Índice

1. Objetivos	1
2. Ejercicios	1
Ejercicio 1: Paralelización de la clasificación de un conjunto de imágenes con Ray	1
Modelo y conjunto de datos	1
Desarrollo del ejercicio	3
Ejercicio 2: Creación de servicio de información sobre imágenes basado en Ray Serve	4
Envío de peticiones POST desde el cliente	4
Procesamiento de peticiones	5
Desarrollo del ejercicio	7

1. Objetivos

El objetivo de esta práctica es iniciarse en la programación distribuida de alto nivel con Python. Se usará para ello el framework Ray, donde se explorará el potencial de las tareas, actores, así como del subsistema Ray Serve.

Para trabajar con esta práctica se utilizará el proyecto de VSCode proporcionado con la práctica anterior, basado en la imagen `ray-os` ya creada.

2. Ejercicios

Ejercicio 1: Paralelización de la clasificación de un conjunto de imágenes con Ray

En este ejercicio se realizarán distintas implementaciones de un proceso de clasificación de imágenes usando un modelo ya entrenado basado en una CNN. El desarrollo se realizará empleando un cuaderno de Jupyter, que se entregará a través del campus virtual. Tanto el conjunto de imágenes para inferencia, como el modelo se descargarán de [Hugging Face](#), plataforma de *machine learning* (ML) y ciencia de datos que facilita a la comunidad el desarrollo, despliegue y entrenamiento de modelos de ML.

Modelo y conjunto de datos

Más concretamente, como modelo para la clasificación se utilizará el [Vision Transformer \(ViT\)](#) de Google, que se aplicará sobre el conjunto `train` del *dataset* [tiny-imagenet](#). Para la carga en memoria del modelo y del *dataset* se hará uso de los módulos `transformers` y `datasets`, respectivamente, proporcionados por Hugging Face. Estos módulos ya están instalados en el entorno de trabajo basado en Docker y VSCode.

Para obtener el conjunto de datos es preciso ejecutar el siguiente código:

```
from datasets import load_dataset
tiny_imagenet = load_dataset('Maysee/tiny-imagenet')
```

Nótese que al ejecutar `load_dataset` por primera vez en una máquina con un *dataset* concreto, dicho *dataset* se descarga en local. Por lo tanto la primera ejecución de `load_dataset` tardará mucho más tiempo que invocaciones posteriores, que solo proceden a cargar en memoria el conjunto de datos (imágenes en este caso).

El objeto devuelto por la llamada anterior es de tipo `DatasetDict`, que puede usarse como un diccionario de Python para acceder a los dos conjuntos de imágenes del *dataset*: `train` (para entrenamiento) y `valid` (para validación). Al imprimir la estructura de datos `tiny_imagenet` desde un *notebook*, observaremos la siguiente salida:

```
DatasetDict({
  train: Dataset({
    features: ['image', 'label'],
    num_rows: 100000
  })
  valid: Dataset({
    features: ['image', 'label'],
    num_rows: 10000
  })
})
```

El conjunto `train` contiene 100000 muestras, frente a las 10000 del conjunto `valid`. Cada muestra incluye una imagen en formato PIL (módulo `pillow`), junto con la etiqueta (clase) de la imagen, que no se usará para el desarrollo del ejercicio.

Para acceder a una imagen aislada (objeto PIL), debemos usar su índice dentro del conjunto deseado, y acceder con la clave "image" en la estructura de diccionario. Así, por ejemplo si deseamos acceder a la imagen del conjunto de `train` con índice 2300, tendremos que utilizar la siguiente expresión de Python:

```
tiny_imagenet['train'][2300]['image']
```

Desde el cuaderno de Jupyter podremos visualizar la imagen al usar dicha expresión en una celda:



Para cargar el modelo para clasificación de imágenes, usaremos el módulo `transformers` de Hugging Face:

```
from transformers import pipeline

clf = pipeline("image-classification")
```

La llamada `pipeline` retorna un clasificador para poder inferir la clase de una imagen en formato PIL, entre otros. Así por ejemplo, para obtener la clase de la imagen considerada anteriormente ejecutaremos lo siguiente:

```
In [8]: clf(tiny_imagenet['train'][2300]['image'])

[{'label': 'American alligator, Alligator mississippiensis',
  'score': 0.9436624646186829},
 {'label': 'African crocodile, Nile crocodile, Crocodylus niloticus',
  'score': 0.025410419330000877},
 {'label': 'platypus, duckbill, duckbilled platypus, duck-billed platypus,
  Ornithorhynchus anatinus', 'score': 0.0012438251869753003},
 {'label': 'alligator lizard', 'score': 0.0007204426219686866},
 {'label': 'frilled lizard, Chlamydosaurus kingi',
  'score': 0.0007057994953356683}]
```

Como muestra la salida de la celda, el clasificador retorna una lista de clasificaciones ordenadas por puntuación, siendo en este caso la más acertada "American alligator", con más de un 94 % de probabilidad.

Desarrollo del ejercicio

En este ejercicio se pide desarrollar varias versiones del proceso de clasificación de un subconjunto de imágenes de `tiny-imagenet` usando el modelo previamente introducido. Concretamente, se realizará la inferencia sobre un total de 250 imágenes del conjunto `train` seleccionadas dentro del siguiente rango: `range(0, 10000, 40)`. El proceso global retornará una lista de tuplas `result` con la clasificación de mayor puntuación (`score`) obtenida para cada imagen, donde cada tupla indicará (clase, puntuación).

El desarrollo se realizará empleando un cuaderno de Jupyter, que se entregará a través del campus virtual. Para cada una de las versiones a desarrollar, su ejecución se incluirá en una celda con la cabecera `%time` para reportar el tiempo de ejecución, y así poder comparar el rendimiento de las distintas implementaciones. La ejecución de cada versión ha de incluir la carga del `dataset` y del modelo en memoria. Las versiones a desarrollar son las siguientes

- **(Versión A)** Versión secuencial de la implementación, que recorra las imágenes con un bucle. Se aconseja desarrollar una función auxiliar `get_best_class_and_score`, que acepte como parámetros el clasificador y la imagen (objeto PIL), y retorne una tupla con la mejor clasificación y su puntuación.

```
def get_best_class_and_score(classifier, pil_image):  
    .... TODO ....
```

- **(Versión B)** Versión paralela de la implementación empleando tareas remotas de Ray. En concreto, se definirá una única tarea para clasificar una imagen, aceptando como parámetro el modelo y el objeto de imagen:

```
@ray.remote  
def inference(classifier, image):  
    return get_best_class_and_score(classifier, image)
```

Al invocar esta tarea via `inference.remote()` se pasará como parámetro una referencia `rclef` al clasificador, que se almacenará en el *object store* de Ray:

```
clf = pipeline("image-classification")  
rclef=ray.put(clf)
```

- **(Versión C)** Versión paralela de la implementación empleando actores de Ray. Se crearán tantos actores como núcleos tenga el equipo en el que estamos trabajando. Cada actor exportará métodos para realizar la inferencia, y la implementación asignará de forma rotatoria “trabajo” de inferencia a cada actor, equilibrando así la carga entre *workers*. Por motivos de eficiencia, cada actor almacenará como atributo una instancia propia del clasificador, y opcionalmente (a determinar en la construcción del actor), el *dataset* de imágenes. Esto permitirá reducir transferencia de datos entre el *driver* y los *workers* del cluster de Ray. Por simplicidad se proporciona a continuación parte de la implementación del actor:

```
@ray.remote  
class InferenceEngine:  
    def __init__(self, load_images=True):  
        self.classifier = pipeline("image-classification")  
        if load_images:  
            self.dataset = load_dataset('Maysee/tiny-imagenet')['train']  
        else:  
            self.dataset = None  
  
    def classify_image(self, i):  
        if self.dataset is None or i >= len(self.dataset):  
            return ("Error", 1)  
  
        ## A completar  
        ...  
  
    def classify_image_object(self, image):  
        ## A completar
```

...

Como puede observarse el constructor del actor carga una instancia privada del modelo, y hace lo propio con el *dataset* si el parámetro `load_images` es `True`. El actor ofrece dos métodos remotos:

1. `classify_image()` : Retorna la tupla de (clasificación, puntuación) asociada a una imagen, especificada únicamente por su índice en el *dataset*. Esta llamada está pensada para instancias del actor con el dataset almacenado en `self.dataset`
2. `classify_image_object()` : Retorna la tupla de (clasificación, puntuación) asociada a una imagen PIL (objeto) pasada como parámetro. En este caso el actor no hará uso de `self.dataset`

De esta versión con actores se probarán dos variantes. En la primera, cada actor mantendrá el *dataset* de imágenes en memoria, y el proceso de inferencia desde el *driver* invocará el método remoto `classify_image()`. En la segunda variante, el *dataset* se cargará en el driver, y los actores se instanciarán pasando `load_images=False` en su constructor. Asimismo, se usará el método remoto `classify_image_object()` para llevar a cabo la inferencia.

Consideraciones para la implementación:

- La celda con cada variante ha de incluir la creación y la destrucción de actores (`ray.kill(actorRef)`).
- Si en algún caso se muestra error de falta de memoria (`OutOfMemory error`), se aconseja detener y relanzar el cluster de Ray, como sigue:

```
ray.shutdown()
ray.init()
```

Considerando los tiempos de ejecución obtenidos, indicar qué implementación es más eficiente y justificar el motivo.

Pista: Se ha de tener presente que la transferencia de parámetros entre nodos/workers del cluster de Ray tiene coste no despreciable, especialmente si se trata de objetos de gran tamaño.

Ejercicio 2: Creación de servicio de información sobre imágenes basado en Ray Serve

En este ejercicio guiado se empleará Ray Serve para servir por HTTP un modelo que detecte imágenes enviadas remotamente por el usuario, identifique qué objeto, animal o cosa representa la imagen, y reporte información sobre ello consultando la Wikipedia.

La realización de este ejercicio requiere la instalación de dos paquetes adicionales con `pip` dentro del contenedor Docker. Para ello abriremos una terminal desde VSCode y ejecutaremos el siguiente comando:

```
(venv) /workspaces/.../PythonProject# pip install python-multipart wikipedia
```

A continuación se muestra información detallada sobre el procedimiento de envío de la petición, el modo de procesamiento de la misma en el servidor, y las partes a desarrollar de la práctica.

Envío de peticiones POST desde el cliente

El usuario realizará una petición POST HTTP a la URL donde se exporte el servicio de inferencia, incrustando la imagen en la petición HTTP. Para ello se utilizará el módulo `requests` de Python. A continuación se muestra un código de ejemplo, que realiza una petición para obtener información sobre la imagen de un martillo (mostrada más abajo), que se asume almacenada en el fichero *example-image.png* del cliente:

```
import requests

image_path = "example-image.png"
url = "http://127.0.0.1:8000/whatis/"
```

```
with open(image_path, "rb") as f:
    files = {"file": (image_path, f, "image/png")}
    response = requests.post(url, files=files)

print("Server response:", response.json())
```



Figura 1: Imagen de muestra

El servicio plenamente funcional, en ejecución, y accesible en el endpoint <http://127.0.0.1:8000/whatis/> desde *localhost*, debería devolver la siguiente salida, codificada en JSON:

```
Server response: {'prediction': 'hammer', 'wikipedia info': 'A hammer is a tool, most often a hand tool, consisting of a weighted "head" fixed to a long handle that is swung to deliver an impact to a small area of an object. This can be, for example, to drive nails into wood, to shape metal (as with a forge), or to crush rock. Hammers are used for a wide range of driving, shaping, breaking and non-destructive striking applications. Traditional disciplines include carpentry, blacksmithing, warfare, and percussive musicianship (as with a gong). \nHammering i'...}
```

Cabe destacar que las imágenes en formato PIL pueden almacenarse como ficheros PNG. De este modo, si empleamos un *dataset* de imágenes con muestras en formato PIL, podemos todavía hacer uso del servicio con una conversión previa a archivo PNG. El siguiente ejemplo pone de manifiesto cómo hacer la conversión con una imagen del *dataset* *tiny-imagenet* empleado en el ejercicio 1:

```
from datasets import load_dataset
from PIL import Image

## Carga del dataset
tiny_imagenet = load_dataset('Maysee/')

## Elección de una de las imágenes (formato PIL)
sample_image=tiny_imagenet['train'][2300]['image']

## Conversión de la imagen a PNG y almacenamiento en un fichero
sample_image.save("sample.png", format="PNG")
```

Procesamiento de peticiones

Para procesar peticiones en el servidor se usarán las siguientes tecnologías:

- **Ray Serve**, como framework básico para exposición del servicio
- **Vision Transformer (ViT)** de Google, para la clasificación de la imagen, usando dicho modelo desde el módulo `transformers` de Hugging Face, como en el ejercicio 1.

- **Módulo wikipedia de Python**, para hacer búsquedas de términos concretos en Wikipedia, y descargar contenido de páginas específicas.

Para ilustrar el uso básico del módulo `wikipedia`, consideremos el siguiente ejemplo. Supongamos que se desea buscar las páginas más frecuentes sobre “Barcelona”. Para ello, se ha de usar la función `search()` de `wikipedia`, como en el siguiente ejemplo, donde se obtienen las 10 entradas más comunes.

```
In [2]: import wikipedia

In [3]: results = wikipedia.search("Barcelona")

In [4]: results
Out[4]:
['Barcelona',
'FC Barcelona',
'2024-25 FC Barcelona season',
'Barcelona Open (tennis)',
'Barcelona (disambiguation)',
'RCD Espanyol',
'FC Barcelona 6-1 Paris Saint-Germain FC',
'1992 Summer Olympics',
'Vicky Cristina Barcelona',
'2025 Barcelona Open Banc Sabadell - Singles']
```

Si consultamos la página correspondiente al primer resultado de búsqueda con `page()` obtenemos un objeto de tipo `WikipediaPage`, cuyo contenido en texto plano podemos obtener usando el atributo `content`:

```
In [5]: page=wikipedia.page('Barcelona')
In [6]: page
Out[6]: <WikipediaPage 'Barcelona'>
In [7]: page.content
Out[7]: 'Barcelona ... is a city on the northeastern coast of Spain. It is the capital and largest city of the autonomous community of Catalonia, as well as the second-most populous municipality of Spain. With a population of 1.6 million within city limits, its urban area extends to numerous neighbouring municipalities within the province of Barcelona and is home to around 5.3 million people, making it the fifth most populous urban area of the European Union after Paris, the Ruhr area, Madrid and Milan. It is one of the largest metropolises on the Mediterranean Sea, located on the coast between the mouths of the rivers Llobregat and Besòs, bounded to the west by the Serra de Collserola mountain range.\nAccording to tradition, Barcelona was founded by either the Phoenicians or the Carthaginians, who had trading posts along the Catalanian coast. In the Middle Ages, Barcelona became the capital of the County of Barcelona. After joining with the Kingdom of Aragon to form the composite monarchy of the Crown of Aragon, Barcelona, which continued to be the capital of the Principality of Catalonia, became the most important city in the Crown of Aragon and its main economic and administrative centre, only to be overtaken by Valencia, wrested from Moorish control by the Catalans, shortly before the dynastic union between the Crown of Castile and the Crown of Aragon in 1516. Barcelona became the centre of Catalan separatism, briefly becoming part of France during the 17th century Reapers\' War and again in 1812 until 1814 under Napoleon. Experiencing industrialization and several workers movements during the 19th and early 20th century, it became the capital of autonomous Catalonia in 1931 and it was the epicenter of the revolution experienced by Catalonia during the Spanish Revolution of 1936, until its capture by the fascists in 1939...
```

El servidor a implementar con Ray Serve deberá utilizar las citadas funciones, para obtener la información sobre la predicción con mayor puntuación que haya devuelto el clasificador de imágenes. Nótese que si la categoría (*label*) de mayor puntuación que retorna el modelo incluye la coma (por devolver varios sinónimos), será preciso quedarse con el primer término. Así por ejemplo, si la imagen se reconoce como 'American alligator, Alligator mississippiensis', debemos usar 'American alligator' al realizar la búsqueda en Wikipedia.

Desarrollo del ejercicio

Desarrollar el servidor anteriormente descrito para servir el modelo de inferencia y consulta de información de imágenes con Ray Serve. A continuación se proporciona un código Python base (despliegue de Ray Serve) a completar para construir el servidor. El código incluye una serie de comentarios explicativos de las funciones/métodos a desarrollar y completar:

```
from PIL import Image
from io import BytesIO
from fastapi import FastAPI, File, UploadFile
from ray import serve
from transformers import pipeline
import ray
import wikipedia

def fetch_wikipedia_page(search_term, chars=500):
    ## Completar
    ## Si página no encontrada o no hay entradas asociadas
    ## a ese termino de búsqueda devolver "No pages found".
    ## En otro caso, retornar un string con los chars primeros
    ## caracteres de la página que aparece en primer lugar
    ## al hacer la búsqueda en Wikipedia

app = FastAPI()

@serve.deployment()
class WikipediaSearch:
    async def __call__(self, input_text):
        contents = fetch_wikipedia_page(input_text)
        return contents

@serve.deployment()
@serve.ingress(app)
class ImageClassifier:
    def __init__(self, wikipedia_search):
        ## Inicializar atributos
        self._classifier = ...
        self._wiki = ...

    def classify_image(self, image):
        ## Completar
        ## Ejecutar modelo self._classifier() sobre imagen.
        ## Si no hay resultados o puntuación (score) es < 0.80
        ## devolver None
        ## En caso contrario quedarse con el mejor resultado
        ## y con primer término (en caso de que hayq varios sinónimos
        ## separados por comas)

@app.post("/whatis")
async def classify(self, file: UploadFile = File(...)):
    ## Extrae fichero de imagen de petición HTTP
    contents = await file.read()
    ## Convierte contenido a imagen PIL
    image = Image.open(BytesIO(contents)).convert("RGB")
    ## Invocar modelo para obtener término de búsqueda
    search_item=self.classify_image(image)

    if search_item is None:
        return { "prediction": "unknown" }
```

```
## Completar conexión con modelo de WikipediaSearch
page_contents = ...

return {"prediction": search_item, "wikipedia info": page_contents }
```

```
wiki=WikipediaSearch.bind()
info=ImageClassifier.bind(wiki)
```

Para lanzar el servidor, se ha de ejecutar el comando `serve run` en una terminal dentro del contenedor gestionado por VSCode, pasando como parámetros el nombre del fichero Python (sin la extensión) y la variable que contiene el modelo principal que actúa de *frontend*, ambos valores separados por “:”. Así, por ejemplo, si el código se incluyera en el fichero *ray-server.py* almacenado en el directorio actual, el comando a ejecutar sería: `serve run ray-server:info`. Más abajo se proporciona la salida:

```
(venv) /workspaces/.../PythonProject# serve run ray-server:info
2025-04-25 11:40:35,155 INFO scripts.py:494 -- Running import path: 'ray-server:info'.
2025-04-25 11:40:38,256 WARNING services.py:2070 -- WARNING: The object store is using /tmp
instead of /dev/shm because /dev/shm has only 67108864 bytes available. This will harm
performance! You may be able to free up space by deleting files in /dev/shm. If you are inside
a Docker container, you can increase /dev/shm size by passing '--shm-size=1.20gb' to 'docker run'
(or add it to the run_options list in a Ray cluster config). Make sure to set this to more than
30% of available RAM. 2025-04-25 11:40:38,378 INFO worker.py:1843 -- Started a local Ray instance.
View the dashboard at http://127.0.0.1:8265 (ProxyActor pid=11132) INFO 2025-04-25 11:40:39,364
proxy 172.17.0.2 -- Proxy starting on node 21e4dd4e6bfa894e78d75a767bf7656b9cdc33b4df53d7ad9cf78801
(HTTP port: 8000).
INFO 2025-04-25 11:40:39,398 serve 10882 -- Started Serve in namespace "serve".
INFO 2025-04-25 11:40:39,406 serve 10882 -- Connecting to existing Serve app in namespace "serve".
New http options will not be applied.
(ProxyActor pid=11132) INFO 2025-04-25 11:40:39,384 proxy 172.17.0.2 -- Got updated endpoints: {}.
(ServeController pid=11135) INFO 2025-04-25 11:40:39,486 controller 11135 -- Deploying new version
of Deployment(name='WikipediaSearch', app='default') (initial target replicas: 1).
(ServeController pid=11135) INFO 2025-04-25 11:40:39,487 controller 11135 -- Deploying new version
of Deployment(name='ImageClassifier', app='default') (initial target replicas: 1).
(ProxyActor pid=11132) INFO 2025-04-25 11:40:39,489 proxy 172.17.0.2 -- Got updated endpoints:
{Deployment(name='ImageClassifier', app='default'): EndpointInfo(route='/',
app_is_cross_language=False)}.
(ProxyActor pid=11132) INFO 2025-04-25 11:40:39,517 proxy 172.17.0.2 --
Started <ray.serve._private.router.SharedRouterLongPollClient object at 0xffff840e6bd0>.
(ServeController pid=11135) INFO 2025-04-25 11:40:39,592 controller 11135 --
Adding 1 replica to Deployment(name='WikipediaSearch', app='default').
(ServeController pid=11135) INFO 2025-04-25 11:40:39,593 controller 11135 --
Adding 1 replica to Deployment(name='ImageClassifier', app='default').
(ServeReplica:default:ImageClassifier pid=11141) No model was supplied, defaulted to
google/vit-base-patch16-224 and revision 3f49326 (https://huggingface.co/google/vit-base-patch16-224).
(ServeReplica:default:ImageClassifier pid=11141) Using a pipeline without specifying a model name
and revision in production is not recommended.
(ServeReplica:default:ImageClassifier pid=11141) Device set to use cpu
INFO 2025-04-25 11:40:43,526 serve 10882 -- Application 'default' is ready at http://127.0.0.1:8000/.
[... Teclear CTRL+C para matar el servidor ...]
```

A partir de este momento, el servidor estará listo para recibir peticiones a través del puerto 8000 (puerto por defecto en Ray Serve). En particular, la URI `http://127.0.0.1:8000/what-is` es la que acepta peticiones POST para obtención de información de imágenes.

Una vez desarrollado y lanzado el servidor, el estudiante ha de generar peticiones de prueba usando un *notebook*, que deberá entregarse junto con el código servidor. Para hacer las peticiones, se aconseja hacer uso de imágenes pertenecientes a *datasets* disponibles en [Hugging Face](https://huggingface.co), como el utilizado en el Ejercicio 1.